

Component-Based Synthesis from Demonstrations under Local Specifications

Verified Robot Programs from Incomplete Contracts

ANONYMOUS AUTHOR(S)

Component-based synthesis constructs programs from reusable components summarized by specifications. Existing approaches implicitly assume that component specifications are complete. Many practical components violate this assumption. Robot manipulation primitives, for example, admit only local contracts over directly manipulated objects, leaving unspecified how execution affects the surrounding environment. Consequently, standard compositional reasoning becomes unsound. We present RoboVerify, a synthesis framework for programs built from components with intentionally incomplete specifications. Our key observation is that demonstrations implicitly reveal the missing global assumptions under which local contracts compose correctly. The system synthesizes a symbolic relational program from demonstrations, recovering control flow, relational abstractions, loop structure, inductive invariants, and frame conditions required for composition. Verification proceeds compositionally across two abstraction levels. Symbolic verification establishes functional correctness of the synthesized program over unbounded object sets, while motion-level verification proves that every component invocation satisfies the frame conditions required for sound composition. Together these proofs establish end-to-end correctness despite components exposing only local specifications. We evaluate our approach on challenging robot manipulation tasks over dynamically sized environments. Our system synthesizes verified programs that substantially outperform existing synthesis baselines while providing formal correctness guarantees.

1 Introduction

Component-based synthesis has become a central paradigm in programming languages because it enables scalable synthesis through abstraction. Rather than searching directly over complete programs, a synthesizer constructs programs from reusable components whose behavior is summarized by specifications [12, 17, 21, 36]. Whether the components are library functions, APIs, or DSL operators, synthesis proceeds entirely over these specifications, and correctness follows from compositional reasoning: the guarantee established by one component provides exactly the assumptions required by the next. Recent work has extended this paradigm to components with stateful behavior, synthesizing heap-manipulating programs from separation-logic specifications [33] and abstract data types by refinement in a proof assistant [8]. Across these settings, component-based synthesis relies on a common premise: *component specifications contain sufficient information to support sound composition*.

This paper studies components whose specifications are *intentionally local*, for which this premise fundamentally fails: each local contract states the relational effects on directly referenced objects and deliberately omits *frame conditions*—the guarantees that unrelated relations are preserved. Standard Hoare-style composition is consequently insufficient: sound composition requires that a component not affect what its contract does not mention, but intentionally local specifications provide no such guarantee. Robot manipulation primitives are a canonical instance. A primitive such as *pick*, *move*, or *release* naturally admits a concise local contract—*pick* establishes that the robot arm holds object *b*, *move* updates the arm’s position—but the primitive simultaneously executes a continuous trajectory through a shared physical workspace. While satisfying its stated postcondition, it may collide with unrelated objects and destroy previously established relations. For example, transporting one object can displace nearby objects, violating global assumptions required by subsequent components—a situation commonly encountered in tabletop manipulation, warehouse automation, and household robotics. Restoring compositionality—recovering the frame

conditions under which these local contracts compose soundly—is therefore itself part of the synthesis problem, and one that demands verification.

Our key observation is that task demonstrations implicitly reveal exactly these missing global assumptions. Rather than viewing demonstrations merely as examples of desired behavior [1], we treat them as describing *how locally specified components are intended to be composed*. Local contracts specify *what* each primitive accomplishes in isolation; demonstrations reveal *how* primitives interact to accomplish a task. Across demonstrations, recurring relational changes expose stable global object organization, repeated control-flow structure, loop-carried dependencies, and inductive relational invariants that no local primitive contract can express. In this sense, program demonstrations recover the global relational context absent from local specifications, complementing rather than replacing component contracts:

Local Contracts + Demonstrations $\Rightarrow$ Composable Specifications.

This perspective underlies the design of RoboVerify. The system synthesizes a symbolic relational program whose basic blocks are abstract actions summarized solely by their relational effects. Local contracts provide the semantics of individual components, while demonstrations are analyzed to recover the global relational structure required for their sound composition. Newly established relations induce refinements of symbolic states; repeated relational transformations are quotiented modulo variable substitutions to recover loop bodies, loop-carried variables, and loop updates; divergent relational transformations induce guarded branches. A Kleene representation of relational control-flow fragments further extends this quotienting mechanism from repeated transitions to repeated subgraphs, enabling the synthesis of unbounded, nested loops from finite demonstrations. Each basic block within the symbolic program is then realized as a concrete straight-line implementation by searching over primitive action sequences, with their continuous parameters tuned to match the demonstration segment assigned to that block and to satisfy its relational postcondition.

The verification architecture mirrors this separation between local and global reasoning. Symbolic verification reasons entirely over relational summaries of synthesized components. Each basic block is treated as an abstract component that realizes its contracted relational effect while preserving every unrelated relation required by subsequent symbolic reasoning. These preservation assumptions cannot themselves be established symbolically, because they concern the concrete execution underlying each component rather than its abstract relational effect. Low-level verification discharges *exactly these assumptions* against the synthesized straight-line implementations. The two verification layers therefore form a single compositional proof. A central insight is that the same relational invariants recovered from demonstrations serve both verification layers simultaneously. Symbolically, they establish inductive correctness of the synthesized program through weakest-precondition reasoning. Geometrically, they translate into global constraints describing the surrounding environment, enabling collision freedom, frame preservation, and realization of each basic block’s relational contract. Demonstrations therefore contribute not only executable program structure, but also the global proof context that local contracts fundamentally lack.

1.1 Contributions

This paper makes the following contributions:

- We introduce a synthesis framework that constructs verified programs from components exposing only local contracts. Rather than assuming compositional specifications, the framework recovers the missing assumptions required for sound composition directly from demonstrations.

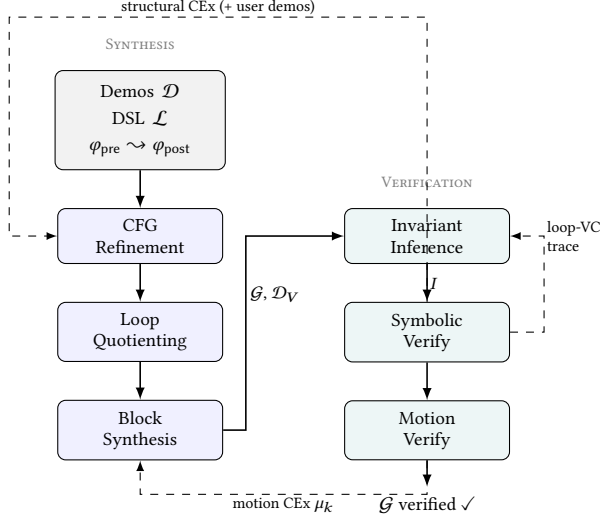


Fig. 1. The RoboVerify synthesis and verification pipeline. Blue boxes: synthesis; teal boxes: verification. Solid arrows show the main data flow; dashed arrows show counterexample-guided refinement loops. A structural symbolic counterexample triggers user demonstrations and re-enters CFG refinement. A loop-VC counterexample adds execution traces that retrigger invariant inference. A motion counterexample μ_k accumulates in $\text{Pen}(v)$ and re-drives block synthesis.

- We present a relational synthesis algorithm that infers symbolic control flow, relational abstractions, loop structure, inductive invariants, and frame conditions from demonstrations, transforming incomplete local specifications into compositional program abstractions.
- We develop a two-level verification methodology in which symbolic reasoning establishes functional correctness over unbounded object sets, while motion-level verification proves that every component invocation satisfies the frame conditions. Synthesized relational invariants connect both reasoning layers to yield end-to-end correctness.
- We evaluate RoboVerify on a suite of challenging robot manipulation benchmarks covering tasks that require sophisticated control flow over unbounded object sets. RoboVerify synthesizes verified programs with end-to-end correctness guarantees across all benchmarks, outperforming baseline methods on every task.

2 Overview

In this section, we motivate the problem and provide an overview of our approach. We introduce RoboVerify through a running example: synthesizing a verified robot program that stacks an arbitrary finite set of blocks into a tower rooted at a designated base block b_0 . Figure 1 gives an end-to-end view of the pipeline.

2.1 Key Assumptions

Object-centric Views. RoboVerify synthesizes programs through interaction with the execution environment. In the robotics domain, each environment scene is represented as a finite set of objects $O = \{\mu, e_1, \dots, e_N\}$, where μ denotes the robot end effector and each e_i denotes an entity in the environment. Programs, specifications, component contracts, and verification conditions are expressed symbolically over these objects. In practice, object-centric scene representations can be

```

Type  $\tau ::= \{\text{EE}, \text{block}, \dots\}$       Variable  $V ::= \mu, v$ 
Predicate  $P ::= R(v_1, \dots, v_k) \mid R^*(v_1, v_2) \mid v_1 = v_2 \mid \neg P \mid P \wedge P \mid P \vee P \mid \exists v : \tau. P \mid \forall v : \tau. P$ 
Action  $c ::= \text{Pick}(v) \mid \text{Move}(v_1, v_2, v_3, \delta) \mid \text{Release}(v, h) \mid \dots$ 
Statement  $S ::= \text{while } (\exists v : \tau. P) \{S\} \mid \text{if } (P) S_1 \text{ else } S_2 \mid S_1; S_2 \mid v := \text{get}(\exists v : \tau. P) \mid c$ 
Program  $::= \text{def } \mathcal{P} (\mu : \text{EE}) : S$ 

```

Fig. 2. The DSL $\mathcal{L}$ used by RoboVerify.

obtained using standard object detection, pose estimation, or object discovery techniques [9, 26, 30, 34, 42].

Demonstrations. We assume access to a finite set of task demonstrations $\mathcal{D} = \{\tau_1, \dots, \tau_n\}$, where each demonstration τ is an *unsegmented* sequence of environment states $\tau = (s_0, s_1, \dots, s_H)$. Here, s_0 is an initial state and each state s_i represents an object-centric view of the environment at timestep i .

2.2 Program and Domain-Specific Language

RoboVerify synthesizes robot-control programs in the DSL $\mathcal{L}$ shown in Fig. 2. The distinguished variable μ denotes the robot end effector, while variables v range over environment objects of type τ . Each object has attributes, such as its 3D pose, denoted $v \downarrow_{x,y,z}$, where $\downarrow$ extracts attribute values from the object referenced by v .

Predicates. The DSL is centered around *relational predicates* representing symbolic properties of the environment. Predicate symbols R are drawn from a predefined relational vocabulary describing task-relevant object relations. For example, in the stacking domain, the predicates include: $\text{on}(b_1, b_2)$, $\text{Higher}(b_1, b_2)$, and $\text{Scattered}(b_1, b_2)$, where b_1 and b_2 denote blocks. Intuitively, $\text{on}(b_1, b_2)$ holds when block b_1 rests directly on top of block b_2 , $\text{Higher}(b_1, b_2)$ holds when b_1 is positioned above b_2 , and $\text{Scattered}(b_1, b_2)$ holds when the two blocks are sufficiently far apart. Formally, predicates are evaluated from the underlying (continuous) environments states:

$$\begin{aligned}
 \text{on}(b_1, b_2) &\equiv \|b_1 \downarrow_{x,y} - b_2 \downarrow_{x,y}\| < \phi_1 \wedge |b_1 \downarrow_z - b_2 \downarrow_z - h| < \phi_2, \\
 \text{Higher}(b_1, b_2) &\equiv b_1 \downarrow_z \geq b_2 \downarrow_z, \\
 \text{Scattered}(b_1, b_2) &\equiv \|b_1 \downarrow_{x,y} - b_2 \downarrow_{x,y}\| > \phi_3.
 \end{aligned}$$

Here, ϕ_1, ϕ_2 , and ϕ_3 are task-specific tolerance thresholds. Such predicates provide a symbolic abstraction of the robot environment on which synthesized programs operate, enabling reasoning over object relations while remaining grounded in the underlying continuous state. Predicates are closed under standard Boolean connectives and support both existential and universal quantification. Quantified predicates are particularly important for expressing loop guards, relational invariants, and task specifications over dynamically sized object sets.

Importantly, the DSL supports *transitive-closure predicates*. For a binary relation R , the predicate $R^*(x, y)$ denotes the reflexive-transitive closure of R ; formally, $R^*(x, y)$ holds iff there exists $k \geq 0$ and elements $c_0, \dots, c_k$ such that $c_0 = x$, $c_k = y$, and $R(c_i, c_{i+1})$ holds for every $0 \leq i < k$. Transitive closure enables the specification of recursive structural properties that are difficult to express using only local relations. For example, $\text{on}^*(b_1, b_2)$ states that block b_1 is transitively supported by block b_2 through zero or more applications of the on relation; that is, b_1 is stacked on b_2 either directly or through a tower of intermediate blocks.

Control Primitives. The DSL includes three motion primitives: $\text{Pick}(b)$, $\text{Move}(u, v, w, \delta)$, and $\text{Release}(b, h_r)$. Each primitive reads and updates two state variables: the arm's position $p^{\text{post}} \in \mathbb{R}^3$ and the held block $h \in \mathcal{O} \cup \{\perp\}$ ($\perp$ if nothing is held). Physical execution is noisy: each primitive assigns p^{post} as a nominal target plus a free noise perturbation. Writing $X(\cdot), Y(\cdot), Z(\cdot)$ for block coordinates, h_r for the release height, $\varepsilon_g, \varepsilon_m, \varepsilon_r \geq 0$ for hardware noise bounds, and $\eta_g, \eta_m, \eta_r \in \mathbb{R}^3$ for the corresponding perturbation vectors:

$$\begin{aligned} \text{Pick}(b) : \quad & h \leftarrow b; \quad p^{\text{post}} = (X(b), Y(b), Z(b)) + \eta_g, \quad \eta_g \in [-\varepsilon_g, \varepsilon_g]^3. \\ \text{Move}(u, v, w, \delta) : \quad & p^{\text{post}} = (X(u) + \delta_x, Y(v) + \delta_y, Z(w) + \delta_z) + \eta_m, \quad \eta_m \in [-\varepsilon_m, \varepsilon_m]^3; \\ & (X(h), Y(h), Z(h)) = p^{\text{post}}. \\ \text{Release}(b, h_r) : \quad & h \leftarrow \perp; \quad p^{\text{post}} = (X(b), Y(b), Z(b) + h_r) + \eta_r, \quad \eta_r \in [-\varepsilon_r, \varepsilon_r]^3. \end{aligned}$$

The held block tracks p^{post} exactly during Move (it is in the gripper); η_g, η_m, η_r are treated as free NRA variables in the verification checks, so the collision and contract queries automatically cover all actual trajectories within the noise bounds. The full specification—including trajectories and the conditional position update for Release —is formalized in Section 5.5.

These contracts are intentionally local: p^{post} and h reference only the objects explicitly named by the primitive. Consequently, they do not characterize the complete effect of a primitive on the environment. For example, while picking up a block, the end-effector may inadvertently collide with a nearby partially constructed tower, causing the tower to collapse and altering multiple on relations elsewhere in the scene, even though none of these effects are captured by the action's contract. As a result, standard Hoare-style sequential composition is generally unsound, since predicates omitted from one contract may be modified in ways that invalidate the assumptions of subsequent actions.

Control flow structures. Programs include sequencing, conditionals, state-conditioned loops, and object-selection statements. The statement $v := \text{get}(\exists v : \tau. P)$ binds v to an arbitrary object witnessing $\exists v : \tau. P$; if no such object exists, the statement fails. The loop form **while** $(\exists v : \tau. P) \{S\}$ selects a fresh witness at each iteration and continues as long as one exists, binding it to v throughout S ; this is the only loop form RoboVerify synthesizes. Every guard synthesized by LEARNCLASSIFIER (Section 3.2.1) has an outermost existential, so both the loop guard and the branch condition in **if** always take the form $\exists v : \tau. P$ for some quantifier-free P .

Specifications. Given predicates $\varphi_{\text{pre}}, \varphi_{\text{post}} \in P$ in our DSL, the synthesis task is to construct a program $\mathcal{P} : \varphi_{\text{pre}} \rightsquigarrow \varphi_{\text{post}}$ whose executions transform any state satisfying φ_{pre} into one satisfying φ_{post} . For example, the stacking task **def** $\mathcal{P} (\mu : \text{EE}, b_0 : \text{block})$ can be specified as

$$\left\{ \begin{array}{l} \varphi_{\text{pre}} \triangleq \forall m, n. (m = n \vee \neg n \langle \text{on}^* \rangle m) \\ \quad \wedge \forall m, n. \text{Higher}(n, m) \\ \quad \wedge \forall m, n. (m \neq n \Rightarrow \text{Scattered}(m, n)) \end{array} \right\} \quad \text{def } \mathcal{P} (\mu, b_0) \cdots \left\{ \varphi_{\text{post}} \triangleq \forall m. m \langle \text{on}^* \rangle b_0 \right\} \quad (1)$$

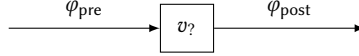
The precondition specifies that the blocks are initially scattered, unobstructed, and at the same height. The postcondition requires that every block is directly or transitively stacked on top of b_0 . RoboVerify must synthesize and verify a program whose overall behavior satisfies this global specification. The central challenge arises from the mismatch between local contracts and global correctness: correctness cannot be established by naively composing primitive contracts, since unmodeled side effects may invalidate the assumptions of subsequent actions.

2.3 Relational Control-Flow Graphs

The core intermediate representation manipulated by RoboVerify during synthesis and verification is a *relational control-flow graph* (CFG):

$$\mathcal{G} = (V, E, \varphi_{\text{pre}}, \varphi_{\text{post}}, \ell),$$

Each node in V corresponds to a basic block. Each edge $e \in E$ is labeled by a relational predicate $\ell(e) \in P$ describing the path condition under which control transfers between the corresponding blocks, where ℓ is the edge-labeling function. The distinguished predicates φ_{pre} and φ_{post} label the incoming and outgoing edges of the CFG, respectively, thereby representing the precondition and postcondition of the synthesis task. For a basic block v with a unique outgoing edge (v, v') , we write φ_v for the label $\ell(v, v')$ of that edge. RoboVerify initializes with the trivial CFG:



where $v?$ is an unknown basic block whose relational summary must establish φ_{post} . RoboVerify refines this CFG incrementally during the synthesis procedure.

2.4 Straight-line Program Synthesis

RoboVerify uses a straight-line program synthesizer to fill in each basic block in a control-flow graph. Initially, RoboVerify populates the unknown block $v?$ with a randomly generated sequence of primitive actions (a.k.a. instructions) instantiated with random parameters (e.g., target end-effector poses). Starting from this initial program, RoboVerify performs stochastic program synthesis.

The key intuition is that a desired program should drive the robot through states similar to those observed in the demonstrations. Given a candidate program $\mathcal{P}$, let $\tau_{\mathcal{P}}$ denote the states visited during its rollouts, and let τ_D denote the states appearing in the demonstrations. RoboVerify scores $\mathcal{P}$ according to the similarity between $\tau_{\mathcal{P}}$ and τ_D , measured as the KL divergence between the corresponding state distributions. Using a Markov Chain Monte Carlo (MCMC) search, the synthesizer iteratively proposes mutations to the current program by modifying an instruction, changing its operands, or reordering instructions. Following each structural mutation, the continuous parameters of the primitive controllers are optimized using the cross-entropy method to minimize the KL divergence. MCMC then preferentially accepts program mutations that improve this score, thereby biasing the search toward straight-line programs whose executions increasingly resemble the demonstrations.

```
def stack( $\mu, b_0$ ):
    Pick( $b_0$ )
    Move( $b_0, b_0, b_0, (0, 0, 0)$ )
    Release( $b_0, 0.3$ )
```

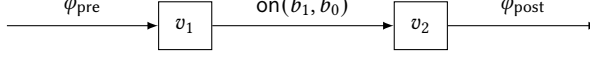
Fig. 3. An initial program synthesized for the stacking task. `Pick( $b_0$ )` grasps block b_0 . `Move( $u, v, w, \delta$ )` moves the end effector to position $(X(u) + \delta_x, Y(v) + \delta_y, Z(w) + \delta_z)$ (which leaves the end effector stationary in this example). `Release( $b, 0.3$ )` opens the gripper at height $Z(b_0) + 0.3$ above block b_0 .

2.5 Relational Refinement from Demonstrations

The resulting program in Fig. 3 does not satisfy φ_{post} . Intuitively, the task requires repeatedly extending a partially constructed tower until every block is incorporated, but the straight-line program provides no representation of this iterative progress. Consequently, synthesis lacks the intermediate assertions needed to decompose the global specification into smaller obligations. This motivates refining $v?$ in the CFG with relational predicates that capture successive stages of progress toward φ_{post} .

To this end, RoboVerify extracts the set of states in the task demonstration $\mathcal{D}$ immediately preceding successful task completion, $P_s = \{s_t \mid s_t \in \mathcal{D} \wedge s_t \not\models \varphi_{\text{post}} \wedge s_{t+1} \models \varphi_{\text{post}}\}$, and compare

them against states N_s collected from the synthesized program’s own (unsuccessful) trajectories. RoboVerify then synthesizes a state-abstraction predicate P , drawn from the grammar of our DSL $\mathcal{L}$ (Figure 2) via enumerative synthesis, that separates P_s from N_s : P holds on (almost) every state in P_s and fails on (almost) every state in N_s . Such a predicate identifies a prerequisite condition that must hold one step before φ_{post} can be reached. In this example, RoboVerify learns the predicate $\exists b_1. \text{on}(b_1, b_0)$: every state one step from success has *some* block already placed on the base b_0 , whereas unsuccessful trajectories typically do not. Treating this predicate as a new cutpoint, we refine the initial CFG by splitting the unknown block $v_?$:

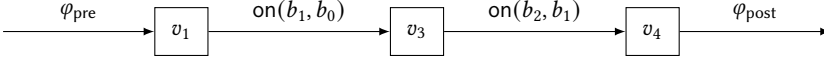


We note that all variables appearing in a learned predicate in the CFG that are not otherwise bound are implicitly existentially quantified e.g. $\text{on}(b_1, b_0)$ is shorthand for $\exists b. \text{on}(b, b_0)$.

This single refinement step effectively decomposes the original task into two subtasks: (1) reach a state satisfying $\exists b_1. \text{on}(b_1, b_0)$ from φ_{pre} , realized by a basic block v_1 ; and (2) reach φ_{post} from a state already satisfying $\text{on}(b_1, b_0)$ in a basic block v_2 . Each basic block represents a local synthesis problem. For v_1 , the straight-line synthesizer (Section 2.4) can easily fill in v_1 with a simple program:

$b_1 := \text{get}(\exists b : \text{block. true}); \text{Pick}(b_1); \text{Move}(b_1, b_1, b_0, \dots);$

If a basic block cannot be synthesized by the straight-line synthesizer, P_s and N_s are recomputed relative to its current specification, and a new splitting predicate is synthesized to further refine the corresponding CFG region. For example, for v_2 —reaching φ_{post} from a state already satisfying $\text{on}(b_1, b_0)$ —RoboVerify repeats the same procedure: applying refinement to v_2 synthesizes the predicate $\exists b_2. \text{on}(b_2, b_1)$. This predicate captures the next unit of progress to extend the partial tower by placing another block on its current top. This yields the further-refined CFG:



Demonstration Partitioning. The refinement predicates simultaneously induce a partition of the demonstrations. Given a refinement predicate ψ , each demonstration is split at the first state satisfying ψ , yielding trajectory segments corresponding to the predecessor and successor CFG nodes connected by ψ . For example, the predicate $\text{on}(b_1, b_0)$ partitions demonstrations into segments associated with v_1 and v_2 , while $\text{on}(b_2, b_1)$ further partitions the latter segment into those associated with v_2 and v_3 . Consequently, CFG refinement and demonstration partitioning proceed in lockstep: each newly introduced predicate simultaneously refines the CFG and the trajectory decomposition used for block-level synthesis.

2.6 Loop Structure Inference from Demonstrations

Demonstrations are finite, yet a synthesized program may need to handle an arbitrary number of objects. The key challenge is to generalize a finite acyclic CFG path—extracted from demonstrations—into a loop. We achieve this by identifying when consecutive CFG edges are *relationally equivalent*: they differ only in which concrete variables play which roles. When such equivalence holds, the acyclic path can be collapsed into a loop.

For example, after refining v_2 into v_3 and v_4 , but **before** synthesizing implementations for either block, RoboVerify observes that the newly introduced relational summaries, $\text{on}(b_1, b_0)$ and $\text{on}(b_2, b_1)$, induce the same relational transformer modulo variable renaming: in both cases, one block is placed on top of another. Formally, both are instances of the *relational template* $\text{on}(b', b)$

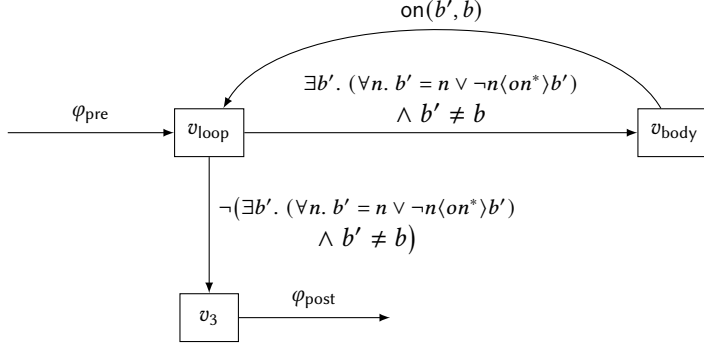


Fig. 4. Synthesized relational CFG for the Stack task. v_{pre} is the entry node and v_{post} is the exit node; their outgoing edges carry φ_{pre} and φ_{post} , respectively. v_{loop} is the loop head (initializing $b \leftarrow b_0$ on entry) and v_{body} executes one stacking iteration. The back-edge from v_{body} to v_{loop} is labeled by the relational summary $on(b', b)$ established by each iteration.

under the substitutions:

$$\begin{aligned} \sigma_1 &= \{b \mapsto b_0, b' \mapsto b_1\} & \text{so that } on(b', b)\sigma_1 &= on(b_1, b_0), \\ \sigma_2 &= \{b \mapsto b_1, b' \mapsto b_2\} & \text{so that } on(b', b)\sigma_2 &= on(b_2, b_1). \end{aligned}$$

RoboVerify *quotients* the CFG fragment modulo relational equivalence, collapsing the repeated path into the loop CFG of Figure 4, where the back-edge from v_{body} to v_{loop} captures the repeated relational transformer $on(b', b)$; the exit edge to v_{post} is guarded by the negation of the loop guard.

Importantly, such loop formation requires synthesizing the loop-carried variables, loop guard, and loop body. RoboVerify performs this synthesis in three steps: (1) *Relational Quotienting*, which extracts loop-carried variables and updates from the transformer; (2) *Loop Guard Synthesis*, which synthesizes a relational query that both determines loop continuation and binds the local variables required by the next iteration; and (3) *Loop Body Synthesis*, which synthesizes an implementation of the repeated transformer from the recovered loop iterations.

(1) Relational Quotienting. The initialization and update for loop-carried variables can be read from the substitutions witnessing relational equivalence between repeated relational transformers.

Reading Off the Loop-Carried Update. In our example, among the template variables b and b' appearing in the repeated relational transformer $on(b', b)$, which should become loop-carried variables? Should it be b , b' , or both? The key observation is that only those whose values persist across iterations become loop-carried variables. A template variable x is loop-carried when the value assigned to some variable in σ_i must be reused as the value of x under σ_{i+1} . The corresponding loop update is obtained directly from this relationship. Principally, at the end of each iteration, the template variables must be updated to reflect the roles they will play in the *next* iteration. Concretely: after iteration i , template variable x holds the value $\sigma_i(x)$. In iteration $i+1$, the same variable x should hold $\sigma_{i+1}(x)$. We therefore need an assignment that maps $\sigma_i(x)$ to $\sigma_{i+1}(x)$. Expressed in terms of template variables, this assignment is:

$$x \leftarrow \sigma_i^{-1}(\sigma_{i+1}(x)).$$

Intuitively, $\sigma_i^{-1} \circ \sigma_{i+1}$ is the *transition* between consecutive iterations, expressed as a map on template variables. Applied to the block example, for the template variable b :

$$b \leftarrow \sigma_1^{-1}(\sigma_2(b)) = \sigma_1^{-1}(b_1) = b'.$$

Consequently, RoboVerify synthesizes the loop-carried update $b \leftarrow b'$, which is appended to the end of the loop body. In contrast, b' is not a loop-carried variable. $b' \leftarrow \sigma_1^{-1}(\sigma_2(b'))$ is undefined because $\sigma_2(b')$ refers to b_2 that is not mapped by σ_1 . Thus no value from iteration i determines the value of b' in iteration $i + 1$. Instead, b' is re-instantiated in each iteration as a local variable.

Reading Off the Loop Initialization. The substitution σ_1 describes the concrete role each template variable plays in the *first* iteration. The loop initialization is therefore obtained by emitting, for each template variable x , the assignment $x \leftarrow \sigma_1(x)$:

$$b \leftarrow \sigma_1(b) = b_0.$$

The variable b' is not initialized here because it is locally bound within the loop body.

Reading Off Loop Iterations. Having identified the repeated relational transformer $\text{on}(b', b)$ and the loop-carried update $b \leftarrow b'$, RoboVerify can recover loop iterations directly from the demonstrations. This partitioning yields a collection of trajectory segments, each corresponding to one execution of the loop body, which are then used collectively to synthesize the loop body. Intuitively, the transformer $\text{on}(b', b)$ identifies the progress achieved by a single iteration—placing a new block b' on the current tower top b —while the update $b \leftarrow b'$ identifies the tower top used by the subsequent iteration. Starting from the first occurrence of $\text{on}(b', b)$, RoboVerify repeatedly applies this transformer-and-update rule to trace the evolution of the loop-carried variable through the demonstration. Each application identifies the state at which an iteration begins and the state at which it establishes the next $\text{on}(b', b)$ relation, thereby partitioning the trajectory into loop-body executions.

(2) Loop Guard Synthesis. The iteration-tracing procedure described above not only partitions each demonstration into loop-body segments but also records, for each iteration, the state at which it begins and the object selected at that iteration. These loop-entry states serve directly as examples for synthesizing the guard: the guard must accept each demonstrated object choice and reject the alternatives present at the same state. RoboVerify learns a per-state predicate by enumerative synthesis over a vocabulary of relational formulas and conjoins the results, so that informative states impose constraints while uninformative ones contribute nothing. For the Stack task this yields

$$G(b, b') \equiv \left(\forall n. b' = n \vee \neg \langle \text{on}^* \rangle b' \right) \wedge b' \neq b, \quad (2)$$

selecting a top block not yet in the tower as the next block to place. The full construction is given in Section 3.3.

(3) Loop Body Synthesis. By inferring a CFG skeleton from demonstrations, RoboVerify reduces whole-program synthesis to synthesizing the contents of individual basic blocks. Fig. 5(a) shows the program sketch induced by the CFG in Fig. 4, including the synthesized loop guard and updates to the loop-carried variable. The loop body is summarized as a basic block $\pi_{\text{on}(b', b)}(\mu, b', b)$, whose intended effect is to establish $\text{on}(b', b)$. Recall that during CFG refinement, through demonstration partitioning and relational quotienting, each basic block is associated with a collection of demonstration segments. Each segment captures the behavior required to drive execution from the block entry to the block exit. To synthesize a basic block, RoboVerify then searches for a straight-line program whose executions induce state trajectories that closely match those observed in the corresponding demonstration segment (as illustrated in Sec. 2.4). At first glance, implementing $\pi_{\text{on}(b', b)}$, the loop body in Fig. 5(b), may appear straightforward: simply pick up b' , directly move it above b , and release it (in 3 steps). However, such a naive implementation may collide with or destabilize a partially constructed tower, causing previously established on relations to be violated. Guided by the demonstration segments associated with $\pi_{\text{on}(b', b)}$, the straight-line synthesizer learns a

```

442 def stack( $\mu$ ,  $b_0$ ):
443    $b \leftarrow b_0$ 
444   while  $\exists b'$ .
445     ( $\forall n. n = b' \vee \neg n \langle on^* \rangle b'$ )
446        $\wedge b' \neq b$ :
447      $\pi(\mu, b', b)[on(b', b)]$ 
448      $b \leftarrow b'$ 

```

(a) Loop sketch with unknown block $\pi(\mu, b', b)$

```

442 def  $\pi(\mu, b', b)$ :
443   Move( $b', b', b', (0, 0, 0, 0.15)$ )
444   Pick( $b'$ )
445   Move( $b', b', b, (0, 0, 0, 0.15)$ )
446   Move( $b, b, b, (0.0, 0.0, 0.15)$ )
447   Move( $b, b, b, (0.0, 0.0, 0.05)$ )
448   Release( $b', 0.15$ )

```

(b) Synthesized straight-line program for $\pi(\mu, b', b)$

Fig. 5. Hierarchical synthesis of the stacking program. For exposition, we separately display (a) the CFG-level program sketch inferred by RoboVerify and (b) the synthesized implementation of the symbolic block $\pi_{on(b', b)}$. The complete program is obtained by inlining the implementation in (b) at the call site in (a).

sequence of primitive actions that move the end effector to (1) approach b' from above, (2) grasp b' , (3) lift b' to a safe transport height, (4) transport b' above the current tower top b , (5) lower b' into position, and (6) release it. The complete synthesized program is shown in Fig. 5.

2.7 Program Verification

Synthesizing the stack program in Fig. 5 is only half the problem: we must also prove that it is *correct* with respect to its specification (the pre/postcondition in Eq. 1). Correctness must be established across two levels of abstraction: the symbolic level, where the loop in Figure 5(a) must satisfy the postcondition for an unbounded number of blocks, and the motion level, where the synthesized implementation of $\pi(\mu, b', b)$ for the loop body in Fig. 5(b) must realize the symbolic relational update $on(b', b)$ through continuous robot motions, where b is the current top block of the tower. Each level can fail for a different reason. At the *symbolic* level, a symbolic proof for the loop has no way to rule out side effects that π silently disturbs other relations on which the rest of the tower under b depends. At the *motion* level, $\pi(\mu, b', b)$ in Figure 5(b) is a few lines of concrete geometric motion to establish $on(b', b)$; the end effector can sweep through space occupied by any blocks π 's specification never references, e.g., the partially built tower. A sound correctness argument must close both gaps at once: the symbolic proof needs a guarantee about relational scope that only the end effector's geometry can supply, and the motion proof needs a geometric description of the environment that only the symbolic loop program can supply.

RoboVerify verifies the two levels *separately*, against one shared artifact: a loop invariant I_{stack} (Figure 6) *learned* from execution traces of the synthesized program (Section 4). At the *symbolic* level, the proof for (a) treats $\pi(\mu, b', b)$ as a black box trusted to do exactly one thing: establish $on(b', b)$ while leaving other relations not involving b' and b in the tower untouched. For example, RoboVerify generates the following verification condition to prove loop invariant preservation based on weakest precondition generation (wp):

$$I_{\text{stack}} \Rightarrow \forall b'. G(b, b') \Rightarrow \text{wp}(\pi(\mu, b', b)[on(b', b)]; b \leftarrow b', I_{\text{stack}}),$$

where $G(b, b')$ is the loop guard requiring b' to be a top block and $b' \neq b$. For the weakest precondition of the abstract action $\pi(\mu, b', b)$, RoboVerify symbolically updates the transitive support relation. Intuitively, after placing b' on b , any block that was transitively above b' also becomes transitively above every block that was below b . Formally, for every transitive-reachability atom $x \langle on^* \rangle y$ appearing in the invariant (where x and y are arbitrary block variables), RoboVerify applies the rewrite:

$$\text{wp}(\pi(\mu, b', b)[on(b', b)], I'_{\text{stack}}) = I'_{\text{stack}}[x \langle on^* \rangle y \vee (x \langle on^* \rangle b' \wedge b \langle on^* \rangle y) / x \langle on^* \rangle y]$$

where $I'_{\text{stack}} = I_{\text{stack}}[b/b]$. RoboVerify applies this rewrite structurally to all $\langle on^* \rangle$ atoms in the invariant, together with analogous rewrites for Higher and Scattered, and discharges the resulting verification conditions with an SMT solver under the domain axioms.

$$\begin{aligned}
\text{inv}(u, v) &\triangleq (\text{Higher}(u, v) \Rightarrow u\langle \text{on}^* \rangle v \vee \text{Scattered}(u, v)) \\
&\quad \wedge (u\langle \text{on}^* \rangle b_0 \Leftrightarrow b\langle \text{on}^* \rangle u) \\
&\quad \wedge (\text{Higher}(b_0, v) \Rightarrow \text{Higher}(u, v)) \\
&\quad \wedge (\text{Higher}(u, v) \Rightarrow u\langle \text{on}^* \rangle v \vee \text{Higher}(b_0, v)) \\
&\quad \wedge (\text{Higher}(b_0, u) \vee u\langle \text{on}^* \rangle b_0) \\
I_{\text{stack}} &\triangleq \forall u, v. \text{inv}(u, v)
\end{aligned}$$

Fig. 6. Synthesized loop invariant for the stacking program in Figure 5(a), where u, v are universally quantified variables. The second conjunct says a block u belongs to the tower rooted at b_0 exactly when it lies at or below the frontier b ; the remaining conjuncts relate $\langle \text{on}^* \rangle$ to the geometric predicates Higher and Scattered, which is what lets I_{stack} also serve as a geometric assumption at the motion level.

The trust on $\pi(\mu, b', b)[\text{on}(b', b)]$ faithfully realizing the specified abstract state update is an assumption, not something the symbolic proof can check on its own—it has no access to what π actually does in continuous space. Discharging that assumption is exactly the job of the *motion* level. Since I_{stack} universally quantifies over all blocks in the environment, RoboVerify translates it into geometric constraints describing the locations of each block, including those not explicitly manipulated by $\pi(\mu, b', b)$. These constraints are used to automatically verify two properties of the synthesized implementation. First, no block in the partially constructed tower rooted at b_0 is disturbed by the execution of $\pi(\mu, b', b)$. Second, the implementation must satisfy its own local contract; that is, executing $\pi(\mu, b', b)$ must indeed establish $\text{on}(b', b)$. The same invariant drives both checks. It must therefore be strong enough to close the symbolic proof of the loop and precise enough to justify, at the motion level, the assumptions on which that proof relies. If either motion-level verification conditions fails, RoboVerify rejects the candidate implementation and continues synthesis until a verifiable implementation of $\pi(\mu, b', b)$ is found. Section 5 presents both verification procedures in detail.

3 Relational CFG-Guided Robot Control Program Synthesis

We present the core algorithms of RoboVerify’s CFG-guided synthesis strategy. We first give the top-level synthesis algorithm, then describe its two refinement mechanisms—node splitting (which proceeds in lockstep with demonstration partitioning) and loop quotienting—followed by straight-line program realization.

3.1 Top-Level Algorithm

The top-level RoboVerify algorithm is presented in Algorithm 1. It takes as input an entity-factored Markov decision process (EFMDP) $\mathcal{M} = \{\Lambda, \mathcal{O}, \mathcal{S}, \mathcal{T}, \mathcal{A}, \mathbb{P}, \eta\}$ modeling the robot environment of Section 2.1—of its components, only the typing function Λ , which assigns each environment entity its object type, plays a direct role in what follows—together with the DSL $\mathcal{L}$ (Fig. 2), a set of task demonstrations $\mathcal{D}$, and a task specification $\varphi_{\text{pre}} \leadsto \varphi_{\text{post}}$. The objective is to synthesize a program $\mathcal{G}^*$ in $\mathcal{L}$ satisfying the

Algorithm 1 The RoboVerify Procedure

- 1: **procedure** ROBOVERIFY($\mathcal{M}, \mathcal{L}, \mathcal{D}, \varphi_{\text{pre}} \leadsto \varphi_{\text{post}}$)
 - 2: $\mathcal{G} \leftarrow (V = \{v_\tau\}, E = \{\varphi_{\text{pre}} \rightarrow v_\tau \rightarrow \varphi_{\text{post}}\})$
 - 3: $\mathcal{D}_V \leftarrow \{v_\tau \mapsto \mathcal{D}\} \triangleright$ every demonstration belongs to v_τ initially
 - 4: $\mathcal{G}^* \leftarrow \text{SYNTHESIZE}(\mathcal{M}, \mathcal{L}, \mathcal{G}, \mathcal{D}_V)$
 - 5: **return** $\mathcal{G}^*$
-

Algorithm 2 The Main Synthesis Procedure

```

1: procedure SYNTHESIZE( $\mathcal{M}, \mathcal{L}, \mathcal{G}, \mathcal{D}_V$ )
2:    $\mathcal{G}, \mathcal{D}_V \leftarrow \text{QUOTIENT}(\mathcal{G}, \mathcal{D}_V)$ 
3:    $v_f \leftarrow \perp$ 
4:   for each block  $v$  of  $\mathcal{G}$ , in topological order do
5:      $\pi_v, \text{ok} \leftarrow \text{STRAIGHTLINESYNTHESIZE}(\mathcal{D}_V(v), \varphi_v)$   $\triangleright \varphi_v$  is the label of  $v$ 's unique
      outgoing edge; Sec. 3.3
6:     if  $\neg \text{ok}$  then  $v_f \leftarrow v$ ; break
7:     Store  $\pi_v$  in  $\mathcal{G}(v)$ 
8:     if  $v_f = \perp$  then
9:       return  $\mathcal{G}$ 
10:    else
11:       $P_s \leftarrow \{s_t \mid s_t \not\models \varphi_{v_f} \wedge s_{t+1} \models \varphi_{v_f} \wedge \{s_t, s_{t+1}\} \in \mathcal{D}_V(v_f)\}$ 
12:       $N_s \leftarrow \{\zeta \sim \text{Exec}(\mathcal{M}, \mathcal{G})\}$ 
13:       $\varphi \leftarrow \text{LEARNCLASSIFIER}(P_s, N_s, \mathcal{L})$ 
14:       $\mathcal{G}, \mathcal{D}_V \leftarrow \text{REFINECFG}(\mathcal{G}, \mathcal{D}_V, v_f, \varphi)$ 
15:      if  $\neg \text{VALIDATE}(\mathcal{G}, \mathcal{D}_V)$  then
16:        return FAIL
17:    return SYNTHESIZE( $\mathcal{M}, \mathcal{L}, \mathcal{G}, \mathcal{D}_V$ )

```

specification: rollouts of $\mathcal{G}^*$ in $\mathcal{M}$ from states satisfying φ_{pre} terminate in states satisfying φ_{post} . The sound guarantee is established subsequently by the verification of Section 5.

Relational Control-Flow Graphs. RoboVerify maintains state and action abstraction of the robot environment directly as a relational CFG $\mathcal{G} = (V, E, \varphi_{\text{pre}}, \varphi_{\text{post}}, \ell)$ (Section 2.3)—a symbolic program skeleton whose nodes are basic blocks and whose edges, labeled by the function ℓ , are relational predicates describing the path condition under which control transfers between them. Crucially, RoboVerify also maintains a *demonstration assignment* $\mathcal{D}_V : V \rightarrow 2^{\mathcal{D}}$, mapping every block to the demonstration segments relevant to realizing it. At line 3, every demonstration in $\mathcal{D}$ is initially assigned to the single unknown block $v_?$, since nothing has yet been partitioned. At line 2, RoboVerify creates the trivial CFG with this one unknown block, whose incoming and outgoing edges carry φ_{pre} and φ_{post} respectively (Section 2.3). At line 4, SYNTHESIZE (Algorithm 2) iteratively refines $\mathcal{G}$ and $\mathcal{D}_V$ together into a CFG of basic blocks and loops, each backed by its own demonstration segment, continuing until a valid task-solving program is obtained.

3.2 The Main Synthesis Procedure

Algorithm 2 gives the full procedure. For each block v of $\mathcal{G}$, SYNTHESIZE calls STRAIGHTLINESYNTHESIZE on $\mathcal{D}_V(v)$ with v 's exit condition (Sec. 3.4); loop bodies are wrapped as **while**-loops by the CFG structure produced by QUOTIENT (Sec. 3.3).

When all blocks are realized ($v_f = \perp$), SYNTHESIZE returns $\mathcal{G}$ as the solution; the synthesized program is read off $\mathcal{G}$ directly. Otherwise, straight-line synthesis did not converge for some block v_f (Section 2.4): RoboVerify compares states just before φ_{v_f} is reached *within* v_f 's own assigned segment $\mathcal{D}_V(v_f)$, $P_s = \{s_t \mid s_t \not\models \varphi_{v_f} \wedge s_{t+1} \models \varphi_{v_f}\}$, against states N_s collected by running $\mathcal{G}$ in $\mathcal{M}$ (the blocks realized so far drive the execution until it fails at v_f), and synthesizes a predicate φ separating P_s from N_s —a prerequisite condition for reaching φ_{v_f} . REFINECFG then splits v_f using φ and *simultaneously partitions* $\mathcal{D}_V(v_f)$ into the two segments needed by the two new blocks (Section 3.2.2); the recursive call inherits the refined assignment $\mathcal{D}'_V$, so every subsequent realization attempt is scored only

Algorithm 3 $\text{REFINECFG}(\mathcal{G}, \mathcal{D}_V, v_0, \varphi)$: split a basic block v_0 of $\mathcal{G}$ on φ , partitioning its demonstration segment in lockstep

```

1: procedure  $\text{REFINECFG}(\mathcal{G}, \mathcal{D}_V, v_0, \varphi)$ 
2:   Let  $v_{\text{in}} \xrightarrow{\varphi_{\text{in}}} v_0 \xrightarrow{\varphi_{v_0}} v_{\text{out}}$  be the unique path through  $v_0$  in  $\mathcal{G}$ 
3:    $V \leftarrow V \setminus \{v_0\} \cup \{v_1, v_2\}$  ▷  $v_1, v_2$  fresh
4:    $E \leftarrow (E \setminus \{(v_{\text{in}}, v_0), (v_0, v_{\text{out}})\}) \cup \{v_{\text{in}} \xrightarrow{\varphi_{\text{in}}} v_1, v_1 \xrightarrow{\varphi} v_2, v_2 \xrightarrow{\varphi_{v_0}} v_{\text{out}}\}$ 
5:   for all  $d \in \mathcal{D}_V(v_0)$  do
6:     Let  $t^*$  be the first timestep in  $d$  with  $d_{t^*} \models \varphi$ 
7:      $\mathcal{D}_V(v_1) \leftarrow \mathcal{D}_V(v_1) \cup \{d_{0:t^*}\}$  ▷ prefix segment, up to  $\varphi$ 
8:      $\mathcal{D}_V(v_2) \leftarrow \mathcal{D}_V(v_2) \cup \{d_{t^*:\cdot}\}$  ▷ suffix segment, from  $\varphi$  onward
9:    $\mathcal{D}_V \leftarrow \mathcal{D}_V \setminus \{v_0 \mapsto \cdot\}$ 
10:  return  $\mathcal{G}, \mathcal{D}_V$ 

```

against the demonstration data relevant to the block being synthesized. We hypothesize that each new block, backed by a more specific and typically shorter segment, is easier to realize than v_f was on its own. This embodies a recursive decomposition that repeatedly splits a single unrealized block—and its demonstration segment—until every block in $\mathcal{G}$ can be directly realized. We defer termination, including `VALIDATE` at line 15, to Sec. 3.5.

3.2.1 Learning Refinement Predicates. At line 13, `SYNTHESIZE` invokes `LEARNCLASSIFIER` to learn the predicate φ used to split v_f .

In-scope variables. For a block v in $\mathcal{G}$, define

$$\text{scope}(v) = \text{free}(\varphi_{\text{pre}}) \cup \{x \mid \text{every path from } \varphi_{\text{pre}} \text{ to } v \text{ traverses some edge } e \text{ with } x \in \text{bv}(\ell(e))\},$$

where $\text{free}(\varphi_{\text{pre}})$ is the set of program inputs and $\text{bv}(\varphi)$ denotes the existentially bound variables of φ . This is the standard all-paths (must-reach) dataflow fixed point over edge bindings: a variable is in scope only if it is definitely introduced before v regardless of which control-flow path was taken. For a chain CFG every edge dominates all successors, so scope accumulates monotonically. For a loop with guard edge $(v_{\text{head}}, v_{\text{body}})$, the guard's bound variable is in scope inside the body but not at the loop exit, because the exit path bypasses the guard edge. For a branch, a variable introduced in one arm only is not in scope at the join point. In the Stack refinement example (Section 2.5), $\text{scope}(v_3) = \{b_0, b_1\}$: $b_0 \in \text{free}(\varphi_{\text{pre}})$ and every path to v_3 traverses the edge labeled $\text{on}(b_1, b_0)$, so $b_1 \in \text{bv}(\text{on}(b_1, b_0))$ enters scope.

Hypothesis class and enumeration. `LEARNCLASSIFIER` enumerates formulas of the form

$$\varphi = \exists x_1, \dots, x_m. \varphi_{\text{QF}}(x_1, \dots, x_m, \text{scope}(v_f)), \quad m \geq 0,$$

where $x_1, \dots, x_m$ are fresh object variables and φ_{QF} is a Boolean combination of atomic predicates from $\mathcal{L}$ (Fig. 2). Bottom-up enumeration sweeps over increasing formula depth and number of fresh variables m , and returns the first φ separating P_s from N_s (φ holds on (almost) every state in P_s and fails on (almost) every state in N_s), as illustrated in Section 2.5. In the Stack refinement example (Section 2.5), the returned formula $\exists b_2. \text{on}(b_2, b_1)$ uses $b_1 \in \text{scope}(v_3)$ as a free variable and introduces fresh b_2 .

3.2.2 Refining Relational CFGs by Node Splitting. `REFINECFG` (Algorithm 7) is exactly the node split shown in Section 2.3: v_0 is replaced by two fresh blocks v_1 and v_2 connected by a new edge labeled φ , with v_0 's former incoming and outgoing edge labels reattached to v_1 and v_2 unchanged.

This is precisely the transformation depicted in Section 2.5: splitting the unknown block $v_?$ on $\text{on}(b_1, b_0)$ yields $\varphi_{\text{pre}} \rightarrow v_1 \xrightarrow{\text{on}(b_1, b_0)} v_2 \rightarrow \varphi_{\text{post}}$.

Demonstration Partitioning is now made explicit at lines 5–8, implementing the mechanism described in Section 2.5: every demonstration segment d previously assigned to v_0 is split at the first timestep t^* satisfying the new predicate φ into a prefix $d_{0:t^*}$, assigned to v_1 , and a suffix d_{t^*} , assigned to v_2 . CFG refinement and demonstration partitioning therefore proceed in lockstep within a single procedure, not as two separately-maintained artifacts: $\mathcal{D}_V$ is updated at exactly the same step that $\mathcal{G}$ is, using exactly the predicate φ that triggered the split. For example, $\text{on}(b_1, b_0)$ partitions every demonstration assigned to $v_?$ into the segment before any block lands on b_0 (assigned to v_1) and the segment after (assigned to v_2); a further split of v_2 on $\text{on}(b_2, b_1)$ partitions $\mathcal{D}_V(v_2)$ again, in exactly the same way.

3.3 Loop Quotienting

Repeated node splitting alone only ever produces acyclic chains, yet a synthesized program may need to handle an arbitrary number of objects. RoboVerify obtains loops not through a separate case inside `REFINECFG`, but through a distinct, subsequent procedure, `QUOTIENT`, invoked once per synthesis attempt (Algorithm 2, line 2) *before* realization is attempted. The same substitution-matching mechanism handles both *flat* loops—where the repeated unit is a single atomic edge—and *nested* loops—where the repeated unit is an entire loop-containing sub-program. In both cases, `QUOTIENT` (i) encodes consecutive CFG fragments as words over a Kleene algebra, (ii) checks whether those words are instances of a common template under a substitution, and (iii) merges matching fragments into a cyclic CFG, from which initialization and update are read off the substitution arithmetic. Iterating this procedure—first over atomic (length-1) words, then over the compound words produced by the inner loops just created—handles arbitrary nesting depth.

Kleene encoding. The encoding $\llbracket \cdot \rrbracket$ maps a CFG fragment to a word over an alphabet whose *letters* are relational formulas. A basic block contributes an atomic relational fact; a loop contributes guard formulas and a Kleene-star subword. An argument appearing in a letter is marked $\underline{x}$ exactly when it is *freshly bound* at that point: obtained by a preceding $v \leftarrow \text{get}(\exists v. G(v))$ (for a basic block), or the outermost existential witness of the loop guard (for a while loop). Arguments already in scope—program parameters and loop-carried variables—are left unmarked even on their first appearance. Variable assignments ($x \leftarrow e$) are erased entirely. The rules are:

$$\llbracket \pi_\varphi \rrbracket = \varphi \quad (\text{arguments freshly bound by a preceding get underlined})$$

$$\llbracket S_1; S_2 \rrbracket = \llbracket S_1 \rrbracket \cdot \llbracket S_2 \rrbracket$$

$$\llbracket \text{while}^I (\exists x. g(x)) \text{ do } S \rrbracket = (g(\underline{x}) \cdot \llbracket S \rrbracket)^* \cdot \neg(\exists x. g(x))$$

In the while rule, $g(x)$ is the guard formula with x as its outermost existential witness; in the body $(g(\underline{x}) \cdot \llbracket S \rrbracket)^*$ the witness is underlined because each iteration freshly binds it, while in the exit condition no fresh binding occurs so x is unmarked. The remaining free variables of g —those already in scope—simply appear unmarked inside the formula-letter; no separate listing is needed.

Internal binders. A guard formula may contain its own quantifiers, such as the $\forall n$ in $(\forall n. n = b' \vee \neg n \langle \text{on}^* \rangle b') \wedge b' \neq b$. The variable n is bound *within* the formula-letter $g(b')$; it does not occupy a word-level position, is not underlined, and is not part of the substitution σ . When two guard-letters are compared for template matching, they are compared as formulas up to alpha-equivalence of their internal binders: $g_1(b')$ and $g_2(b'')$ match the template $g(\underline{x}')$ if they become equal after substituting $b' \mapsto x'$ and $b'' \mapsto x'$ and renaming internal bound variables consistently.

Algorithm 4 QUOTIENT($\mathcal{G}, \mathcal{D}_V$): collapse relationally equivalent consecutive fragments into loops at each nesting level, then recurse into loop bodies

```

1: procedure QUOTIENT( $\mathcal{G}, \mathcal{D}_V$ )                                ▶ Phase 1: quotient the current level
2:    $(u_1, \dots, u_k) \leftarrow$  top-level nodes of  $\mathcal{G}$  in order, with each existing loop treated as a single
   atomic unit
3:   repeat
4:      $found \leftarrow \text{false}$ 
5:     for  $l \leftarrow 1$  to  $\lfloor k/2 \rfloor$  do
6:       for  $i \leftarrow 1$  to  $k - 2l + 1$  do
7:         if  $\exists W(\vec{x}), \sigma_1, \sigma_2$  s.t.  $\llbracket u_i \cdots u_{i+l-1} \rrbracket = W(\vec{x})\sigma_1$  and  $\llbracket u_{i+l} \cdots u_{i+2l-1} \rrbracket = W(\vec{x})\sigma_2$ 
           then
8:            $x \leftarrow \sigma_1(x)$  for each loop-carried  $x$                                 ▶ initialization
9:            $x \leftarrow \sigma_1^{-1}(\sigma_2(x))$  for each loop-carried  $x$                                 ▶ update
10:           $\mathcal{D}_{\text{body}} \leftarrow \{\}$ 
11:          for all  $d \in \mathcal{D}_V$  reaching  $u_i$  do
12:             $\mathcal{D}_{\text{body}} \leftarrow \mathcal{D}_{\text{body}} \cup \text{EXTRACTITERATIONS}(d, W(\vec{x}), x \leftarrow x')$ 
13:             $g(\vec{x}) \leftarrow \text{LOOPGUARDSYNTHESIS}(\mathcal{D}_{\text{body}}, W, \sigma_1, \sigma_2)$ 
14:            Replace  $u_i \cdots u_{i+2l-1}$  in  $\mathcal{G}$  with loop  $v_{\text{loop}} \xrightarrow{g(\vec{x})} v_{\text{body}} \xrightarrow{W(\vec{x})} v_{\text{loop}}$ , exit on
            $\neg g(\vec{x})$ ; update  $(u_1, \dots, u_k)$ 
15:             $\mathcal{D}_V(v_{\text{body}}) \leftarrow \mathcal{D}_{\text{body}}$ ;  $found \leftarrow \text{true}$ ; break
16:          if  $found$  then break
17:   until  $\neg found$                                 ▶ Phase 2: recurse into each loop body at this level
18:   for all loop  $L$  among  $(u_1, \dots, u_k)$  do
19:      $L.\text{body}, \mathcal{D}_V \leftarrow \text{QUOTIENT}(L.\text{body}, \mathcal{D}_V)$ 
20:   return  $\mathcal{G}, \mathcal{D}_V$ 
21:
22: procedure EXTRACTITERATIONS( $d, W(\vec{x}), x \leftarrow x'$ )
23:    $\text{segs} \leftarrow []$ ;  $t \leftarrow 0$ 
24:   while  $\exists t' > t. d_{t'} \models W(\vec{x})$  do
25:     Let  $t'$  be the first such timestep
26:      $\text{segs} \leftarrow \text{segs} \cup [d_{t:t'}]$                                 ▶ one iteration: entry to state establishing  $W$ 
27:      $x \leftarrow x'$ ;  $t \leftarrow t'$ 
28:   return  $\text{segs}$ 

```

Definition 3.1 (Kleene-Encoded Fragment Merging). Let $w_1, \dots, w_n$ be the Kleene encodings of consecutive CFG fragments at the same nesting level of $\mathcal{G}$, each fragment a sub-sequence of frontier units (Algorithm 8). If there exists a template word $W(\vec{x})$ and substitutions $\sigma_1, \dots, \sigma_n$ such that $w_k = W(\vec{x})\sigma_k$ for every k —matched structurally over the Kleene grammar, with starred subterms matching only starred subterms of identical shape—then the nodes realizing $w_1, \dots, w_n$ are identified into a single node, and the edge following w_n is redirected to close a cycle back to this node, labeled by the template’s leading predicate.

Phase 1 — scan and match (lines 2–17). The procedure first reads off the *top-level* sequence $(u_1, \dots, u_k)$: the nodes of $\mathcal{G}$ in order, with every existing loop collapsed to a single atomic unit. It then repeatedly scans that sequence for the first pair of adjacent equal-length sub-sequences whose Kleene encodings are instances of a common template $W(\vec{x})$. Shorter lengths l are tried

before longer ones, so inner structure (which produces shorter repeated units) is matched before the compound units it creates become candidates for outer matching. After each successful merge the sequence shrinks and the scan restarts; the repeat terminates when no matching pair remains at this level.

In the flat case each sub-sequence is a single basic block, and matching reduces to checking whether two atomic edge labels share a predicate template: $\text{on}(b_1, b_0)$ and $\text{on}(b_2, b_1)$ both match $\text{on}(x', x)$ under $\sigma_1 = \{x \mapsto b_0, x' \mapsto b_1\}$ and $\sigma_2 = \{x \mapsto b_1, x' \mapsto b_2\}$. In the nested case each sub-sequence may itself be a loop unit; two such units match only if their Kleene encodings share the same starred-subterm shape. When a matching pair is found, the loop-carried initializations and updates are read off the substitutions (lines 8–9):

Definition 3.2 (Relational Quotienting). Given substitutions $\sigma_1, \dots, \sigma_n$ witnessing that consecutive CFG fragments $u_i \cdots u_{i+l-1}$ and $u_{i+l} \cdots u_{i+2l-1}$ are instances of a common template $W(\vec{x})$, quotienting collapses them into a loop as follows:

- (1) **Initialization.** Before the loop head, emit $x \leftarrow \sigma_1(x)$ for each non-existentially-bound $x \in \vec{x}$.
- (2) **Loop update.** At the end of the loop body, emit $x \leftarrow \sigma_i^{-1}(\sigma_{i+1}(x))$ for each non-existentially-bound $x \in \vec{x}$. Well-definedness requires this composition to be identical for all consecutive pairs (σ_i, σ_{i+1}) , which is checked syntactically; a variable for which it is undefined is rebound fresh each iteration by the loop guard.

Phase 2 — recurse into loop bodies (line 18). After Phase 1 stabilizes at the current level, QUOTIENT is called recursively on the body of each loop that appears among $(u_1, \dots, u_k)$. This handles the case where REFINECFG has split a node *inside* an already-formed loop: those new nodes are invisible to the top-level scan but become the top-level sequence of the recursive call, where Phase 1 detects and collapses any repeating pattern they exhibit.

Loop Body Preparation (lines 11–12) re-traces *every* demonstration d originally reaching the fragments' entry from its start, repeatedly applying the template-and-update rule via EXTRACTITERATIONS. Starting from the loop entry, EXTRACTITERATIONS finds the next timestep at which $W(\vec{x})$ holds under the current variable binding (line 24), cuts off the segment as one loop-body iteration (line 26), advances the loop-carried variable (line 27), and repeats. This recovers every iteration the demonstration actually exhibits—possibly more than the two that triggered detection—and the full collection becomes $\mathcal{D}_V(v_{\text{body}})$ (line 15).

Loop Guard Synthesis (line 13) synthesizes the guard $G(\vec{x})$ from the loop-entry states recovered by EXTRACTITERATIONS. Each loop-entry state s contributes one *positive example* (s, b_s) , where b_s is the object assigned to the existentially bound template variable in that iteration, and *local negative examples* (s, b_{neg}) for every other candidate object present in s . Positive examples are pooled across all loop-entry states—the synthesized guard must accept the demonstrated choice in every iteration—whereas negative examples remain local to their originating state. Enumerative synthesis learns one predicate $G_s(\vec{x})$ from each loop-entry state. If no predicate in the synthesis grammar distinguishes the demonstrated choice from the remaining candidates in s , the weakest consistent solution is $G_s = \text{true}$. Such states provide no additional discriminative information and therefore disappear under conjunction. The final guard is

$$G(\vec{x}) = \bigwedge_s G_s(\vec{x}),$$

where informative predicates accumulate constraints (e.g. Eq. 2 for Stack). Together with the loop-carried update $x \leftarrow x'$ recovered in Phase 1, the synthesized guard determines the loop's executable

Algorithm 5 STRAIGHTLINESYNTHESIZE($\mathcal{D}_v, \varphi$): synthesize a straight-line program from a demonstration segment, preferring candidates that establish φ

```

1: procedure STRAIGHTLINESYNTHESIZE( $\mathcal{D}_v, \varphi$ )
2:    $\pi \leftarrow$  a random sequence of primitive actions with random continuous parameters
3:    $\tau_{\mathcal{D}} \leftarrow$  the state distribution induced by  $\mathcal{D}_v$ 
4:   pool  $\leftarrow []$  ▷ candidates within  $\epsilon$  of the best KL score seen so far
5:   for  $k = 1, \dots, K$  do ▷ MCMC budget
6:      $\pi' \leftarrow \text{MUTATE}(\pi)$  ▷ modify an instruction, change its operands, or reorder instructions
7:      $\pi' \leftarrow \text{CROSSENTROPYOPTIMIZE}(\pi', \tau_{\mathcal{D}})$  ▷ optimize continuous parameters of  $\pi'$  to
       minimize  $\text{KL}(\tau_{\pi'} \parallel \tau_{\mathcal{D}})$ 
8:      $\Delta \leftarrow \text{KL}(\tau_{\pi} \parallel \tau_{\mathcal{D}}) - \text{KL}(\tau_{\pi'} \parallel \tau_{\mathcal{D}})$ 
9:     With probability  $\min(1, e^{\Delta/T_k})$ :  $\pi \leftarrow \pi'$  ▷ MCMC accept;  $T_k$  an annealed temperature
10:    pool  $\leftarrow \{ \rho \in \text{pool} \cup \{ \pi \} \mid \text{KL}(\tau_{\rho} \parallel \tau_{\mathcal{D}}) \leq \min_{\rho' \in \text{pool} \cup \{ \pi \}} \text{KL}(\tau_{\rho'} \parallel \tau_{\mathcal{D}}) + \epsilon \}$  ▷ keep
       all candidates within  $\epsilon$  of the current best
11:    if  $\text{KL}(\tau_{\pi} \parallel \tau_{\mathcal{D}}) < \theta$  then ▷ convergence threshold
12:       $\pi^* \leftarrow \arg \max_{\rho \in \text{pool}} \text{POSTSCORE}(\rho, \mathcal{D}_v, \varphi)$  ▷ among KL-close candidates, prefer
       postcondition success
13:      return  $\pi^*$ , true
14:    return  $\pi$ , false ▷ budget exhausted without converging
15:
16: procedure POSTSCORE( $\pi, \mathcal{D}_v, \varphi$ )
17:   return  $\Pr_{s_0 \sim \mathcal{D}_v} [ \pi \text{ rolled out from } s_0 \text{ reaches a state } s \models \varphi ]$  ▷ fraction of rollouts from
       demonstration-initial states reaching  $\varphi$ 

```

form:

$$\text{while } (\exists x'. G(x, x')) \{ \pi_{\text{body}}; x \leftarrow x' \}.$$

The enumeration follows the same bottom-up strategy as LEARNCLASSIFIER (Section 3.2.1). Unlike LEARNCLASSIFIER, guard synthesis searches over the full first-order vocabulary of $\mathcal{L}$, permitting universal quantifiers. The increased expressivity may place the generated verification conditions outside the decidable fragment; we characterize exactly when this occurs in Theorem 5.3.

Flat-loop illustration (Stack task). Phase 1 of QUOTIENT at $l = 1$ sees two length-1 words on (b_1, b_0) and on (b_2, b_1) , matches the atomic template on (x', x) under σ_1, σ_2 , and collapses them into the Stack loop of Definition 3.2. Phase 2 recurses into the newly formed loop body, which contains a single basic block, so nothing further matches.

A worked example of nested-loop discovery—a grid-building task requiring an outer anchor loop and an inner row-extension loop—is given in Appendix D.

3.4 Straight-Line Program Synthesis

STRAIGHTLINESYNTHESIZE (Algorithm 9) is the procedure introduced in Section 2.4, called for each block v of the CFG at line 3 of SYNTHESIZE. Given a demonstration segment $\mathcal{D}_v$ and the block's exit condition φ_v , it searches directly over straight-line programs—finite sequences of primitive actions instantiated with continuous parameters—scoring candidates by a combination of trajectory similarity to $\mathcal{D}_v$ and whether they actually establish φ_v , rather than by trajectory similarity alone. Any object identifier o still free in the returned program π^* is prepended as $v := \text{get}(\exists v : \Lambda(o), \text{True})$, generalizing the output across environments.

STRAIGHTLINESYNTHESIZE initializes π as a random straight-line program (line 2; Fig. 3 shows the initial program for the Stack task’s first block) and repeatedly proposes mutations: modifying an instruction, changing its operands, or reordering instructions (line 6). Following each structural mutation, the continuous parameters of the mutated program—e.g. target end-effector poses—are optimized using the cross-entropy method to minimize the KL divergence between the state distribution $\tau_{\pi'}$ induced by rolling out the mutated program and the target distribution $\tau_{\mathcal{D}}$ induced by the assigned demonstration segment $\mathcal{D}_v$ (line 7). The MCMC search accepts the mutation with probability increasing in how much it improves this KL-divergence score (lines 8–9).

Combining KL Divergence with Postcondition Satisfaction. Trajectory similarity alone can admit several candidates that resemble the demonstration equally well in distribution but differ in whether they actually establish the block’s postcondition—e.g. a near-miss that releases a block fractionally too early. Rather than folding postcondition satisfaction into a single weighted score with KL divergence, STRAIGHTLINESYNTHESIZE *filters, then ranks*: at every step it maintains pool, the set of all candidates seen so far whose KL-divergence score is within ϵ of the best score found (line 10). Once some candidate’s score drops below the convergence threshold θ (line 11), STRAIGHTLINESYNTHESIZE does not simply return it; instead it selects, from among every candidate currently in pool, the one with the highest POSTSCORE—the fraction of rollouts from the demonstration-initial states that actually reach φ (line 12). This way, postcondition satisfaction is only ever used to break ties *among* candidates that are already trajectory-faithful to the demonstration, rather than trading off against trajectory fidelity directly; a candidate far from $\tau_{\mathcal{D}}$ is never preferred merely because it satisfies φ more often. If no candidate is found within budget K , STRAIGHTLINESYNTHESIZE reports failure (line 14), which is exactly the non-convergence condition that triggers the REFINECFG step in Algorithm 2 for this block.

For example, realizing v_1 in Section 2.5—the block summarized by $\text{on}(b_1, b_0)$, with entry condition φ_{pre} and exit condition $\text{on}(b_1, b_0)$ itself—begins from a random sequence of Pick/Move/Release instructions (Fig. 3) and converges, under repeated mutation and parameter optimization against the segment $\mathcal{D}_V(v_1)$, to the sequence of approach, grasp, lift, transport, lower, and release actions for $\pi_{\text{on}(b', b)}$ shown in Fig. 5(b) (Section 2.6); among any KL-close variants of this sequence—e.g. differing only in release height or timing—POSTSCORE prefers the one whose rollouts most reliably leave b_1 actually resting on b_0 , rather than merely passing nearby in trajectory space. When the loop body of Fig. 4 is realized, the same procedure is instead scored against the pooled per-iteration segments $\mathcal{D}_V(v_{\text{body}})$ from EXTRACTITERATIONS (Section 3.3), so the synthesized loop body must match every recovered iteration simultaneously rather than any one of them, while still being selected for reliably establishing $\text{on}(b', b)$.

3.5 Termination Guarantee

SYNTHESIZE (Algorithm 2) terminates after finitely many refinements. This follows from the bounded size of relational CFGs, constrained by the length of demonstrations: (1) REFINECFG always splits a single block into two using a predicate witnessed by a demonstration segment (Section 3.2.2), and simultaneously partitions that block’s demonstration assignment, so the number of distinct splits is bounded by the number of distinct relational cutpoints occurring in $\mathcal{D}$, of which there are finitely many; and (2) QUOTIENT only ever *decreases* the number of unrealized acyclic blocks by collapsing matched chains into a bounded loop representation, so it cannot increase the bound from (1). Consequently the number of refinements is bounded, and Algorithm 2 must terminate, either by synthesizing a valid program or returning FAIL (line 16) when $\mathcal{G}$ cannot be further refined.

Each refinement of $\mathcal{G}$ (Algorithm 2, line 14) is followed by a VALIDATE call (line 15), which scans $\mathcal{G}$ against $\mathcal{D}_V$ to determine whether it can still yield a task-solving program. Given a demonstration segment $d = (s_0, s_1, \dots, s_H)$ and a predicate φ , if $\varphi(s_i)$ holds for some i , let $i_s(d, \varphi)$ be the first such

timestep and $i_f(d, \varphi)$ the last. For each edge $v_\varphi \xrightarrow{\psi} v_\psi$ in $\mathcal{G}$, VALIDATE checks, over every segment $d \in \mathcal{D}_V(v_\varphi) \cup \mathcal{D}_V(v_\psi)$, whether $i_s(d, \psi) \leq i_f(d, \varphi)$ when φ is not φ_{pre} , or $i_s(d, \psi) = 0$ otherwise. If this holds, the new block does not meaningfully advance the task beyond its predecessor, and the split is rejected.

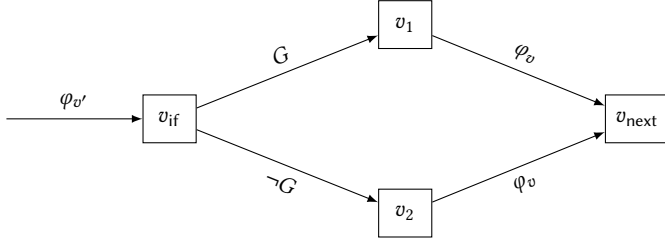
What is the main cause for RoboVerify to return FAIL? It occurs when STRAIGHTLINESYNTHESIZE fails to converge even for a short, single-step demonstration segment within its search budget, leading to repeated splits and eventual failure. This may stem from a demonstration segment whose state trajectory cannot be matched by any short composition of primitive actions in $\mathcal{L}$, or from the MCMC search becoming trapped away from a good local optimum; we have rarely observed this in practice.

3.6 Branch Refinement

Branch refinement handles a situation distinct from synthesis failure: the straight-line synthesizer for block v (with predecessor v') *partially* succeeds— π_v achieves the postcondition φ_v on some runs from $\mathcal{D}_V(v)$ but not all. When this occurs, the success or failure of π_v is correlated with the relational state at block entry: some entry configurations make π_v sufficient to establish φ_v ; others require a different straight-line implementation.

RoboVerify detects this by partitioning $\mathcal{D}_V(v)$ into *success traces*—runs of π_v that establish φ_v —and *failure traces*—runs that do not—and collects the block-entry states of each. It then calls LEARNCLASSIFIER (Section 3.2.1) with entry states of success traces as positives and entry states of failure traces as negatives, learning a guard predicate G that separates the two cases.

RoboVerify replaces v with a branch node v_{if} that forks on G , introducing two fresh blocks v_1 and v_2 that each link forward to the successor of v :



The demonstration assignment is partitioned in lockstep: success traces go to $\mathcal{D}_V(v_1)$ and failure traces to $\mathcal{D}_V(v_2)$. Block v_1 is then synthesized from the success traces alone—the learned guard G captures exactly the entry conditions under which π_v 's approach works, so v_1 realizes φ_v under G . Block v_2 is synthesized from the failure traces, which collectively describe the configurations under which a different strategy is required; it realizes φ_v under $\neg G$ and may itself be further split by additional refinement rounds. Both branches converge at v_{next} , the original successor of v , preserving the downstream CFG structure unchanged.

4 Automatic Loop Invariant Inference

After synthesizing a candidate program, RoboVerify infers universally quantified loop invariants from the execution traces stored in $\mathcal{D}_V$. Figure 6 shows the invariant for the stack task; this section describes how it is learned.

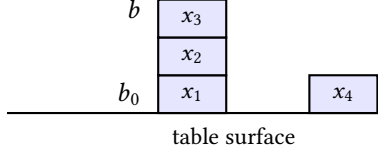


Fig. 7. Stacking scene: three blocks x_1, x_2, x_3 form a tower with base $b_0=x_1$ and top $b=x_3$, and an extra block x_4 lies on the table surface.

4.1 Learning $\forall$ -Invariants

We focus on invariants of the form $\forall \bar{x}. \psi(\bar{x})$, where $\bar{x}$ denotes a tuple of environment objects and ψ is a quantifier-free formula over the domain’s relational predicates ($\langle on^* \rangle$, Higher, Scattered, and any task-specific relations declared in $\mathcal{L}$) and equalities.

Building a hypothesis domain. We construct a finite vocabulary $\Omega_{\text{Inv}}(k)$ of candidate atomic predicates parameterized by a *view arity* k —the number of logical variables in each hypothesis. Typical choices are $k=1$ for unary invariants or $k=2$ for binary relational invariants over pairs of objects. For each relation symbol R of arity r in the domain we generate atomic predicates by instantiating R with up to k logical variables: $R(x_{i_1}, \dots, x_{i_r})$. We also include equality predicates ($x_i = x_j$) and program constants such as the base block b_0 and the loop-carried variable b as additional arguments. The resulting vocabulary defines the invariant language:

$$\text{Spec}_{\text{Inv}} = \{ \forall x_1, \dots, x_k. \xi \mid \xi \in \text{BF}(\Omega_{\text{Inv}}(k)) \},$$

where $\text{BF}(\cdot)$ denotes Boolean formulas over $\Omega_{\text{Inv}}(k)$.

Collecting loop-head states. For each quotiented loop, we extract environment states at the loop head from the demonstration traces in $\mathcal{D}_V$. These states capture the intermediate configurations the synthesized program actually visits, and serve as positive examples for invariant learning. Fig. 22 illustrates a representative loop-head state for the STACK task (Fig. 10): blocks x_2 and x_3 are stacked above the base block x_1 (the program variable b_0), while block x_4 remains on the table. The program variable b denotes the current top of the partially constructed tower.

Algorithm. The learning algorithm is adapted from [43]. For each target predicate $\omega \in \Omega_{\text{Inv}}(k)$, the loop-head rows are partitioned into those where ω holds (S) and those where it does not (U). The algorithm produces two kinds of clause. A *sufficient* clause $\phi \Rightarrow \omega$ is derived from rows in $S \setminus U$: ϕ is a conjunction of predicates in $\Omega_{\text{Inv}}(k)$ that holds on every S -row but fails on some U -row, found by solving a set-cover minimization over the rows. A *necessary* clause $\omega \Rightarrow \phi'$ is derived symmetrically from $U \setminus S$. In both cases the objective minimizes the number of selected predicates, preferring the most compact characterization supported by the data. After iterating over all target predicates, tautological and redundant clauses are dropped (using the domain axioms), and the survivors are conjoined to form the loop invariant. The novel element here is the instantiation of this framework to relational robot predicates ($\langle on^* \rangle$, Higher, Scattered) with loop-head states drawn from $\mathcal{D}_V$ rather than from symbolic model checking; the worked example below illustrates the process concretely.

Learning Procedure. We illustrate the proposed learning procedure on the STACK task (Fig. 10). For clarity, we consider the predicate set $\Omega_{\text{Inv}} = \{a \langle on^* \rangle b_0, y \langle on^* \rangle a, a \langle on^* \rangle b, a=y, b=a\}$, where a and y are universally quantified variables. Here, $a \langle on^* \rangle b_0$ indicates that a is in the partial stack above b_0 , $y \langle on^* \rangle a$ that y is above a , $a \langle on^* \rangle b$ that a is above the current top b , and $a=y, b=a$ denote equality relations. Fig. 23(a) lists the Boolean valuations of these predicates for all observed block pairs. The goal is to infer logical dependencies among these predicates.

a	y	$a\langle on^* \rangle b_0$	$y\langle on^* \rangle a$	$a\langle on^* \rangle b$	$a=y$	$b=a$
x_1	x_2	1	1	0	0	0
x_1	x_3	1	1	0	0	0
x_1	x_4	1	0	0	0	0
x_2	x_3	1	1	0	0	0
x_2	x_4	1	0	0	0	0
x_2	x_1	1	0	0	0	0
x_3	x_1	1	0	1	0	1
x_3	x_2	1	0	1	0	1
x_3	x_4	1	0	1	0	1
x_4	x_1	0	0	0	0	0
x_4	x_2	0	0	0	0	0
x_4	x_3	0	0	0	0	0
x_1	x_1	1	1	0	1	0
x_2	x_2	1	1	0	1	0
x_3	x_3	1	1	1	1	1
x_4	x_4	0	1	0	1	0

(a) Predicates Valuations over all block pairs (a, y) .

$a\langle on^* \rangle b_0$	$y\langle on^* \rangle a$	$a\langle on^* \rangle b$	$a=y$	$b=a$
S	1	0	0	0
S	0	1	0	1
S	1	1	1	1
U	0	0	0	0
U	1	0	1	0

(c) $S - U$, U partition for $a\langle on^* \rangle b_0$ from (b).

$a\langle on^* \rangle b_0$	$y\langle on^* \rangle a$	$a\langle on^* \rangle b$	$a=y$	$b=a$
S	1	0	0	0
S	0	0	0	0
S	0	1	0	1
S	1	0	1	0
S	1	1	1	1
U	0	0	0	0
U	1	0	1	0

(b) S , U partition for $a\langle on^* \rangle b_0$ from (a).

$a\langle on^* \rangle b_0$	$y\langle on^* \rangle a$	$a=y$	$b=a$
S	0	0	1
S	0	1	0
S	0	1	1
S	1	0	0
S	1	0	1
S	1	1	1
U	0	0	0
U	1	1	0

(d) Projection onto separating features (blue) from (c).

Fig. 8. Invariant learning for the Tower task.

We take $a\langle on^* \rangle b_0$ as the *target predicate* whose relation to the others we wish to learn. Samples are partitioned into sets S and U (Fig. 23(b)) according to whether $a\langle on^* \rangle b_0$ holds. The learning task is then to synthesize a classifier formula that separates the samples in S from those in U based on the remaining predicates, thereby capturing the conditions under which $a\langle on^* \rangle b_0$ holds. After removing overlapping samples between S and U , the learner infers a separating formula φ ; if derived from the reduced S , the resulting invariant takes the form $\varphi \Rightarrow a\langle on^* \rangle b_0$, and if derived from the reduced U , it takes the form $a\langle on^* \rangle b_0 \Rightarrow \varphi'$. Fig. 23(c) illustrates the resulting partition ($S - U$, U).

Since the hypothesis space of possible invariants grows exponentially with the number of atomic predicates, our approach promotes simplicity and generalizability by selecting a *minimal* subset of predicates sufficient for separation. This principle mirrors Occam's razor in logic learning: among all separating formulas, the most concise one is most likely to generalize across unseen program states. We associate an integer variable sel_i with each predicate $\Pi_i \in \Omega_{\text{inv}}$ ($0 \leq i < n$). If Π_i is selected as part of the separating classifier, $sel_i = 1$; otherwise, $sel_i = 0$. The learning problem is

encoded as the following constraint system:

$$\begin{aligned}
 \varphi_1 : & \bigwedge_{s \in S, u \in U} \bigvee_{0 \leq i < n} (s(\Pi_i) \neq u(\Pi_i) \wedge sel_i = 1) \\
 \varphi_2 : & \bigwedge_{0 \leq i < n} 0 \leq sel_i \leq 1 \\
 \varphi_c : & \min \left(\sum_{0 \leq i < n} sel_i \right)
 \end{aligned} \tag{3}$$

Here, φ_1 enforces separation between satisfying (S) and unsatisfying (U) samples: for every pair (s, u) , there must exist at least one predicate Π_i with differing evaluations and $sel_i = 1$. Constraint φ_2 ensures that each sel_i is binary, and φ_c minimizes the number of selected predicates, yielding a compact, generalizable invariant. The final selection of predicates forming a sufficient classifier is illustrated in Fig. 23(c). A truth table is then constructed over these predicates (Fig. 23(d)), rejecting all U -samples and accepting all remaining states, including those in S . Standard Boolean minimization is applied to derive the classifier’s logical structure. The resulting formula is:

$$(\neg y \langle on^* \rangle a \wedge a=y) \vee (y \langle on^* \rangle a \wedge a \neq y) \vee (b=a \Rightarrow a \langle on^* \rangle b_0).$$

This expression induces three candidate invariants:

$$\forall a, y. y \langle on^* \rangle a \vee a \neq y \vee a \langle on^* \rangle b_0, \quad \forall a, y. \neg y \langle on^* \rangle a \vee a=y \vee a \langle on^* \rangle b_0, \quad \forall a. b=a \Rightarrow a \langle on^* \rangle b_0.$$

The first and third are tautological, while the second captures the desired inductive invariant: each block a either has no block y above it (i.e., is a top block) or already belongs to the partial stack above b_0 .

Quantifiers are then added to generalize a and y over all objects, extending the learned classifier to unseen states. To infer all salient relations, we iterate over each predicate Π_i as the target and learn its dependencies on the remaining predicates. For instance, setting $a \langle on^* \rangle b$ as the target yields:

$$\forall a. a \langle on^* \rangle b \Rightarrow b=a, \quad \forall a. b=a \Rightarrow a \langle on^* \rangle b.$$

The first states that b is the current top of the stack, while the second is tautological. Combining these learned relations yields the final inductive invariant:

$$(\forall a, y. \neg y \langle on^* \rangle a \vee a=y \vee a \langle on^* \rangle b_0) \wedge (\forall a. a \langle on^* \rangle b \Rightarrow b=a) \wedge b \langle on^* \rangle b_0,$$

which compactly characterizes the structural invariant required to verify the synthesized TOWER program.

5 Program Verification

Synthesizing a program from a relational CFG is only half the problem: we must also guarantee that the synthesized program is *correct*, for an unbounded number of objects and at the level of continuous robot motion that the demonstrations never modeled directly. We verify every program RoboVerify produces at two levels: a *symbolic* level, where we reason about the relational CFG itself using standard weakest-precondition (WP) verification-condition (VC) generation [20], and a *motion* level, where we reason about the concrete robot motions realizing each basic block in continuous space. The two levels are connected by a single artifact—the loop invariant learned for each quotiented loop in the CFG (Section 4)—which must be strong enough to discharge the symbolic VCs and precise enough to translate, without loss, into the geometric assumptions the motion-level proof needs. This section states both levels generally, for an arbitrary program produced by our synthesis pipeline, and uses the Stack task only as a running illustration.

5.1 The AFR Fragment

Following [20], we restrict pre/postconditions, loop invariants, and CFG edge labels to a fragment of first-order logic for which wp is guaranteed to stay inside a decidable class, so that no VC ever escapes into a logic we cannot discharge automatically.

Definition 5.1 (QFR and AFR). Let R range over the domain’s relation symbols ($\cdot\langle on^* \rangle$), Higher, Scattered, and any task-specific relations declared in $\mathcal{L}$. An *atom* is $R(t_1, \dots, t_k)$ or $t_1 = t_2$, for object variables or constants t_i . A *quantifier-free* formula (QFR) is a Boolean combination of atoms. An *alternation-free* formula (AFR) is a QFR formula, or a formula $\forall \bar{u}. \varphi$ or $\exists \bar{u}. \varphi$ with φ QFR; in particular $\text{QFR} \subset \text{AFR}$.

Every pre/postcondition and every loop invariant synthesized by RoboVerify is purely universal, $\forall \bar{u}. \varphi$ —hence AFR—by construction: the enumerative predicate learners of Sections 3.2.1 and 3.3 only ever search over universally quantified templates. The one place an existential appears anywhere in our specification language is a **get**-guarded condition, of the shape

$$B \equiv \exists v. G(v),$$

arising from a $v \leftarrow \text{get}(\exists v. G(v))$ statement (Section 3.4) used either as a loop guard or as the condition of an **if**. Unlike pre/postconditions and invariants, G itself is *not* always quantifier-free in this paper: the Stack guard $G(b') \equiv \forall n. n = b' \vee \neg n\langle on^* \rangle b'$ is itself universally quantified, so B is AFR (one $\exists$ block wrapping a $\forall$ block, still alternation-free *within the guard itself*) but not QFR. Whether G is QFR turns out to determine the decidability of every VC that reasons about G *failing*—a loop’s exit, or an **if**’s else-branch—though not of reasoning about G *succeeding*, as Section 5.4 shows. No other existential occurs anywhere else in our fragment, in contrast to [20], where alternation originates from *inverting* a function symbol out of a reachability relation: every abstract action in our DSL is an edge *addition*, $\text{put}(b', b)$ or $\text{put}(b', \text{tbl})$, never a value read that needs inverting, so that source of alternation does not arise for us at all.

THEOREM 5.2 (CLOSURE UNDER wp). *For every abstract action a of the form $\text{put}(b', b)$ or $\text{put}(b', \text{tbl})$, and every AFR formula Q , $\text{wp}(a, Q)$ is expressible as an AFR formula.*

We establish Theorem 5.2 constructively, by exhibiting, for each relation symbol, an atom-rewriting operator that computes the effect of a directly on that relation’s truth value, without introducing any quantifier. Section 5.2 gives these operators in general; Theorem 5.2 follows because each operator only ever combines existing quantifier-free atoms with $\wedge$, $\vee$, and substitution of the action’s named objects.

5.2 General Weakest-Precondition Rules

Table 1 gives the wp rules used to verify any program produced by RoboVerify, stated for the standard control constructs (**skip**, sequencing, assignment, **while** and **if** with a **get**-guarded condition), the binding statement $v \leftarrow \text{get}(\exists v. G(v))$ that both kinds of guard are built from, and the two abstract action forms our DSL provides. As in [20], the rule for **while** returns only the loop invariant I ; preservation and exit are separate verification conditions, generated by VCaux below. For the two action forms, $R_{\langle on^* \rangle}$, R_{Higher} , and $R_{\text{Scattered}}$ each traverse a formula Q recursively and rewrite *only* the atoms of their corresponding relation, leaving every other atom of Q untouched; the exact rewriting rule for each relation, for each action form, is given in full in Appendix A (Table 7).

The rule for **get** deserves comment, since it is what the rest of this section turns on. Operationally, $v \leftarrow \text{get}(\exists v. G(v))$ binds v to an *arbitrary* witness satisfying G ; we do not control, and cannot

Table 1. General weakest-precondition rules used to verify any RoboVerify-synthesized program. S ranges over statements realized from a relational CFG; $b', b, \text{tbl} \in \text{Block}$ are the objects named by an abstract action.

Statement S	Weakest precondition $\text{wp}(S, Q)$
skip	Q
$S_1; S_2$	$\text{wp}(S_1, \text{wp}(S_2, Q))$
$x \leftarrow y$	$Q[y/x]$
$v \leftarrow \text{get}(\exists v. G(v))$	$\forall v. G(v) \Rightarrow Q$
while $\text{get}(\exists v. G(v))$ do S inv I	I
if $\text{get}(\exists v. G(v))$ then S_1 else S_2	$(\forall v. G(v) \Rightarrow \text{wp}(S_1, Q)) \wedge (\forall v. \neg G(v) \Rightarrow \text{wp}(S_2, Q))$
$\text{put}(b', b)$	$\neg b\langle \text{on}^* \rangle b' \wedge R_{\text{Scattered}}(R_{\text{Higher}}(R_{\langle \text{on}^* \rangle}(Q)))$
$\text{put}(b', \text{tbl})$	$R_{\text{Scattered}}(R_{\text{Higher}}(R_{\langle \text{on}^* \rangle}(Q)))$

assume anything about, which witness is returned. A correctness proof must therefore hold for every possible witness, so the weakest precondition universally quantifies over all of them:

$$\text{wp}(v \leftarrow \text{get}(\exists v. G(v)), Q) \triangleq \forall v. G(v) \Rightarrow Q.$$

This rule is the only place an existential quantifier in our specification language is consumed: it never propagates into wp as an existential, only as a universally quantified implication. The same **get**-guarded condition can gate either a **while** loop or an **if** branch inside a loop body; the **if** rule applies the same universal closure on its positive arm, and additionally requires the postcondition to hold when no witness exists at all, on its negative arm:

$$\text{wp}(\text{if } \text{get}(\exists v. G(v)) \text{ then } S_1 \text{ else } S_2, Q) \triangleq (\forall v. G(v) \Rightarrow \text{wp}(S_1, Q)) \wedge (\forall v. \neg G(v) \Rightarrow \text{wp}(S_2, Q)).$$

As Section 5.4 shows, these two arms behave very differently under negation: the positive arm's $\forall v. G(v) \Rightarrow (\dots)$ never flips G 's own internal quantifiers, while the negative arm's $\forall v. \neg G(v) \Rightarrow (\dots)$ does—exactly the same asymmetry as a loop's guard succeeding versus failing.

5.3 General Verification Condition Generation

Every program $\mathcal{P}$ realized from a relational CFG $\mathcal{G}$ (Section 3.4) is, by construction, a sequence of basic blocks and quotiented loops with no other control structure (Section 3.3), *except* that a loop body may itself contain a **get**-guarded **if** (Section 3.6). We generate VCs structurally over this shape, mirroring the standard VCgen/VCAux split of [20], writing both the loop and branch rules directly in terms of their guard's witness v rather than treating $\exists v. G(v)$ as an opaque condition $[[B]]$:

$$\begin{aligned} \text{VCAux}(S_1; S_2, Q) &\triangleq \text{VCAux}(S_1, \text{wp}(S_2, Q)) \cup \text{VCAux}(S_2, Q) \\ \text{VCAux}(\text{if } \text{get}(\exists v. G(v)) \text{ then } S_1 \text{ else } S_2, Q) &\triangleq \text{VCAux}(S_1, Q) \cup \text{VCAux}(S_2, Q) \\ \text{VCAux}(\text{while } \text{get}(\exists v. G(v)) \text{ do } S \text{ inv } I, Q) &\triangleq \text{VCAux}(S, I) \cup \left\{ I \Rightarrow \forall v. G(v) \Rightarrow \text{wp}(S, I), \right. \\ &\quad \left. I \wedge \forall v. \neg G(v) \Rightarrow Q \right\} \\ \text{VCgen}(\{\varphi_{\text{pre}}\} S \{\varphi_{\text{post}}\}) &\triangleq \varphi_{\text{pre}} \Rightarrow \text{wp}(S, \varphi_{\text{post}}) \wedge \bigwedge \text{VCAux}(S, \varphi_{\text{post}}). \end{aligned}$$

The loop's *preservation* obligation $I \Rightarrow \forall v. G(v) \Rightarrow \text{wp}(S, I)$ says: assuming the invariant, for every witness v the guard might produce, running the body preserves the invariant. The *exit* obligation $I \wedge \forall v. \neg G(v) \Rightarrow Q$ says: if no witness exists, the invariant alone must already imply

the postcondition. An **if** contributes no separate top-level obligation of its own—its correctness is folded directly into $wp(\text{if } \dots, Q)$ wherever it occurs (Table 1)—but, as we show next, it can still affect whether the *surrounding* VC is decidable, since that wp becomes part of $wp(S, I)$ when the **if** sits inside a loop body S . By Theorem 5.2, every $wp(S, Q)$ generated for an atomic statement S stays AFR whenever Q is AFR. What remains is to determine the quantifier shape of preservation and exit themselves, accounting for any **if** nested inside the loop body, which we do next.

5.4 Decidability of the Generated VCs

A formula is in the Bernays–Schönfinkel class (EPR) if its prenex negation has the shape $\exists^* \forall^*$ over a function-symbol-free matrix; EPR is decidable for satisfiability, and Z3 decides it directly [31].

THEOREM 5.3 (CONDITIONAL DECIDABILITY). *Suppose a program $\mathcal{P}$ satisfies: (1) every pre/postcondition and loop invariant is AFR; (2) every abstract action preserves AFR under wp (Theorem 5.2); (3) every **get-guarded condition** $\exists v. G(v)$ —**loop guard** or **if condition**—has quantifier-free G . Then every VC in $VCgen(\{\varphi_{pre}\} \mathcal{P} \{\varphi_{post}\})$ is decidable by reduction to EPR.*

PROOF. Under (3), $\neg G(v)$ is quantifier-free whenever $G(v)$ is, so no guard failure introduces an existential inside the scope of a surrounding universal. Because all pre/postconditions and invariants are AFR, the negation Z3 checks is $\forall^* \exists^*$ for initialization (decidable by a symmetric finite-model argument) and $\exists^* \forall^*$ for all remaining VCs. Every VC therefore reduces to a satisfiability check in the Bernays–Schönfinkel class [31], decidable by Z3. $\square$

Condition (3) is not always satisfied, and when it fails the VCs may escape EPR. To see why, suppose G contains an internal universal. Consider a loop’s preservation obligation $I \Rightarrow \forall v. G(v) \Rightarrow wp(S, I)$, where S is a straight-line body. Writing $I = \forall u. I_0(u)$ and $wp(S, I) = \forall x. R_0(x, v)$ (AFR by Theorem 5.2), the negation Z3 checks is

$$(\forall u. I_0(u)) \wedge \exists v. (G(v) \wedge \exists x. \neg R_0(x, v)).$$

Since $G(v)$ appears un-negated and u, x are independent of each other and of $G(v)$, any internal quantifier in G commutes freely into the leading existential block:

$$\exists v, x. \forall u. (I_0(u) \wedge G(v) \wedge \neg R_0(x, v)),$$

which is $\exists^* \forall^*$ —still EPR. The obstruction arises at a loop’s *exit* obligation $I \wedge \forall v. \neg G(v) \Rightarrow Q$, with $Q = \forall r. Q_0(r)$. If $G(v) = \forall n. G_0(n, v)$, as in Stack’s guard $\forall n. n = v \vee \neg n \langle on^* \rangle v$, then $\neg G(v) = \exists n. \neg G_0(n, v)$, and the negated exit VC becomes

$$\exists r. \forall u, v. \exists n. (I_0(u) \wedge \neg G_0(n, v) \wedge \neg Q_0(r)).$$

The existential $\exists n$ sits inside $\forall v$, making n functionally dependent on v ; pulling it outward requires Skolemizing n as $f(v)$ —exactly the function symbol EPR disallows. The same obstruction applies to the else-arm of any **if** whose guard contains an internal $\forall$.

Condition (3) is violated by Stack’s guard $\exists b'. \forall n. (n = b' \vee \neg n \langle on^* \rangle b') \wedge b' \neq b$, placing Stack’s exit VC outside EPR. We discharge it nonetheless via the domain axioms of Section ??, whose bounded alternation is within reach of Z3’s finite-model finding (Section 6). Theorem 5.3 identifies quantifier-free guards as the concrete, uniform target for a guaranteed-decidable fragment.

5.5 Motion-Level Verification

Symbolic verification reasons about abstract actions rather than continuous motions. Each synthesized controller π_φ is treated as an abstract transformer that establishes its contracted relational summary φ , updating only the relations modeled by the weakest-precondition rule for π_φ and preserving all others. Since the symbolic proof has no access to π_φ ’s continuous trajectory in the

environment, these assumptions cannot be justified symbolically and must instead be discharged by motion-level verification against the straight-line program synthesized for π_φ (Section 3.4).

Motion-level verification establishes exactly the two assumptions the symbolic proof relies on for each synthesized controller $\pi_\varphi = [\text{instr}_0, \dots, \text{instr}_{n-1}]$. First, *contract realization*: executing π_φ actually establishes the promised relational effect φ . Second, *frame preservation*: the controller does not disturb any object outside its abstract footprint—no object not explicitly referenced by π_φ collides with the motion or is displaced. Together these two properties close the gap between the symbolic model and physical execution.

Primitive specification and symbolic execution. We formalize the motion-level semantics of $\pi_\varphi = [\text{instr}_0, \dots, \text{instr}_{n-1}]$ as symbolic execution over a unified model of its control primitives. Let $\mathcal{O}$ be the finite set of all program constants in scope for π_φ (defined precisely below). Let $\mathcal{F}$ be a finite set of real-valued object attributes. A *symbolic state* $\sigma_i = ((\mathbf{F}_i)_{F \in \mathcal{F}}, p_i^{\text{arm}}, h_i)$ consists of an array $\mathbf{F}_i \in \text{Array}(\mathcal{O}, \mathbb{R})$ for each attribute $F \in \mathcal{F}$ (in Z3’s combined theory of arrays and NRA), a symbolic arm position p_i^{arm} , and a held-object constant $h_i \in \mathcal{O} \cup \{\perp\}$. The initial state is $\sigma_0 = ((\mathbf{F}_0)_{F \in \mathcal{F}}, p_{\text{init}}, \perp)$.

Assuming a single-arm robot, each primitive instr_i is associated with a four-field *specification*

$$(h_{i+1}, p_i^{\text{post}}, c_i, \Delta_i)$$

comprising the object held *after* instruction execution (h_{i+1}); the arm’s target position (p_i^{post}), a tuple indexed by $\mathcal{F}$ whose components are read from the current attribute arrays $\mathbf{F}_i$; the nominal Cartesian trajectory ($c_i : [0, 1] \rightarrow \mathbb{R}^{|\mathcal{F}|}$); and the object-position update (Δ_i). The trajectory (c_i) serves solely as a geometric specification: its induced swept volume is used for motion-level collision verification but does not modify the symbolic state. In contrast, (Δ_i) captures the state transition by updating the position of the manipulated object h_i , and is non-empty only for primitives that transport the held object. Given σ_i and instr_i ’s specification, the successor σ_{i+1} is:

- h_{i+1} : the post-held constant from the specification;
- $p_{i+1}^{\text{arm}} = p_i^{\text{post}}$;
- if $\Delta_i \neq \emptyset$ (when $h_i \neq \perp$: the arm carries the held object for transportation), every attribute of h_i is updated to the corresponding component of p_i^{post} —the held object lands wherever the arm ends up:

$$\mathbf{F}_{i+1}[a] = \begin{cases} (p_i^{\text{post}})_F & \text{if } a = h_i, \\ \mathbf{F}_i[a] & \text{otherwise,} \end{cases} \quad \text{for each } F \in \mathcal{F},$$

encoded as the Z3 array store $\mathbf{F}_{i+1} = \mathbf{F}_i[h_i \mapsto (p_i^{\text{post}})_F]$; when $\Delta_i = \emptyset$, all attribute arrays are unchanged: $\mathbf{F}_{i+1} = \mathbf{F}_i$. Each $\mathbf{F}_n$ is a closed-form composed-store expression over the initial arrays.

For the geometric verification problems considered in this paper, we assume $\mathcal{F} = \{X, Y, Z\}$, representing the three-dimensional position of each object. Physical execution is noisy. We model this by letting each primitive assign p_i^{post} as a nominal target perturbed by a free noise vector: $\varepsilon_g, \varepsilon_m, \varepsilon_r \geq 0$ are scalar noise bounds fixed by the robot’s hardware, and $\eta_g, \eta_m, \eta_r \in \mathbb{R}^3$ are the corresponding perturbation vectors, treated as free NRA variables subject to the interval constraints

shown in the spec. In our framework, the three control primitives have specifications:

$$\text{Pick}(v) : h_{i+1} = v, \quad p_i^{\text{post}} = (\mathbf{X}_i[v], \mathbf{Y}_i[v], \mathbf{Z}_i[v]) + \eta_g, \quad \eta_g \in [-\varepsilon_g, \varepsilon_g]^3, \quad \Delta_i = \emptyset.$$

$$\begin{aligned} \text{Move}(r_1, r_2, r_3, \delta) : h_{i+1} = h_i, \quad p_i^{\text{post}} &= (\mathbf{X}_i[r_1] + \delta_x, \mathbf{Y}_i[r_2] + \delta_y, \mathbf{Z}_i[r_3] + \delta_z) + \eta_m, \quad \eta_m \in [-\varepsilon_m, \varepsilon_m]^3, \\ \Delta_i &= \begin{cases} \{h_i \mapsto p_i^{\text{post}}\} & \text{if } h_i \neq \perp, \\ \emptyset & \text{otherwise.} \end{cases} \end{aligned}$$

$$\begin{aligned} \text{Release}(v, h_r) : h_{i+1} = \perp, \quad p_i^{\text{post}} &= (\mathbf{X}_i[v], \mathbf{Y}_i[v], \mathbf{Z}_i[v] + h_r) + \eta_r, \quad \eta_r \in [-\varepsilon_r, \varepsilon_r]^3, \\ \Delta_i &= \begin{cases} \emptyset & \text{if } \exists n \in \mathcal{O}. \text{on}(v, n), \\ \{v \mapsto \star\} & \text{otherwise.} \end{cases} \end{aligned}$$

The perturbations η_g, η_m, η_r are free NRA variables. Symbolic execution carries them forward: Δ_i stores p_i^{post} (which contains η_i) into the position arrays, and $p_{i+1}^{\text{arm}} = p_i^{\text{post}}$ propagates η_i into the next step, so noise accumulates through the sequence.

For each primitive, the Cartesian trajectory is the straight line

$$c_i(t) = (1 - t) p_i^{\text{arm}} + t p_i^{\text{post}},$$

where $p_i^{\text{arm}} = p_{i-1}^{\text{post}}$ (from the prior step) and p_i^{post} carries the noise variable η_i . Because η_i appears inside $c_i(t)$, the collision query (5) is an NRA formula over η_i as a free variable bounded by $[-\varepsilon_i, \varepsilon_i]^3$; Z3's unsatisfiability check ranges over all actual trajectories within the noise bound simultaneously. No separate tightening of the collision threshold is required. Pick targets the block position directly ($\Delta_i = \emptyset$, since holding begins here); Release targets height $\mathbf{Z}_i[v] + h_r$ above v . The conditional in Release reflects physical support: if something is beneath v upon release, v rests in place ($\Delta_i = \emptyset$); otherwise v falls freely ($v \mapsto \star$). Evaluating $\exists n \in \mathcal{O}. \text{on}(v, n)$ over a symbolic state is deferred to Definition 5.4 below.

Translating symbolic context to geometric constraints. Symbolic execution of π_φ reasons only about the objects explicitly referenced by its instructions. Motion-level verification, however, must establish collision freedom with respect to all objects in the environment, including those never mentioned by the program, and consequently requires a global geometric description of the environment. Symbolic program specifications such as loop invariants supply this description: they establish relational properties over $\langle \text{on}^* \rangle$, Higher, and Scattered that characterize the spatial arrangement of every environment object. The geometric translation $\widehat{(\cdot)}$ (Definition 5.4 below) converts this relational context into quantifier-free NRA constraints that Z3 can discharge directly.

Fix $\mathcal{O}$ as the finite set of all program constants in scope for π_φ : the loop-level constants, the **get**-witness v bound by $v \leftarrow \text{get}(\exists v. G(v))$, and a fresh constant $w \notin \pi_\varphi$ as the collision-check witness. Applying $\widehat{(\cdot)}$ (Definition 5.4 below) to I and $G(v)$ (Section 5.1) yields the quantifier-free NRA formulas

$$\widehat{I}_O := \widehat{I}, \quad \widehat{G}_O(v) := \widehat{G(v)};$$

v remains free in $\widehat{G}_O(v)$.

Definition 5.4 (Geometric translation $\widehat{(\cdot)}$). Given a symbolic state $\sigma_i = ((\mathbf{F}_i)_{F \in \mathcal{F}}, p_i^{\text{arm}}, h_i)$, the translation $\widehat{(\cdot)}[\sigma_i]$ maps each relational atom to a quantifier-free NRA formula over $(\mathbf{F}_i)_{F \in \mathcal{F}}$:

$$\widehat{\text{on}}(x, y)[\sigma_i] := |\mathbf{X}_i[x] - \mathbf{X}_i[y]| \leq \delta_X \wedge |\mathbf{Y}_i[x] - \mathbf{Y}_i[y]| \leq \delta_Y \wedge |\mathbf{Z}_i[x] - \mathbf{Z}_i[y]| \leq \delta_Z,$$

$$\widehat{x \langle \text{on}^* \rangle y}[\sigma_i] := |\mathbf{X}_i[x] - \mathbf{X}_i[y]| \leq N_X \wedge |\mathbf{Y}_i[x] - \mathbf{Y}_i[y]| \leq N_Y \wedge \mathbf{Z}_i[x] \geq \mathbf{Z}_i[y],$$

$$\widehat{\text{Higher}}(x, y)[\sigma_i] := \mathbf{Z}_i[x] > \mathbf{Z}_i[y],$$

$$\widehat{\text{Scattered}}(x, y)[\sigma_i] := \text{dist}(\text{bbox}_i(x), \text{bbox}_i(y)) \geq d,$$

where $\delta_X, \delta_Y, \delta_Z$ are per-step tolerances for the direct relation (δ_F for $F \in \{X, Y, Z\}$); N_X, N_Y are chain-root horizontal bounds for the transitive closure (N_F for $F \in \{X, Y\}$, with Z handled one-sidedly); and d is a minimum separation distance, with $\text{bbox}_i(a)$ the axis-aligned bounding box of a at σ_i . The required consistency $N_F \geq 2\delta_F$ is certified by the alignment VC (Section F). Boolean connectives translate homomorphically, and universal quantifiers are instantiated over $\mathcal{O}$:

$$\widehat{\varphi \wedge \psi}[\sigma_i] := \widehat{\varphi}[\sigma_i] \wedge \widehat{\psi}[\sigma_i], \quad \widehat{\forall x. \varphi}[\sigma_i] := \bigwedge_{a \in \mathcal{O}} \widehat{\varphi[a/x]}[\sigma_i].$$

Since $\mathcal{O}$ is finite, $\widehat{I}_O[\sigma_i]$ and $\widehat{G}_O(v)[\sigma_i]$ are quantifier-free for any σ_i . The translation is sound by construction: any concrete state satisfying $\widehat{I}_O[\sigma_0]$ is a model of I under the standard geometric interpretation of on , $\langle \text{on}^* \rangle$, Higher , and Scattered ; in particular, $\widehat{I}_O[\sigma_0]$ is exactly as strong as the loop invariant I the symbolic proof established.

Symbolic execution propagates attribute values forward, updating $\mathbf{F}_0 \rightarrow \mathbf{F}_1 \rightarrow \dots \rightarrow \mathbf{F}_n$ via array stores (when $\Delta_i \neq \emptyset$), yielding symbolic state σ_i just before instruction i . The loop invariant and guard are evaluated at the initial state σ_0 , giving $\widehat{I}_O[\sigma_0]$ and $\widehat{G}_O(v)[\sigma_0]$; the collision predicate collide_i (below) uses σ_i to position objects at the start of segment i . The Release supporting condition $\exists n \in \mathcal{O}. \text{on}(v, n)$ is evaluated at the release step's state σ_i as the disjunction $\bigvee_{o \in \mathcal{O}} \widehat{\text{on}}(v, o)[\sigma_i]$.

Verification conditions. These VCs are stated for the canonical case in which π_φ is a loop-body block; the geometric context is then

$$A := \widehat{I}_O[\sigma_0] \wedge \widehat{G}_O(v)[\sigma_0] = \widehat{I \wedge G(v)}[\sigma_0],$$

combining the loop invariant and the witnessed get-guard at the loop-entry state σ_0 . How A is derived for an arbitrary program point – including blocks before the loop, blocks in branches, and blocks after loop exit – is formalized by the propagation rules at the end of this section. All three VCs are discharged by Z3's NRA engine.

Contract realization. After running $\pi_\varphi = [\text{instr}_0, \dots, \text{instr}_{n-1}]$ symbolically to obtain $\sigma_n = ((\mathbf{F}_n)_{F \in \mathcal{F}}, p_n^{\text{arm}}, h_n)$, contract realization holds iff

$$A \models \widehat{\varphi}[\sigma_n], \tag{4}$$

where $\widehat{\varphi}[\sigma_n]$ substitutes each attribute array $\mathbf{F}[a]$ in the geometric postcondition with the final $\mathbf{F}_n[a]$ for each $F \in \mathcal{F}$ and $a \in \mathcal{O}$. Because each $\mathbf{F}_n[a]$ is a closed-form composed-store expression (no existential quantifiers over the trace), this is a quantifier-free NRA validity check—equivalently, $A \wedge \neg \widehat{\varphi}[\sigma_n]$ is UNSAT.

Frame Preservation. For each instruction instr_i , frame preservation requires that no environment object collides with the arm or its payload along the trajectory c_i . Writing $\text{collide}_i(w, v)$ for the predicate asserting that the bounding box of w (at its current position, formalized below) intersects the arm's swept volume during segment i , the safety condition is:

$$\forall v. \forall w. A \wedge w \neq h_i \Rightarrow \neg \text{collide}_i(w, v).$$

RoboVerify establishes this by proving the negation **unsatisfiable**:

$$\exists v. \exists w. A \wedge \text{collide}_i(w, v) \wedge w \neq h_i. \quad (5)$$

Here h_i is the pre-held field of σ_i (a concrete constant from the primitive specification above, not universally quantified); when $h_i = \perp$, the condition $w \neq h_i$ is vacuous. All constants in $O(v, w)$, and the loop-level constants) are uninterpreted; Z3 checks whether any assignment satisfies the conjunction.

A block $w \neq h_i$ (the pre-held field of σ_i) collides with segment i if its bounding box at $(X_i[w], Y_i[w], Z_i[w])$ intersects the arm's swept volume:

$$\text{collide}_i(w, v) \equiv \exists t \in [0, 1]. |X_i[w] - c_{i,x}(t)| < L \wedge |Y_i[w] - c_{i,y}(t)| < L \wedge |Z_i[w] - c_{i,z}(t)| < L.$$

The trajectory $c_i(t)$ depends on v through the arm chain: if $\text{Pick}(v)$ preceded segment i , then p_i^{arm} involves $X_0[v], Y_0[v], Z_0[v]$ (Pick leaves arrays unchanged), making the $\exists v$ in formula (5) non-vacuous. Collision-freedom requires formula (5) to be UNSAT for each segment i . When $h_i = \perp$ (e.g., during Pick), the exclusion $w \neq h_i$ is vacuous; the noise bound $\varepsilon_g < L$ then ensures Pick does not register a spurious self-collision with its target block, since the arm's noise ball does not overlap a block at distance exceeding ε_g from the target. The bounding-box check is a sound over-approximation—it covers a superset of actual collision events—so UNSAT certifies collision-freedom regardless of the specific controller realizing $c_i(t)$. The bounded existential $\exists t \in [0, 1]$ is discharged by Z3 via interval elimination.

Alignment Certification. Transitive-closure atoms $R^*(x, y)$ in the invariant I introduce a certification obligation beyond contract realization. A chain $x \xrightarrow{R} a_1 \xrightarrow{R} \dots \xrightarrow{R} y$ of k R -steps, where each step contributes drift at most δ_F , can accumulate total drift up to $k\delta_F$ and push the endpoints arbitrarily far apart as k grows — while still satisfying $R^*(x, y)$ at the relational level. The geometric translation captures this with bounds N_F in $\widehat{R^*}(x, y)$ (Definition 5.4), but those bounds are not automatically guaranteed by the chain structure.

LEMMA 5.5 (TRANSITIVE ALIGNMENT). *Let r be a root element and S a finite set of elements each satisfying $|\mathbf{F}[a] - \mathbf{F}[r]| \leq \delta_F$ for every attribute F . Then any two elements $a, a' \in S$ satisfy $|\mathbf{F}[a] - \mathbf{F}[a']| \leq 2\delta_F \leq N_F$.*

PROOF. By induction on $|S|$, maintaining:

- (A) every $a \in S$ satisfies $|\mathbf{F}[a] - \mathbf{F}[r]| \leq \delta_F$; and
- (B) every pair $a, a' \in S$ satisfies $|\mathbf{F}[a] - \mathbf{F}[a']| \leq 2\delta_F$.

Base case ($S = \{r\}$): (A) holds trivially; (B) is vacuous.

Inductive step: Assume $\text{IH}(n)$ and add element b satisfying $|\mathbf{F}[b] - \mathbf{F}[r]| \leq \delta_F$. **(A) for $n+1$:** b satisfies the bound by assumption; existing elements retain theirs by $\text{IH}(A)$. **(B) for $n+1$:** existing pairs satisfy (B) by $\text{IH}(B)$; for any existing a paired with the new b :

$$|\mathbf{F}[b] - \mathbf{F}[a]| \leq |\mathbf{F}[b] - \mathbf{F}[r]| + |\mathbf{F}[r] - \mathbf{F}[a]| \leq \delta_F + \delta_F = 2\delta_F.$$

□

The pairwise bound $2\delta_F$ is derived automatically from $\widehat{R^*}$ in Definition 5.4; no separate parameter is needed.

The alignment VC certifies the per-step hypothesis of Lemma J.1 for each element x newly placed by π_ϕ . For each atom $R(x, y)$ in ϕ — where x and y are meta-variables matched structurally to

program constants, exactly as in the WP rules — define the *symmetric projection*

$$\widehat{R_{\Pi}(x, r)} := \bigwedge_{\substack{F \in \mathcal{F} \\ |F[x] - F[r]| \leq N_F \in \widehat{R^*(x, r)}}} |F[x] - F[r]| \leq \delta_F,$$

retaining only those attributes F for which $\widehat{R^*(x, r)}$ carries a two-sided bound N_F (one-sided bounds such as $Z[x] \geq Z[r]$ are excluded), but replacing each chain-level N_F with the tighter per-step tolerance δ_F from $\widehat{R(x, r)}$. The *alignment* VC substitutes $R_{\Pi}(x, r)$ for each $R(x, y)$ in φ and checks the result against the final symbolic state σ_n :

$$A \models \varphi[\widehat{R_{\Pi}(x, r)} / R(x, y)][\sigma_n]. \quad (6)$$

The root $r \in \mathcal{O}$ is identified from $\text{wp}(\pi_{\varphi}, I)$, computed at the relational level using the abstract WP rules (Section 5.3). Enumerate $\mathcal{O}$ to find the constant r for which

$$\text{wp}(\pi_{\varphi}, I) \Rightarrow \forall u \in \mathcal{O}. R^*(y, u) \Rightarrow R^*(u, r),$$

meaning every in-scope element reachable from y can further reach r , making r the chain root of y . Instantiating at $u = y$ with reflexivity $R^*(y, y)$ gives $R^*(y, r)$. By transitivity: $R(x, y) \wedge R^*(y, r) \Rightarrow R^*(x, r)$, so x lies in the R -chain rooted at r .

The two checks combine naturally: replacing $R(x, y)$ in φ by $R_{\Pi}(x, r) \wedge R(x, y)$ yields a single VC simultaneously certifying contract realization and alignment,

$$A \models \varphi[\widehat{R_{\Pi}(x, r) \wedge R(x, y)} / R(x, y)][\sigma_n]. \quad (7)$$

Like contract realization (4), both are quantifier-free NRA checks under A ; no parameter is needed beyond what appears in Definition 5.4.

Symbolic context by forward propagation. The VCs above used the loop-body context $A = \widehat{I \wedge G(v)}[\sigma_0]$ as a fixed example. In general, the geometric context for *any* basic block in the relational CFG is obtained by a single forward traversal. We formalize this via the judgment

$$\langle A, \sigma \rangle \vdash S \rightsquigarrow \langle A', \sigma' \rangle,$$

where A is a geometric (QF NRA) formula representing the current assumption, σ is the current symbolic state, and A is always already evaluated at its entry state — there is no separate relational component. Relational formulas Φ appear only when a new context is introduced (at loop heads or branch points), where they are immediately instantiated via $\widehat{\Phi}[\sigma]$ before entering the pair. $\text{fresh}()$ denotes a fresh symbolic state with unconstrained attribute arrays. The rules are syntax-directed:

$$\text{(BLOCK)} \quad \frac{\text{discharge VCs for } \pi_{\varphi} \text{ under } A}{\langle A, \sigma \rangle \vdash \pi_{\varphi} \rightsquigarrow \langle A, \text{exec}(\pi_{\varphi}, \sigma) \rangle}$$

A is unchanged; the state advances by symbolic execution. VCs (above) are discharged directly against the geometric A .

$$\text{(SEQ)} \quad \frac{\langle A, \sigma \rangle \vdash S_1 \rightsquigarrow \langle A_1, \sigma_1 \rangle \quad \langle A_1, \sigma_1 \rangle \vdash S_2 \rightsquigarrow \langle A_2, \sigma_2 \rangle}{\langle A, \sigma \rangle \vdash S_1; S_2 \rightsquigarrow \langle A_2, \sigma_2 \rangle}$$

Both A and σ thread forward; no reset occurs between consecutive blocks.

$$\text{(WHILE)} \quad \frac{\sigma^* = \text{fresh()} \quad \langle \widehat{I \wedge G(v)}[\sigma^*], \sigma^* \rangle \vdash S \rightsquigarrow \langle _, _ \rangle}{\langle A, \sigma \rangle \vdash \text{while}^I \text{ get}(\exists v. G(v)) \text{ do } S \rightsquigarrow \langle \widehat{I \wedge \neg \exists v. G}[\sigma^{**}], \sigma^{**} \rangle}$$

where $\sigma^{**} = \text{fresh}()$. The loop resets to a fresh σ^* ; the invariant I and guard $G(v)$ are instantiated at σ^* to form the body's geometric context — exactly the A used in the VCs above. The pre-loop

pair and the body's exit pair are both discarded. The invariant I is established separately by VCgen (Section 5.3). Post-loop, a fresh σ^{**} carries context $I \wedge \neg \exists v. G[\sigma^{**}]$.

$$(If) \quad \frac{\langle A \wedge \widehat{C}[\sigma], \sigma \rangle \vdash S_1 \rightsquigarrow \langle A_1, \sigma_1 \rangle \quad \langle A \wedge \neg \widehat{C}[\sigma], \sigma \rangle \vdash S_2 \rightsquigarrow \langle A_2, \sigma_2 \rangle}{\langle A, \sigma \rangle \vdash \text{if } C \text{ then } S_1 \text{ else } S_2 \rightsquigarrow \langle A_1 \vee A_2, \text{fresh}() \rangle}$$

Branching acts as assume: σ is unchanged into both arms, and the branch condition C is instantiated at the *current* σ to yield $\widehat{C}[\sigma]$ or $\neg \widehat{C}[\sigma]$. Since both A and $\widehat{C}[\sigma]$ are geometric formulas, their conjunction is unambiguous regardless of when A was formed. The join uses a fresh state, since the two arms may reach different final states.

Verification begins as $\langle \widehat{\varphi}_{\text{pre}}[\sigma_0], \sigma_0 \rangle \vdash S_{\text{program}} \rightsquigarrow \langle _, _ \rangle$, where $\sigma_0 = \text{fresh}()$.

At the symbolic level, VCgen generates three obligations for the Stack loop (Fig. 4), with invariant I_{stack} (Fig. 6) and guard $\exists b'. G(b', b)$, $G(b', b) \equiv (\forall n. n = b' \vee \neg n \langle \text{on}^* \rangle b') \wedge b' \neq b$. **Preservation** reduces, by Section 5.4, to $\exists b', x. \forall u. (I_{\text{stack}} \text{'s matrix} \wedge G(b', b) \wedge \neg wp(\pi(\mu, b', b); b \leftarrow b', I_{\text{stack}}) \text{'s matrix})$ in negated form—EPR, regardless of G 's internal $\forall n$, exactly as Theorem 5.3 guarantees for any positive occurrence. Two facts close the case: $I_{\text{stack}} \Rightarrow b \langle \text{on}^* \rangle b_0$ (instantiating at $u := b$ with Reflexivity) and G 's $\forall n$ forces $u \langle \text{on}^* \rangle b' \Leftrightarrow u = b'$ for every u . **Exit** falls outside EPR (the negated exit VC has genuine $\exists^* \forall^* \exists^*$ shape) but Z3 discharges it directly under the background axioms of Section ??.

Illustration: motion-level verification (Stack task). We illustrate both motion checks on the Stack loop body. At the motion level, v instantiates as b' and w as sym; writing $G(b', b) = \forall n. G_0(b', b, n)$ with $G_0 \equiv (n = b' \vee \neg n \langle \text{on}^* \rangle b') \wedge b' \neq b$, the instantiated guard is $\widehat{G}_O(b') \equiv \bigwedge_{a \in O} (a = b' \vee \neg a \langle \text{on}^* \rangle b') \wedge b' \neq b$.

Incorrect candidate (rejected). Consider a 6-step candidate with the naive alignment step 5:

$$\pi_\varphi^\times = [\text{Move}(b', b', b', (0, 0, 0.15)), \text{Pick}(b'), \text{Move}(b', b', b, (0, 0, 0.15)), \\ \text{Move}(b, b, b, (0, 0, 0.15)), \text{Move}(\mathbf{b}, \mathbf{b}, \mathbf{b}, (0, 0, 0.05)), \text{Release}(b', 0.15)].$$

Steps 1–4 pre-position; step 5 (bold) lowers the arm to release height targeting ($X[b], Y[b], Z[b] + 0.05$); step 6 releases.

Collision check (step 4). Step 4, $\text{Move}(b, b, b, (0, 0, 0.15))$, is the key transport segment carrying b' horizontally from above its original position to above b . The in-scope constants are $O = \{b_0, b, b', \text{sym}\}$ (Figure 21). The arm enters at $p^{\text{arm}} = p_{\text{Pick}}^{\text{post}} + \eta_g$, $\eta_g \in [-\varepsilon_g, \varepsilon_g]^3$, and targets $p^{\text{post}} = (X[b], Y[b], Z[b] + 0.15) + \eta_m$, $\eta_m \in [-\varepsilon_m, \varepsilon_m]^3$, so $c(t)$ spans from above b' to above b . With $h_i = b'$, formula (5) concretizes to

$$\exists b'. \exists \text{sym}. \widehat{I_{\text{stack}O}}[\sigma_0] \wedge \widehat{G_O}(b, b')[\sigma_0] \wedge \text{collide}_i(\text{sym}, b') \wedge \text{sym} \neq b',$$

where $\widehat{I_{\text{stack}O}}[\sigma_0] \equiv \bigwedge_{o_1, o_2 \in O} \widehat{\text{inv}}(o_1, o_2)[\sigma_0]$ (Figure 21). Z3 proves this unsatisfiable: $\widehat{I_{\text{stack}}}$ forces every block other than b' to be in the tower below b or geometrically scattered, leaving no object in the swept volume for any guard-valid b' .

Alignment VC (step 5). Step 5 stores $X[b'] = X[b] + \eta_{m,x}$ via Δ . The alignment VC (6) requires $|X[b'] - X[b_0]| \leq \delta_X$ in the final state (the root is $r = b_0$, found by propagating $\text{on}^*(b, b_0)$ unchanged through the WP). Substituting: $|X[b] + \eta_{m,x} - X[b_0]| \leq |X[b] - X[b_0]| + \varepsilon_m$. Since $I_{\text{stack}} \Rightarrow \text{on}^*(b, b_0)$, $\widehat{I_{\text{stack}}}$ includes $\widehat{\text{on}^*(b, b_0)}$, which by Definition 5.4 carries the chain-root bound $|X[b] - X[b_0]| \leq N_X$. The total bound $N_X + \varepsilon_m$ must satisfy $\leq \delta_X$; since $N_X \geq 2\delta_X$ (Definition 5.4), we have $N_X + \varepsilon_m \geq 2\delta_X + \varepsilon_m > \delta_X$. Z3 finds a satisfying assignment ($\eta_{m,x} = \varepsilon_m$, $X[b] = X[b_0] + N_X$) that witnesses the violation; $\pi_\varphi^\times$ is rejected.

$$\mathcal{O} \equiv \{b_0, b, b', \text{sym}\}, \quad \widehat{I_{\text{stack}}\mathcal{O}}[\sigma_0] \equiv \bigwedge_{o_1, o_2 \in \mathcal{O}} \widehat{\text{inv}(o_1, o_2)}[\sigma_0].$$

Fig. 9. Stack instantiation of the general finite expansion (Definition 5.4, formula (5)). The in-scope constants are b_0 (base block), b (current tower top), b' (get-witness v), and sym (the fresh collision witness w). Each $\widehat{\text{inv}(o_1, o_2)}$ is obtained by instantiating $I_{\text{stack}} \equiv \forall u, v. \text{inv}(u, v)$ at $(u, v) := (o_1, o_2)$ and applying $\widehat{(\cdot)}$.

Corrected program (accepted). The synthesizer replaces step 5 with $\text{Move}(b_0, b_0, b, (0, 0, 0.05))$:

$$\pi_\varphi = [\text{Move}(b', b', b', (0, 0, 0.15)), \text{Pick}(b'), \text{Move}(b', b', b, (0, 0, 0.15)), \\ \text{Move}(b, b, b, (0, 0, 0.15)), \text{Move}(\mathbf{b}_0, \mathbf{b}_0, \mathbf{b}, (0, 0, 0.05)), \text{Release}(b', 0.15)].$$

Collision check. The collision query for step 5 (a short downward move within the same cleared column) is unsatisfiable by the same invariant argument as step 4.

Alignment VC. Step 5 now targets $(X[b_0], Y[b_0], Z[b] + 0.05)$, storing $X[b'] = X[b_0] + \eta_{m,x}$. Thus $|X[b'] - X[b_0]| = |\eta_{m,x}| \leq \varepsilon_m \leq \delta_X$ —a fixed hardware-vs-geometry bound; Z3 discharges this without appeal to $\widehat{I_{\text{stack}}}$. The same holds for Y . By Lemma J.1, the pairwise separation across the whole stack is at most $2\delta_X$ in X and $2\delta_Y$ in Y , closing the gap that frame preservation relies on.

THEOREM 5.6 (MOTION-LEVEL SOUNDNESS). *Let π_φ be a straight-line program synthesized to establish φ from entry condition φ_{in} (Section 3.4), and let A be its geometric context derived by the propagation rules of Section E. Suppose (1) the symbolic execution of π_φ from initial state σ_0 produces final state $\sigma_n = ((F_n)_{F \in \mathcal{F}}, p_n^{\text{arm}}, h_n)$ such that check (4) holds, (2) for every motion segment i of π_φ , formula (5) is unsatisfiable, and (3) for every transitive-closure atom of φ , the alignment VC (6) holds. Then π_φ realizes its contract: every execution from a state satisfying A establishes φ , keeps every R -chain within the bounds N_F assumed by Definition 5.4, and does not disturb any object outside the controller’s abstract footprint. Consequently, the symbolic VC for this block (Section 5.3) is fully justified by motion-level evidence.*

PROOF. Condition (1) establishes contract realization. Each primitive’s noise perturbation (η_g , η_m , or η_r) appears in p_i^{post} as a free NRA variable; Δ_i stores p_i^{post} into the position arrays, so η_i propagates into σ_n . Check (4) is an NRA formula over these free variables; Z3’s unsatisfiability check universally quantifies $\eta_r \in [-\varepsilon_r, \varepsilon_r]^3$, certifying $\widehat{\varphi}[\sigma_n]$ for every actual Release position within the noise bound—this is exactly contract realization under noise. Condition (2) provides frame preservation: η_i appears inside $c_i(t)$, so formula (5) is an NRA formula over $\eta_i \in [-\varepsilon_i, \varepsilon_i]^3$; its unsatisfiability certifies that *no* actual trajectory within the noise bound—not just the nominal—produces a collision; since $\widehat{(\cdot)}$ faithfully translates the block’s relational context by Definition 5.4, this covers every geometric state the context permits. Condition (3) certifies the per-step hypothesis of Lemma J.1 for each element newly placed by π_φ , so every R -chain the context describes with a two-sided bound N_F in fact satisfies it ($N_F \geq 2\delta_F$); this keeps the geometric interpretation of transitive-closure atoms in A sound when the same translation is applied again at the next iteration. Together, (1)–(3) discharge exactly the assumptions the symbolic VC makes about π_φ : that it establishes φ , and that it leaves all unreferenced objects undisturbed. $\square$

THEOREM 5.7 (END-TO-END SOUNDNESS). *Let $\mathcal{P}$ be a program realized from a relational CFG (Section 3.4). Suppose (1) every VC in $\text{VCgen}(\{\varphi_{\text{pre}}\} \mathcal{P} \{\varphi_{\text{post}}\})$ is valid, and (2) every synthesized controller π_φ in $\mathcal{P}$ satisfies the conditions of Theorem 5.6. Then every execution of $\mathcal{P}$ from a geometric state satisfying $\widehat{\varphi_{\text{pre}}}$ terminates in a state satisfying $\widehat{\varphi_{\text{post}}}$ without any collision.*

PROOF. By Theorem 5.6, condition (2) establishes that each block’s symbolic abstraction is fully justified: contract realization and frame preservation hold for every controller. Condition (1) and the standard soundness of wp-based VCgen (Section 5.3) then propagate these local guarantees globally: any execution from a state satisfying φ_{pre} must satisfy φ_{post} symbolically. Faithfulness of $\widehat{(\cdot)}$ (Definition 5.4) lifts this to the geometric level, and frame preservation at each block ensures no unreferenced object is disturbed at any step. $\square$

Theorem 5.6 justifies each symbolic abstraction locally by discharging contract realization and frame preservation against concrete motion; Theorem 5.7 propagates these local justifications globally, connecting symbolic verification of the entire program to the correctness of every physical execution. The loop invariant I is the single artifact bridging both levels: it must be strong enough to discharge the symbolic VCs (Section 5.4) and precise enough that $\widehat{I}$ covers the geometric states actually visited, making each collision query meaningful.

5.6 Scope

The translation $\widehat{(\cdot)}$ of Section 5.5 assumes, as in [20]’s treatment of singly-linked structures, that objects are related by reachability through a single “stacking” relation at a time; verifying tasks that simultaneously reason about two independent placement relations (e.g. a column and a row, Section D) requires translating one shared invariant over both relations, exactly as [20] extends their method to doubly-linked lists by duplicating the analysis for a second pointer field, as long as the two relations are not combined inside a single un-decomposed atom.

Unlike [20], where alternation is forced by inverting a function symbol out of a reachability relation, our only source of alternation is internal to a **get**-guarded condition’s body G , and Theorem 5.3 characterizes its effect precisely by occurrence polarity: every *positive* occurrence of a guard—a loop entered, an **if**’s then-branch taken—is EPR regardless of G ’s internal quantifier structure, because the guard’s witness and the surrounding formula’s own variables never become separated by an intervening universal once wp is expanded; every *negative* occurrence—a loop exited, an **if**’s else-branch taken—is EPR exactly when G is quantifier-free, and falls back to general quantifier-instantiation otherwise. This identifies a concrete target for any task where a decidability guarantee is required rather than empirical solver performance: phrase every guard—loop or branch alike—without an internal universal quantifier where possible—for instance, by maintaining a guard-relevant predicate incrementally as a primitive of $\mathcal{L}$ rather than spelling out its definition inline each time a guard is checked, in the spirit of how [20] maintains next^* incrementally rather than recomputing it from next . We leave a systematic treatment of which guards admit such a reformulation to future work; Section 6 reports that Z3 discharges Stack’s exit VC efficiently in practice despite the absence of an EPR guarantee.

5.7 Counterexample-Guided Refinement

The two-level verification architecture is not a one-shot check: when either level fails, the counterexample is recycled as constructive evidence for the synthesizer and invariant learner, forming a refinement loop that terminates when both levels certify the same artifact simultaneously. Algorithm 10 presents the complete synthesis–verification loop; the paragraphs below explain each feedback path.

Symbolic counterexamples. When any VC in $\text{VCgen}(\mathcal{P})$ is invalid, the SMT solver returns a model s_0 that falsifies the respective implication. Such a counterexample serves two roles.

Invariant refinement. If the failure occurs in a loop VC (preservation or exit), s_0 is a concrete loop-entry state that the current invariant candidate admits but exposes its weakness. The execution

Algorithm 6 VERIFIEDSYNTHESIS: the complete synthesis–verification loop of RoboVerify

```

1: procedure VERIFIEDSYNTHESIS( $\mathcal{M}, \mathcal{L}, \mathcal{D}, \varphi_{\text{pre}} \leadsto \varphi_{\text{post}}$ )
2:    $G \leftarrow (\{v?\}, \varphi_{\text{pre}} \rightarrow v? \rightarrow \varphi_{\text{post}}); \quad \mathcal{D}_V \leftarrow \{v? \mapsto \mathcal{D}\}; \quad \text{Pen} \leftarrow \lambda v. \{\}$ 
3:    $\triangleright \text{Pen}(v)$ : accumulated counterexample environments for block  $v$ 
4:    $\mathcal{P}, G \leftarrow \text{SYNTHESIZE}(\mathcal{M}, \mathcal{L}, G, \mathcal{D}_V) \quad \triangleright \text{Alg. 2: CFG refinement + straight-line synthesis}$ 
5:    $I \leftarrow \text{INVINFERENCE}(\mathcal{D}_V) \quad \triangleright \text{Sec. 4: initial invariant from demonstration traces}$ 
6:   while true do  $\triangleright \text{symbolic CEGIS: strengthen } I \text{ or refine } \mathcal{P}$ 
7:      $\text{ok}_s, s_0 \leftarrow \text{SYMBOLICVERIFY}(\mathcal{P}, I, \varphi_{\text{pre}}, \varphi_{\text{post}}) \quad \triangleright \text{Sec. 5.3: WP-based VC discharge}$ 
8:     if  $\text{ok}_s$  then
9:       break
10:    else if  $s_0$  is a loop-VC counterexample then
11:      Extend  $\mathcal{D}_V$  by traces from executing  $\mathcal{P}$  from loop-head state  $s_0$ 
12:    else  $\triangleright \mathcal{P}$  cannot reach  $\varphi_{\text{post}}$  from  $s_0$ 
13:      Request  $\mathcal{D}_{s_0}$  from user  $\triangleright$  demonstrations from  $s_0$ -configurations
14:      Extend  $\mathcal{D}_V$  by traces from  $\mathcal{D}_{s_0}$ 
15:       $\mathcal{P}, G \leftarrow \text{SYNTHESIZE}(\mathcal{M}, \mathcal{L}, G, \mathcal{D}_V)$ 
16:       $I \leftarrow \text{INVINFERENCE}(\mathcal{D}_V) \quad \triangleright \text{re-learn loop invariants from updated } \mathcal{D}_V$ 
17:    while true do  $\triangleright \text{motion CEGIS; symbolic proof holds throughout}$ 
18:       $\text{ok}_m, \text{CEx} \leftarrow \text{MOTIONVERIFY}(\mathcal{P}, I) \triangleright \text{Sec. 5.5: contract realization, frame preservation}$ 
19:      if  $\text{ok}_m$  then
20:        return  $\mathcal{P} \quad \triangleright \text{program is fully verified}$ 
21:      for all  $(v, \mu_k) \in \text{CEx}$  do
22:         $\text{Pen}(v) \leftarrow \text{Pen}(v) \cup \{\mu_k\} \quad \triangleright \text{concrete failure-causing object attribute values}$ 
23:        Apply Alg. 9 to synthesize a straight-line program for block  $v$  under objective (11)

```

traces of $\mathcal{P}$ from s_0 are added to $\mathcal{D}_V$ (line 11), and $\text{INVINFERENCE}(\mathcal{D}_V)$ re-learns from the augmented trace set. Convergence rests on a key property of the learning algorithm [43]: it never produces an invariant that is true for all possible loop-head states—even unseen ones—but fails to be inductive. Consequently, when the verifier finds s_0 as a counterexample, s_0 's execution traces reveal a loop-head state not yet in $\mathcal{D}_V$; adding it forces the next learned invariant to cover a strictly larger set of loop-head states, making monotone progress in the finite space Spec_{Inv} (unless the vocabulary is exhausted; Appendix L). Section 6 (RQ2) reports iteration counts empirically.

Program refinement. If the failing VC is the entry condition ($\varphi_{\text{pre}} \Rightarrow \text{wp}(\mathcal{P}, \varphi_{\text{post}})$, whose left-hand side contains no invariant symbol), the failure is not attributable to a weak invariant but to an uncovered case in the program structure: s_0 witnesses an initial configuration from which the synthesized program does not reach φ_{post} . This gap reveals a demonstration coverage problem: the synthesizer has seen only configurations up to a certain scale and has produced a straight-line or under-looped program that cannot generalize. Consider stacking n blocks from demonstrations of $n = 2$: the synthesizer produces a straight-line program, which the verifier refutes with s_0 containing $n = 3$ blocks. New demonstrations cannot be inferred from s_0 alone—they require the user to perform the task in an s_0 -configuration. The user is shown s_0 (line 14) and asked to provide demonstrations from such configurations ($\mathcal{D}_{s_0}$); these are added to $\mathcal{D}$. With 3-block demonstrations now in evidence, the synthesizer discovers a loop; an execution on $n = 4$ blocks is then added to $\mathcal{D}_V$ so that INVINFERENCE learns an invariant that is truly inductive over all n (Section 6).

Motion counterexamples and penalty-augmented synthesis. Each motion check is a quantifier-free NRA formula over the trajectory parameters θ and the positions of environment objects. The motion verifier either certifies the formula or returns a counterexample model μ_k (consistent with the invariant context) recording the concrete object positions that witnessed failure. μ_k is stored in $\text{Pen}(v)$ (line 22).

The straight-line synthesizer for block v is re-invoked with $\text{Pen}(v)$ as an additional input. For each candidate controller π' with parameters θ' , the motion VCs are instantiated with the concrete positions from each $\mu_k \in \text{Pen}(v)$, leaving θ' as the only free variables; the synthesizer penalizes candidates that fail these instantiated checks:

$$\text{KL}(\tau_{\pi'} \parallel \tau_{\mathcal{D}}) + \lambda \cdot |\{\mu_k \in \text{Pen}(v) \mid \pi'(\theta') \text{ fails the motion check at } \mu_k\}|. \quad (8)$$

Crucially, motion re-synthesis does not invalidate the symbolic proof: the relational summary φ_o of each block is unchanged, so the WP proof over the CFG remains valid throughout the motion CEGIS loop (line 17). `MOTIONVERIFY` is called again directly after re-synthesis; `SYMBOLICVERIFY` is never re-run within this loop.

Two properties hold uniformly across all three checks. *Monotone accumulation:* $\text{Pen}(v)$ only grows, so any θ satisfying all accumulated constraints is free of all witnessed failures. *Precision:* each μ_k is an actual Z3 model—a maximally adversarial geometry the invariant permits—so no constraint is imposed beyond what verification has actually witnessed.

Together, symbolic and motion counterexamples close the synthesis loop. Loop-VC failures drive invariant refinement from execution traces, fully automatically. Structural symbolic failures surface relational coverage gaps and request user demonstrations from s_0 -configurations. Motion failures accumulate Boolean NRA constraints into $\text{Pen}(v)$, steering CEM toward parameter regions that have not yet been witnessed to fail. Each counterexample is reused constructively: as a training example for the invariant learner, as a seed configuration for program re-synthesis, or as a geometric constraint for the CEM optimizer.

Limitations. Both loops may fail to terminate when the predicate vocabulary $\mathcal{L}$ is insufficient to express the reasoning the task requires.

Symbolic CEGIS. `INVINFERENCE`($\mathcal{D}_V$) learns the strongest invariant expressible in the current predicate basis. If that invariant is not inductive, `SYMBOLICVERIFY` returns a fresh counterexample s_0 ; its traces are added to $\mathcal{D}_V$ and `INVINFERENCE` re-runs. If the newly learned invariant is unchanged—because no predicate in $\mathcal{L}$ distinguishes s_0 from the states the invariant already admits—the CEGIS is stuck. The fix requires adding predicates to $\mathcal{L}$ that can express the missing distinction.

Motion CEGIS. $\text{Pen}(v)$ grows monotonically, but if no instruction sequence in the current vocabulary can simultaneously satisfy all accumulated environments in $\text{Pen}(v)$, CEM exhausts its budget on every iteration and the loop does not terminate. A concrete case: a block b sits too close to the partially assembled tower. Moving b directly would sweep through the tower, causing a collision; the motion verifier reports counterexample μ_k describing exactly this geometry. With only a Move primitive, no choice of offset δ avoids the collision—the task requires *pushing b clear first*, which in turn requires a relational predicate `Scattered`(b) (block b is not adjacent to any in-progress stack) in $\mathcal{L}$. Without `Scattered`, the invariant cannot condition on clearance, the CFG cannot guard the Move block on it, and CEM cannot discover the necessary two-phase strategy. The fix is to extend $\mathcal{L}$ with `Scattered` and provide demonstrations of the push-then-move strategy, enabling the synthesizer to introduce a guarded branch that handles the constrained geometry.

In both cases the accumulating counterexamples are diagnostic: they identify exactly which geometric or relational distinction the current vocabulary cannot express, and thus guide what predicate to add next.

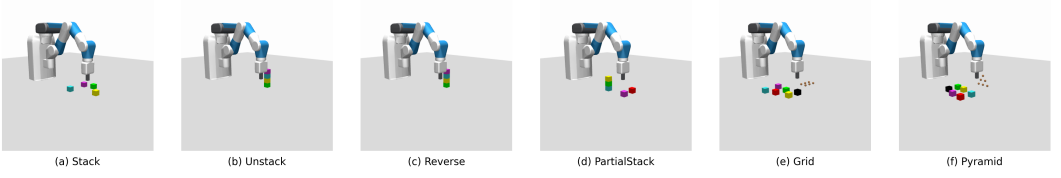


Fig. 10. Testing Environments for RoboVerify

6 Experiments

RoboVerify is implemented in Python and uses z3 [7] to verify the synthesized programs in both symbolic and motion level. Our experiments are designed to evaluate RoboVerify to answer the following research questions:

- (RQ1) Is RoboVerify able to learn effective and verifiable programs?
- (RQ2) Is the automatic loop invariant inference algorithm of RoboVerify able to learn from counterexamples?

Main Baselines. We consider the following baselines in our experiments:

- MCTS-style Best-First Search (MCTS): We use a best-first variant of Monte Carlo Tree Search (MCTS) based on the UCT selection rule [5, 22]. We maintain a global priority queue of frontier partial programs, where nodes are prioritized by the Upper Confidence Bound (UCB), combining the negative KL divergence heuristic score with an exploration bonus based on visit counts. After each expansion, visit counts are backpropagated to ancestors, and UCB priorities are maintained using lazy recomputation, whereby a node’s priority is recomputed only when dequeued and reinserted if its priority has become stale.
- Stochastic Synthesizer (SS): This baseline method adopts an MCMC-based stochastic search strategy similar to the MCMC component of RoboVerify. In addition to straight-line programs, it explores programs with while loops by allowing mutations that introduce, delete, and modify while loops and their guards.

Benchmarks. We evaluate RoboVerify and baselines with a suite of challenging robot manipulation benchmarks, consisting of the Stack, Unstack, Reverse, ReStack, Grid, Pyramid environments. In Stack (Fig. 10a), the goal is to assemble all scattered blocks into a tower with a designated base block b_0 as input.; Unstack environment shown in Fig. 10b requires disassemble the existing tower by placing all blocks on the countertop; Reverse environment shown in Fig. 10c where a robot needs to construct a new tower in the reverse block order of the existing tower with b_0 as the base block; ReStack environment shown in Fig. 10d where the goal is to use all blocks to constructing a tower with b_0 as the base block while the initial environment states may contain existing towers and b_0 maybe in the middle of such tower; Grid environment visualized in Fig. 10e requires the blocks are put into a grid format and the grid location are represented by the brown dots; and Pyramid environment shown in Fig. 10f where the goal is construct a pyramid at the specified location. Note that these dots are only markers to illustrate destination locations in the environment but not objects involved in collision. The precondition and postcondition specifications of the testing environments are shown in Appendix B.

6.1 RQ1: Learning Effective and Verifiable Programs

We train RoboVerify and all baseline methods in each environment using 5 random seeds and report the partial reward during training in Fig. 11. The partial reward for Stack, Unstack, Reverse are

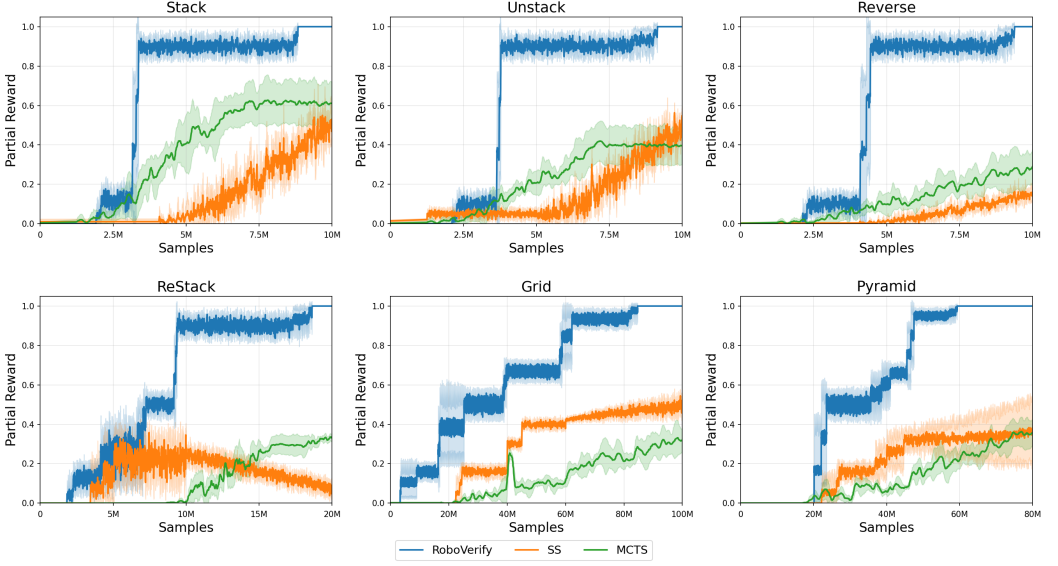


Fig. 11. Partial rewards for RoboVerify and baseline methods during training. The solid curve represents the mean across 5 random seeds. The shaded area represents the stand deviation.

assigned uniformly for each block that is successfully stacked on the base block b_0 , placed on the table, or reversed reversed, respectively. For ReStack, we assign a reward of 0.5 to each of the unstack and stack subtasks, with the reward further distributed uniformly among the blocks successfully unstacked or stacked. For Grid and Pyramid, partial rewards are assigned uniformly for each block at the goal location. We note that none of the tools use these partial rewards as learning feedback; they are only used for evaluating and visualizing training progress.

During the initial training phase, RoboVerify receives zero partial reward because the MCMC search initially focuses on discovering a suitable program structure that minimizes KL divergence while satisfying the identified subtask postconditions. Once MCMC discovers a promising program, it begins to obtain partial rewards from individual blocks. When RoboVerify introduces an appropriate loop structure, the partial reward increases rapidly. MCMC then continues refining the synthesized program while performing program verification until a verified program is found. This process may require a large number of samples because the set of programs with high partial rewards is larger than the set of programs that also satisfy the verification conditions. For example, for a program with a similar structure similar to that shown Fig. 5, except that the first move instruction is removed. Such a program can still achieve a low KL-divergence score after the cross-entropy method optimization and partially complete the task. However, it fails the motion level verification due to the potential collision possibility with the existing tower. Consequently, MCMC must continue exploring more additional candidate programs before discovering one that can pass the verification. In contrast, although both baselines achieve modest partial rewards, they are less sample-efficient than RoboVerify because they lack a principled mechanism for introducing loop structures and loop guards required to solve the complete task. Furthermore, the long, unsegmented demonstrations make the KL-divergence optimization noisy, reducing its effectiveness as a reliable feedback signal for program execution. The synthesized programs and inferred loop invariants are shown in Appendix C.

Table 2. Number of counterexample-guided learning iterations with automatic loop inference

Stack	Unstack	Reverse	ReStack	Grid	Pyramid
4	5	5	5	7	7

Table 3. Loop invariant learned for each counterexample-guided iterations for Stack environment

Iteration	Number of blocks in counterexample	Learned loop invariant
1	1	False
2	2	$\forall u, v. u \langle on^* \rangle v$ $\forall u, v. (\text{Higher}(v, b) \vee u \langle on^* \rangle v)$
3	3	$\wedge (b \langle on^* \rangle u \Leftrightarrow u \langle on^* \rangle b_0)$ $\wedge (\text{Scattered}(u, v) \Rightarrow \text{Higher}(u, v) \vee u \langle on^* \rangle v)$ $\wedge (\text{Higher}(u, v) \Rightarrow u \langle on^* \rangle v \vee \text{Scattered}(u, v))$
4	N/A	See Fig 6

During training, RoboVerify is provided with 15 demonstrations for each environment. While RoboVerify can identify the underlying CFG from fewer demonstrations, we observe that the estimated KL divergence becomes unreliable when the demonstrations cover only a small number of environment states.

6.2 RQ2: Learning Loop Invariant from Counterexamples

In RoboVerify, loop invariants are learned using the automatic loop inference algorithm from the environment states encountered at the beginning of each loop iteration. It's also useful to generate and learn from counterexamples if the current environment is not sufficient, for example, the number of blocks is too small to learn a loop invariant that is generalizable to arbitrary number of blocks. In this experiment, the automatic loop inference algorithm is provided with the correct program while initializing the loop invariant to False. The algorithm then attempts to verify the inferred loop invariant for arbitrary number of blocks. If verification fails, the underlying SMT solver generates a counterexample satisfying the environment initial conditions, choosing one with the smallest number of blocks. The correct program is subsequently executed on the generated environment, and the state at the beginning of each loop iteration are added to the training dataset, and this process repeats until a generalizable loop invariant is obtained.

Table 2 reports, for each environment, the number of counterexample-guided iterations required to learn a loop invariant that generalizes to arbitrary numbers of blocks within that environment. Table 3 reports the loop invariant learned at each counterexample-guided iteration for Stack.

As the difficulty of the environment increases, more counterexamples are required to learn a loop invariant that generalizes to arbitrary number of blocks. For example, in iteration 2 and 3 in Table 3, the inference algorithm learns spurious invariants that hold only initial states with a small number of blocks. After incorporating the generated counterexamples, the algorithm eventually learns a generalizable loop invariant in iteration 4.

7 Related Work

Component-based program synthesis. A long line of work synthesizes programs by composing components drawn from a library: oracle-guided synthesis of loop-free bit-vector programs [17, 21],

type-directed composition of API methods by graph reachability [12] and type-guided abstraction refinement [18], component-based synthesis with control structures [35], and sketch-based completion of partial programs [36]. Type-and-effect-guided synthesis [19] and conflict-driven learning [11] extend the core search to settings with stateful side effects. Deductive approaches target components with complete state contracts, synthesizing heap-manipulating programs from separation-logic specifications [33], programs from polymorphic refinement types [32], and abstract data types by refinement in a proof assistant [8]; tree-automata representations [23, 41] and constraint-annotated variants [27] provide compact encodings of the synthesis search space. The most directly related prior work is Cobalt [28], which synthesizes programs from libraries with complete Hoare-triple specifications—contracts that declare both the intended effects and implicit frame conditions stating what the operation leaves unchanged—using bidirectional WP/SP search with conflict-driven pruning. All of these share an implicit frame assumption: what a specification does not mention, the component does not affect—so a composition is correct by construction once the specifications line up. RoboVerify targets a setting where this assumption fails twice over. First, the components are *physically realized*: a robot primitive’s true semantics is a continuous trajectory through a shared workspace, and its local contract deliberately omits the frame conditions (which objects remain undisturbed) that sequential composition needs. Second, the code being composed is itself synthesized from demonstrations, so the relational summary it is meant to realize is an obligation to be discharged rather than an axiom. RoboVerify therefore synthesizes not just a composition but the artifacts that make composition sound: relational summaries at the CFG level, quantified loop invariants that carry frame information, and per-controller motion-level proofs of contract realization and frame preservation.

Inductive synthesis from examples and demonstrations. Synthesizing programs from observations of desired behavior dates back to trace-based construction of programs [3] and recursion discovery from input–output examples [37], and matured into programming-by-example systems such as version-space algebras [25] and FlashFill [16]. These systems induce loops or recursion from symbolic, discrete data, where segmentation is either given or trivial. Neural approaches extract programs from demonstration videos [38], but produce programs in a small, fixed DSL with no correctness guarantee. Closest to our work, RoboScribe [6] synthesizes iterative robot programs from raw, unsegmented demonstrations by abstraction refinement, inferring recurring subroutines from continuous state. RoboVerify differs in both mechanism and guarantee: structure discovery is organized around a relational CFG whose refinement proceeds in lockstep with demonstration partitioning; loops—including nested loops—are obtained by quotienting Kleene-encoded CFG fragments, with loop-carried variables, initializations, and updates read off the witnessing substitutions rather than heuristically recovered; and the synthesized program is verified end-to-end, symbolically for unboundedly many objects and geometrically down to the controllers’ swept volumes. No prior demonstration-driven synthesizer we are aware of produces a machine-checked correctness argument for its output.

Additional related work is discussed in Appendix M.

8 Conclusion

Component-based synthesis traditionally assumes that component specifications contain enough information for sound composition. This paper studied a class of components with intentionally local specifications—robot manipulation primitives—for which that assumption fundamentally fails: local contracts describe only the relational effects on directly referenced objects, omitting the frame conditions that guarantee unrelated relations are preserved, while their continuous executions may invalidate precisely those omitted assumptions. Restoring compositionality therefore becomes part

of the synthesis problem itself. Our key observation is that demonstrations already contain the missing information. Rather than treating demonstrations as behavior to imitate, RoboVerify interprets them as evidence of how locally specified components are intended to compose, recovering the global relational context absent from local contracts. In this sense, local contracts together with demonstrations yield composable specifications.

Concretely, RoboVerify synthesizes a symbolic relational program whose components are summarized solely by their relational effects, while simultaneously recovering the proof artifacts that enable modular reasoning, including relational summaries, refinement predicates, and quantified loop invariants. Verification mirrors this separation between local and global reasoning. Symbolic weakest-precondition reasoning establishes correctness over unbounded object sets, while motion-level verification discharges the contract-realization and frame-preservation obligations that symbolic reasoning necessarily assumes. A single learned relational invariant connects these two layers: it closes the symbolic induction and, when translated geometrically, provides the global environment description under which each component invocation is verified. Finally, an occurrence-polarity analysis characterizes when the resulting verification conditions lie within a decidable fragment, identifying quantifier-free guards as a design point for decidable verification.

One natural direction is extending the motion-level reasoning to richer physical domains. Our current verifier reasons about relatively simple geometric interactions, but the synthesis framework itself is independent of the underlying motion model. As more expressive geometric reasoning becomes available, it can be incorporated without changing the symbolic synthesis or verification architecture. More broadly, we believe the central principle developed here—recovering the missing frame conditions that make intentionally local specifications compositional from evidence of their intended use, then verifying that those conditions hold—extends well beyond robot manipulation to component-based synthesis wherever component specifications are necessarily incomplete.

References

- [1] Brenna D. Argall, Sonia Chernova, Manuela Veloso, and Brett Browning. 2009. A Survey of Robot Learning from Demonstration. *Robotics and Autonomous Systems* 57, 5 (2009), 469–483.
- [2] Osbert Bastani, Yewen Pu, and Armando Solar-Lezama. 2018. Verifiable Reinforcement Learning via Policy Extraction. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*. 2494–2504.
- [3] Alan W. Biermann and Ramachandran Krishnaswamy. 1976. Constructing Programs from Example Computations. *IEEE Transactions on Software Engineering* 2, 3 (1976), 141–153.
- [4] Rose Bohrer, Yong Kiam Tan, Stefan Mitsch, Magnus O. Myreen, and André Platzer. 2018. VeriPhy: Verified Controller Executables from Verified Cyber-Physical System Models. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 617–630. doi:10.1145/3192366.3192406
- [5] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 1 (2012), 1–43. doi:10.1109/TCIAIG.2012.2186810
- [6] Guofeng Cui, Yuning Wang, Wensen Mao, Yuanlin Duan, and He Zhu. 2025. Abstraction Refinement-Guided Program Synthesis for Robot Learning from Demonstrations. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 292 (Oct. 2025), 29 pages. doi:10.1145/3763070
- [7] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: an efficient SMT solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (Budapest, Hungary) (TACAS'08/ETAPS'08)*. Springer-Verlag, Berlin, Heidelberg, 337–340.
- [8] Benjamin Delaware, Clément Pit-Claudel, Jason Gross, and Adam Chlipala. 2015. Fiat: Deductive Synthesis of Abstract Data Types in a Proof Assistant. In *Proceedings of the 42nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 689–700. doi:10.1145/2676726.2677006
- [9] Quentin Delfosse, Wolfgang Stammer, Thomas Rothenbächer, Dwarak Vittal, and Kristian Kersting. 2023. Boosting Object Representation Learning via Motion and Object Continuity. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*.

- [10] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. 2007. The Daikon System for Dynamic Detection of Likely Invariants. *Science of Computer Programming* 69, 1–3 (2007), 35–45.
- [11] Yu Feng, Ruben Martins, Osbert Bastani, and Isil Dillig. 2018. Program Synthesis Using Conflict-Driven Learning. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 420–435. doi:10.1145/3192366.3192382
- [12] Yu Feng, Ruben Martins, Yuepeng Wang, Isil Dillig, and Thomas W. Reps. 2017. Component-Based Synthesis for Complex APIs. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 599–612. doi:10.1145/3009837.3009851
- [13] Cormac Flanagan and K. Rustan M. Leino. 2001. Houdini, an Annotation Assistant for ESC/Java. In *Formal Methods Europe (FME)*. 500–517.
- [14] Nathan Fulton, Stefan Mitsch, Jan-David Quesel, Marcus Völpe, and André Platzer. 2015. KeYmaera X: An Axiomatic Tactical Theorem Prover for Hybrid Systems. In *International Conference on Automated Deduction (CADE)*. 527–538.
- [15] Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. 2014. ICE: A Robust Framework for Learning Invariants. In *Computer Aided Verification (CAV)*. 69–87.
- [16] Sumit Gulwani. 2011. Automating String Processing in Spreadsheets Using Input-Output Examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 317–330. doi:10.1145/1926385.1926423
- [17] Sumit Gulwani, Susmit Jha, Ashish Tiwari, and Ramarathnam Venkatesan. 2011. Synthesis of Loop-Free Programs. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 62–73. doi:10.1145/1993498.1993506
- [18] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. 2019. Program Synthesis by Type-Guided Abstraction Refinement. *Proc. ACM Program. Lang.* 4, POPL, Article 12 (2019). doi:10.1145/3371080
- [19] Sankha Narayan Guria, Jeffrey S. Foster, and David Van Horn. 2021. RbSyn: Type- and Effect-Guided Program Synthesis. In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 344–358. doi:10.1145/3453483.3454048
- [20] Shachar Itzhaky, Anindya Banerjee, Neil Immerman, Aleksandar Nanevski, and Mooly Sagiv. 2013. Effectively-Propositional Reasoning about Reachability in Linked Data Structures. In *Proceedings of the 25th International Conference on Computer Aided Verification - Volume 8044 (Saint Petersburg, Russia) (CAV 2013)*. Springer-Verlag, Berlin, Heidelberg, 756–772.
- [21] Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. 2010. Oracle-Guided Component-Based Program Synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE)*. 215–224. doi:10.1145/1806799.1806833
- [22] Levente Kocsis and Csaba Szepesvári. 2006. Bandit Based Monte-Carlo Planning. In *Machine Learning: ECML 2006*, Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 282–293.
- [23] James Koppel, Zheng Guo, Edsko de Vries, Armando Solar-Lezama, and Nadia Polikarpova. 2022. Searching Entangled Program Spaces. *Proc. ACM Program. Lang.* 6, ICFP, Article 91 (2022). doi:10.1145/3547622
- [24] Hadas Kress-Gazit, Georgios E. Fainekos, and George J. Pappas. 2009. Temporal-Logic-Based Reactive Mission and Motion Planning. *IEEE Transactions on Robotics* 25, 6 (2009), 1370–1381.
- [25] Tessa Lau, Steven A. Wolfman, Pedro Domingos, and Daniel S. Weld. 2003. Programming by Demonstration Using Version Space Algebra. *Machine Learning* 53, 1–2 (2003), 111–156.
- [26] Zhixuan Lin, Yi-Fu Wu, Skand Vishwanath Peri, Weihao Sun, Gautam Singh, Fei Deng, Jindong Jiang, and Sungjin Ahn. 2020. SPACE: Unsupervised Object-Oriented Scene Representation via Spatial Attention and Decomposition. In *International Conference on Learning Representations (ICLR)*.
- [27] Anders Miltner, Ziteng Wang, Swarat Chaudhuri, and Isil Dillig. 2024. Relational Synthesis of Recursive Programs via Constraint Annotated Tree Automata. In *Computer Aided Verification (CAV)*. 41–63. doi:10.1007/978-3-031-65633-0_3
- [28] Ashish Mishra and Suresh Jagannathan. 2022. Specification-Guided Component-Based Synthesis from Effectful Libraries. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 147 (2022). doi:10.1145/3563310
- [29] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. 2016. Ivy: Safety Verification by Interactive Generalization. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 614–630. doi:10.1145/2908080.2908118
- [30] Sida Peng, Yuan Liu, Qixing Huang, Xiaowei Zhou, and Hujun Bao. 2019. PVNet: Pixel-wise Voting Network for 6DoF Pose Estimation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 4561–4570.
- [31] Ruzica Piskac, Leonardo de Moura, and Nikolaj Bjørner. 2010. Deciding Effectively Propositional Logic Using DPLL and Substitution Sets. *Journal of Automated Reasoning* 44, 4 (2010), 401–424. doi:10.1007/s10817-009-9161-6

- [32] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. 2016. Program Synthesis from Polymorphic Refinement Types. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 522–538. doi:10.1145/2908080.2908093
- [33] Nadia Polikarpova and Ilya Sergey. 2019. Structuring the Synthesis of Heap-Manipulating Programs. *Proc. ACM Program. Lang.* 3, POPL, Article 72 (2019). doi:10.1145/3290385
- [34] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 779–788.
- [35] Kensen Shi, Jacob Steinhardt, and Percy Liang. 2019. FrAngel: Component-Based Synthesis with Control Structures. *Proc. ACM Program. Lang.* 3, POPL, Article 73 (2019). doi:10.1145/3290386
- [36] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 404–415. doi:10.1145/1168857.1168907
- [37] Phillip D. Summers. 1977. A Methodology for LISP Program Construction from Examples. *J. ACM* 24, 1 (1977), 161–175.
- [38] Shao-Hua Sun, Hyeonwoo Noh, Sriram Somasundaram, and Joseph Lim. 2018. Neural Program Synthesis from Diverse Demonstration Videos. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 4790–4799.
- [39] Dweep Trivedi, Jesse Zhang, Shao-Hua Sun, and Joseph J. Lim. 2021. Learning to Synthesize Programs as Interpretable and Generalizable Policies. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*. 25146–25163.
- [40] Abhinav Verma, Vijayaraghavan Murali, Rishabh Singh, Pushmeet Kohli, and Swarat Chaudhuri. 2018. Programmatically Interpretable Reinforcement Learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 5045–5054.
- [41] Xinyu Wang, Isil Dillig, and Rishabh Singh. 2018. Program Synthesis using Abstraction Refinement. *Proc. ACM Program. Lang.* 2, POPL, Article 63 (2018). doi:10.1145/3158151
- [42] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. 2018. PoseCNN: A Convolutional Neural Network for 6D Object Pose Estimation in Cluttered Scenes. In *Robotics: Science and Systems (RSS)*.
- [43] He Zhu, Stephen Magill, and Suresh Jagannathan. 2016. Automatically Learning Shape Specifications. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '16)*. ACM, 491–507. doi:10.1145/2908080.2908089

A Predicate Definitions Axioms and Weakest Precondition Calculation

Table 4. Axioms for $\cdot\langle on^* \rangle\cdot$. **Transitivity**: if a is above b and b above c , then a is above c . **Reflexivity**: every box is above itself. **Common-predecessor linearity**: boxes below the same box are comparable. **Common-successor linearity**: boxes above the same box are comparable. **Antisymmetry**: mutual above-ness implies equality. **Table uniqueness** (if use_tbl): anything comparable with the table is the table. **Top existence** (if exists_top): every box has a unique top above it.

$$\begin{array}{l}
 \hline
 \forall x, y, c. x\langle on^* \rangle y \wedge y\langle on^* \rangle c \Rightarrow x\langle on^* \rangle c \\
 \forall x. x\langle on^* \rangle x \\
 \forall x, y, c. x\langle on^* \rangle y \wedge x\langle on^* \rangle c \Rightarrow y\langle on^* \rangle c \vee c\langle on^* \rangle y \\
 \forall x, y, c. x\langle on^* \rangle c \wedge y\langle on^* \rangle c \Rightarrow x\langle on^* \rangle y \vee y\langle on^* \rangle x \\
 \forall x, y. x\langle on^* \rangle y \Rightarrow (y\langle on^* \rangle x \Rightarrow x = y) \\
 \forall x. x\langle on^* \rangle \text{tbl} \vee \text{tbl}\langle on^* \rangle x \Rightarrow x = \text{tbl} \\
 \forall x. \exists y. \left(\forall c. c\langle on^* \rangle y \Rightarrow c = y \right) \wedge y\langle on^* \rangle x \\
 \hline
 \end{array}$$

Table 5. Axioms for $\text{Higher}(\cdot, \cdot)$. **Transitivity**: height order is transitive. **Reflexivity**: every box is at least as high as itself. **Common-predecessor linearity**: boxes below the same box are height-comparable. **Stack-above implies higher**: $x\langle on^* \rangle y$ implies $\text{Higher}(x, y)$. **No reverse height for strict stack-above**: if $x\langle on^* \rangle y$ and $x \neq y$, then not $\text{Higher}(y, x)$. **Table uniqueness** (if use_tbl): anything height-comparable with the table is the table.

$$\begin{array}{l}
 \hline
 \forall x, y, c. \text{Higher}(x, y) \wedge \text{Higher}(y, c) \Rightarrow \text{Higher}(x, c) \\
 \forall x. \text{Higher}(x, x) \\
 \forall x, y, c. \text{Higher}(x, y) \wedge \text{Higher}(x, c) \Rightarrow \text{Higher}(y, c) \vee \text{Higher}(c, y) \\
 \forall x, y. x\langle on^* \rangle y \Rightarrow \text{Higher}(x, y) \\
 \forall x, y. x\langle on^* \rangle y \wedge x \neq y \Rightarrow \neg \text{Higher}(y, x) \\
 \forall x. \text{Higher}(x, \text{tbl}) \vee \text{Higher}(\text{tbl}, x) \Rightarrow x = \text{tbl} \\
 \hline
 \end{array}$$

Table 6. Axioms for $\text{Scattered}(\cdot, \cdot)$. **Symmetry**: scattered-ness is mutual. **Irreflexivity**: no box is scattered with itself. **Constant along stack chains**: if $x\langle on^* \rangle y$, then x and y have the same scattered partners. **Table isolation** (if use_tbl): nothing is scattered with the table.

$$\begin{array}{l}
 \hline
 \forall x, y. \text{Scattered}(x, y) \Leftrightarrow \text{Scattered}(y, x) \\
 \forall x. \neg \text{Scattered}(x, x) \\
 \forall x, y, c. x\langle on^* \rangle y \Rightarrow \text{Scattered}(x, c) = \text{Scattered}(y, c) \\
 \forall x. \neg \text{Scattered}(x, \text{tbl}) \\
 \hline
 \end{array}$$

Table 7. Weakest-precondition rules for high-level verification. Here $b', b, \alpha, \beta, m, n, t \in \text{Block}$. For $\text{put}(b', b)$ and $\text{put}(b', \text{tbl})$, the operators $R_{\langle \text{on}^* \rangle}$, R_{Higher} , and $R_{\text{Scattered}}$ traverse Q recursively and replace only the corresponding atomic formulas as specified below. For a loop, $\text{wp}(S, Q)$ returns only the invariant; preservation and exit are checked by separate verification conditions.

Statement S	Weakest precondition $\text{wp}(S, Q)$
skip	$\text{wp}(\text{skip}, Q) = Q$
$S_1; S_2$	$\text{wp}(S_1; S_2, Q) = \text{wp}(S_1, \text{wp}(S_2, Q))$
$x := y$	$\text{wp}(x := y, Q) = Q[y/x]$
while B do S inv I	$\text{wp}(\text{while } B \text{ do } S \text{ inv } I, Q) = I$
$\text{put}(b', b)$	$\text{wp}(\text{put}(b', b), Q) = \neg b\langle \text{on}^* \rangle b' \wedge R_{\text{Scattered}}(R_{\text{Higher}}(R_{\langle \text{on}^* \rangle}(Q)))$
Atom replacement for $R_{\langle \text{on}^* \rangle}$ in $\text{wp}(\text{put}(b', b), Q)$:	
$\alpha\langle \text{on}^* \rangle \beta$	$\alpha\langle \text{on}^* \rangle \beta \vee (\alpha\langle \text{on}^* \rangle b' \wedge b\langle \text{on}^* \rangle \beta)$
Atom replacement for R_{Higher} in $\text{wp}(\text{put}(b', b), Q)$:	
$\text{Higher}(m, n)$	$(m \neq b' \wedge m \neq b \wedge n \neq b' \wedge n \neq b \wedge \text{Higher}(m, n))$ $\vee (m \neq b' \wedge m \neq b \wedge n = b' \wedge \exists t \in \text{Block}. t \neq n \wedge \text{Higher}(n, t) \wedge \text{Higher}(t, b))$ $\vee (m \neq b' \wedge m \neq b \wedge n = b \wedge \text{Higher}(m, n))$ $\vee (m = b \wedge n \neq b' \wedge n \neq b \wedge \text{Higher}(m, n))$ $\vee (m = b \wedge n = b)$ $\vee (m = b' \wedge n \neq b' \wedge n \neq b \wedge (\text{Higher}(b, n) \vee \forall t \in \text{Block}. \text{Higher}(n, b) \wedge (t \neq n \wedge \text{Higher}(n, t) \Rightarrow \text{Higher}(t, b))))$ $\vee (m = b' \wedge n = b)$ $\vee (m = b' \wedge n = b')$
Atom replacement for $R_{\text{Scattered}}$ in $\text{wp}(\text{put}(b', b), Q)$:	
$\text{Scattered}(m, n)$	$(m \neq b' \wedge m \neq b \wedge n \neq b' \wedge n \neq b \wedge \text{Scattered}(m, n))$ $\vee (m \neq b' \wedge m \neq b \wedge n = b \wedge \text{Scattered}(m, n))$ $\vee (m \neq b' \wedge m \neq b \wedge n = b' \wedge \text{Scattered}(m, b))$ $\vee (m = b' \wedge n \neq b' \wedge n \neq b \wedge \text{Scattered}(b, n))$ $\vee (m = b \wedge n \neq b' \wedge n \neq b \wedge \text{Scattered}(m, n))$
$\text{put}(b', \text{tbl})$	$\text{wp}(\text{put}(b', \text{tbl}), Q) = R_{\text{Scattered}}(R_{\text{Higher}}(R_{\langle \text{on}^* \rangle}(Q)))$
Atom replacement for $R_{\langle \text{on}^* \rangle}$ in $\text{wp}(\text{put}(b', \text{tbl}), Q)$:	
$\alpha\langle \text{on}^* \rangle \beta$	$\alpha\langle \text{on}^* \rangle \beta \wedge (\neg \alpha\langle \text{on}^* \rangle b' \vee \beta\langle \text{on}^* \rangle b')$
Atom replacement for R_{Higher} in $\text{wp}(\text{put}(b', \text{tbl}), Q)$:	
$\text{Higher}(m, n)$	$(m = b' \wedge n = b')$ $\vee (m \neq b' \wedge n \neq b' \wedge \text{Higher}(m, n))$ $\vee (m = b' \wedge n \neq b' \wedge \forall t \in \text{Block}. t \neq \text{tbl} \Rightarrow \text{Higher}(t, n))$ $\vee (m \neq b' \wedge m \neq \text{tbl} \wedge n = b' \wedge n \neq \text{tbl})$
Atom replacement for $R_{\text{Scattered}}$ in $\text{wp}(\text{put}(b', \text{tbl}), Q)$:	
$\text{Scattered}(m, n)$	$(m = b' \wedge n \neq b')$ $\vee (m \neq b' \wedge n = b')$ $\vee (m \neq b' \wedge n \neq b' \wedge \text{Scattered}(m, n))$

$$\begin{aligned}
u\langle on \rangle v &\triangleq |X[u] - X[v]| < 0.025 \wedge |Y[u] - Y[v]| < 0.025 \wedge (0.05 \leq Z[u] - Z[v] \leq 0.075) \\
\text{Higher}(u, v) &\triangleq Z[v] \leq Z[u] \\
\text{Scattered}(u, v) &\triangleq |X[u] - X[v]| \geq 0.1 \vee |Y[u] - Y[v]| \geq 0.1
\end{aligned}$$

Fig. 12. Numerical definition of predicates used in motion level verification.

B Task Specifications

The precondition and postcondition specifications for testing environments are shown Table 8. For the Reverse task, $\langle on_0^* \rangle$ describes the initial state of the environment and remain unchanged during program execution. For the Grid and Pyramid environment, we define a new type called Goal representing a target location of a block. These goals are connected by the $\langle next_c \rangle$ and $\langle next_r \rangle$ predicates to form the geometric shape of target locations [20]. $\text{reach}(x, y)$ is defined as $\exists z. x\langle next_c \rangle z \wedge z\langle next_r \rangle y$. The $\text{HasBlock}(g)$ predicate indicates whether a block is present at g 's location.

Table 8. Testing Environment Precondition and Postcondition Specifications

Environment	φ_{pre}	φ_{post}
Stack	$\forall m, n. (m = n \vee \neg n\langle on^* \rangle m)$	$\forall m. m\langle on^* \rangle b_0$
	$\wedge \forall m, n. \text{Higher}(n, m)$	
	$\wedge \forall m, n. (m \neq n \Rightarrow \text{Scattered}(m, n))$	
Unstack	$\forall m. m\langle on^* \rangle b_0$	$\forall m, n. (m = n \vee \neg m\langle on^* \rangle n)$
Reverse	$\forall m. m\langle on^* \rangle b_0$	$\forall m, n. m\langle on^* \rangle n \Leftrightarrow n\langle on_0^* \rangle m$
ReStack	$\forall m, n. ((m \neq n \wedge m\langle on^* \rangle n) \Rightarrow \text{Scattered}(m, n))$	$\forall m. m\langle on^* \rangle b_0$
Grid	$\forall g : \text{Goal}. \text{reach}(h_0, x) \Rightarrow \neg \text{HasBlock}(g)$	$\forall g : \text{Goal}. \text{reach}(h_0, x) \Rightarrow \text{HasBlock}(g)$
	$\wedge \forall m, n. \text{Scattered}(m, n)$	
Pyramid	$\forall g : \text{Goal}. \text{reach}(h_0, g) \Rightarrow \neg \text{HasBlock}(g)$	$\forall g : \text{Goal}. \text{reach}(h_0, g) \Rightarrow \text{HasBlock}(g)$
	$\wedge \forall m, n. \text{Scattered}(m, n)$	

C Synthesized Programs and Invariants

The synthesized programs by RoboVerify are shown in Fig. 13 to Fig. 17. The symbolic variables X_m and Z_m denote the blocks with the largest x -, z -coordinates, respectively, and are determined dynamically when the corresponding instruction is executed. Each target location (h_0, d, r) has associated (x, y, z) coordinates. Owing to their size, the learned invariants for Grid and Pyramid are omitted from the appendix. Their correctness will be examined during artifact evaluation.

D Illustration: Nested Loops

Consider a task in which a robot builds a grid structure: it places anchor blocks in a vertical column, and after each anchor extends a horizontal row of blocks rightward from it before placing the next anchor. Two predicates suffice: $\text{on}(x, y)$ (anchor x sits on y) and $\text{right}(x, y)$ (block x is placed to the right of y), established by primitive actions $\pi_{\text{place-on}}$ and $\pi_{\text{place-right}}$.

Two-loop program before outer quotienting. Suppose the demonstrations exhibit two complete outer iterations: anchor i placed on base b_0 with a row extending from i , then anchor k placed on i with a row extending from k . After REFINECFG splits the joint trajectory into a chain of blocks and

```

2206 def stack( $b_0$ ):
2207    $b \leftarrow b_0$ 
2208   while  $\exists b'$ .
2209     ( $\forall n. n = b' \vee \neg n\langle on^* \rangle b'$ )
2210      $\wedge b' \neq b$ :
2211       {
2212          $I_{stack} \triangleq \forall u, v. (Higher(u, v) \Rightarrow u\langle on^* \rangle v \vee Scattered(u, v))$ 
2213          $\wedge (u\langle on^* \rangle b_0 \Leftrightarrow b\langle on^* \rangle u)$ 
2214          $\wedge (Higher(b_0, v) \Rightarrow Higher(u, v))$ 
2215          $\wedge (Higher(u, v) \Rightarrow u\langle on^* \rangle v \vee Higher(b_0, v))$ 
2216          $\wedge (Higher(b_0, u) \vee u\langle on^* \rangle b_0)$ 
2217       }
2218     Move( $b', b', b, (0, 0, 0.15)$ )
2219     Pick( $b'$ )
2220     Move( $b', b', b, (0, 0, 0.15)$ )
2221     Move( $b_0, b_0, b, (0.0, 0.0, 0.15)$ )
2222     Move( $b_0, b_0, b, (0.0, 0.0, 0.05)$ )
2223     Release( $b', 0.15$ )
2224      $b \leftarrow b'$ 

```

Fig. 13. Synthesized Stack Program

```

2225 while  $\exists b'$ .
2226   ( $\forall n. n \neq tbl \wedge (b_0\langle on^* \rangle b' \vee n\langle on^* \rangle b_0)$ 
2227    $\Rightarrow b'\langle on^* \rangle n$ :
2228     {
2229        $I_{unstack} \triangleq \forall u, v. (Higher(u, v) \Rightarrow Scattered(u, v) \vee u\langle on^* \rangle v)$ 
2230       ( $Higher(b, u) \Rightarrow Higher(u, b) \vee u\langle on^* \rangle b_0$ )
2231       ( $Higher(u, b) \Rightarrow Higher(b, u) \vee u\langle on^* \rangle b_0$ )
2232       ( $Higher(b, u) \wedge u\langle on^* \rangle b_0 \Rightarrow b_0 = u$ )
2233       ( $u\langle on^* \rangle b_0 \vee Higher(u, tbl) \vee Higher(b_0, u)$ )
2234     }
2235     Move( $b', b', b', (0, 0, 0.15)$ )
2236     Pick( $b'$ )
2237     Move( $b', b', b', (0, 0, 0.15)$ )
2238     Move( $X_m, b', b', (0.15, 0.0, 0.15)$ )
2239     Move( $X_m, b', b_0, (0.0, 0.0, 0.0)$ )
2240     Release( $Z_m, 0.15$ )

```

Fig. 14. Synthesized Unstack Program

```

2241 def reverse( $\mu, b_0$ ):
2242    $b \leftarrow tbl$ 
2243   while  $\exists b'$ .
2244     ( $\forall n. n \neq tbl \wedge (b_0\langle on^* \rangle b' \vee n\langle on^* \rangle b_0)$ 
2245      $\Rightarrow b'\langle on^* \rangle n$ :
2246       {
2247          $I_{reverse} \triangleq \forall u, v. (u\langle on_0^* \rangle v \Rightarrow u\langle on_0^* \rangle b \vee u\langle on^* \rangle v)$ 
2248         ( $v\langle on_0^* \rangle u \wedge u\langle on_0^* \rangle b \Rightarrow u\langle on^* \rangle v$ )
2249         ( $u\langle on^* \rangle v \Rightarrow b\langle on^* \rangle u \vee u\langle on_0^* \rangle v$ )
2250         ( $b\langle on^* \rangle u \wedge u\langle on^* \rangle v \Rightarrow v\langle on_0^* \rangle u$ )
2251         ( $u\langle on_0^* \rangle b \vee u = tbl \vee u\langle on^* \rangle b_0$ )
2252         ( $b\langle on^* \rangle u \Rightarrow \neg u\langle on^* \rangle b_0$ )
2253         ( $u\langle on^* \rangle b_0 \Rightarrow \neg u\langle on_0^* \rangle tbl$ )
2254         ( $tbl\langle on^* \rangle b \vee b\langle on_0^* \rangle b_0$ )
2255       }
2256     Move( $b', b', Z_m, (0, 0, 0.15)$ )
2257     Pick( $b'$ )
2258     Move( $b', b', Z_m, (0, 0, 0.15)$ )
2259     Move( $b, b, Z_m, (0.0, 0.0, 0.15)$ )
2260     Move( $b, b, b, (0.0, 0.0, 0.05)$ )
2261     Release( $Z_m, 0.15$ )
2262      $b \leftarrow b'$ 

```

Fig. 15. Synthesized Reverse Program

```

2255 def restack( $\mu, b_0$ ):
2256   while  $\exists b'$ .
2257     ( $\forall n. n = b' \vee \neg n\langle on^* \rangle b'$ )
2258      $\wedge b'\langle on^* \rangle b_0$ 
2259     {
2260        $I_1 \triangleq \forall u, v. (Higher(u, v) \Rightarrow Scattered(u, v) \vee u\langle on^* \rangle v)$ 
2261        $(Scattered(u, v) \Rightarrow u\langle on^* \rangle b_0 \vee Higher(u, b_0) \vee Higher(b_0, u))$ 
2262        $(Scattered(u, v) \Rightarrow u\langle on^* \rangle b_0 \vee Higher(b_0, v) \vee v\langle on^* \rangle b_0 \vee Higher(b_0, u))$ 
2263        $(v\langle on^* \rangle b_0 \wedge u\langle on^* \rangle b_0 \Rightarrow \neg Higher(v, tbl))$ 
2264        $(u\langle on^* \rangle b_0 \wedge \neg Scattered(u, v) \Rightarrow b_0\langle on^* \rangle v \vee Higher(v, tbl) \vee v\langle on^* \rangle b_0)$ 
2265        $(u \neq v \wedge v\langle on^* \rangle u \Rightarrow Higher(u, b_0) \vee b_0\langle on^* \rangle u)$ 
2266        $(b_0\langle on^* \rangle v \wedge b_0\langle on^* \rangle u \Rightarrow v\langle on^* \rangle u \vee u = b_0)$ 
2267        $(Higher(u, v) \Rightarrow \neg Higher(b_0, v) \vee Higher(v, b_0) \vee \neg Higher(b_0, u))$ 
2268        $(Higher(u, v) \Rightarrow \neg u\langle on^* \rangle b_0 \vee \neg Higher(v, b_0) \vee v\langle on^* \rangle u \vee Higher(b_0, u))$ 
2269        $(Higher(u, v) \Rightarrow \neg Higher(v, b_0) \vee \neg Higher(u, b_0) \vee Higher(v, u))$ 
2270     }
2271   Move( $b', b', Z_m, (0, 0, 0, 0.15)$ )
2272   Pick( $b'$ )
2273   Move( $b', b', Z_m, (0, 0, 0, 0.15)$ )
2274   Move( $X_m, b', Z_m, (0.15, 0.0, 0.15)$ )
2275   Move( $X_m, b', tbl, (0.0, 0.0, 0.0)$ )
2276   Release( $Z_m, 0.15$ )
2277    $b \leftarrow b_0$ 
2278   while  $\exists b'$ .
2279     ( $\forall n. n = b' \vee \neg n\langle on^* \rangle b'$ )
2280      $\wedge b' \neq b$ :
2281     {
2282        $I_2 \triangleq \forall u, v. (Higher(u, v) \Rightarrow u\langle on^* \rangle v \vee Scattered(u, v))$ 
2283        $(b\langle on^* \rangle u \Leftrightarrow b\langle on^* \rangle x)$ 
2284        $(Higher(b_0, v) \Rightarrow Higher(u, v))$ 
2285     }
2286   Move( $b', b', Z_m, (0, 0, 0, 0.15)$ )
2287   Pick( $b'$ )
2288   Move( $b', b', Z_m, (0, 0, 0, 0.15)$ )
2289   Move( $b_0, b_0, Z_m, (0.0, 0.0, 0.15)$ )
2290   Move( $b_0, b_0, b, (0.0, 0.0, 0.05)$ )
2291   Release( $Z_m, 0.15$ )
2292    $b \leftarrow b'$ 

```

Fig. 16. Synthesized ReStack Program

```

2287 def grid( $\mu, h_0$ ):
2288    $d \leftarrow h_0$ 
2289   while  $d \neq \text{NULL}$ 
2290      $r \leftarrow d$ 
2291     while  $\exists b', \neg \text{AtGoal}(b') \wedge r \neq \text{NULL}$ :
2292       Move( $b', b', r, (0, 0, 0, 0.15)$ )
2293       Pick( $b'$ )
2294       Move( $b', b', r, (0, 0, 0, 0.15)$ )
2295       Move( $r, r, r, (0.0, 0.0, 0.15)$ )
2296       Move( $r, r, r, (0.0, 0.0, 0.0)$ )
2297       Release( $r, 0.15$ )
2298        $r \leftarrow r.next_r$ 
2299      $d \leftarrow d.next_c$ 

```

Fig. 17. Synthesized Grid Program

Phase 1 of QUOTIENT runs at $l = 1$, the two right-pairs are each collapsed into an inner **while**

```

def pyramid( $\mu, h_0$ ):
   $d \leftarrow h_0$ 
  while  $d \neq \text{NULL}$ 
     $r \leftarrow d$ 
    while  $\exists b', \neg \text{AtGoal}(b') \wedge r \neq \text{NULL}$ :
      Move( $b', b', r, (0, 0, 0.15)$ )
      Pick( $b'$ )
      Move( $b', b', r, (0, 0, 0.15)$ )
      Move( $r, r, r, (0.0, 0.0, 0.15)$ )
      Move( $r, r, r, (0.0, 0.0, 0.0)$ )
      Release( $r, 0.15$ )
       $r \leftarrow r.\text{next}_r$ 
     $d \leftarrow d.\text{next}_c$ 

```

Fig. 18. Synthesized Pyramid Program

loop. The result is the following program and its corresponding CFG (Figure 19):

```

def Build( $b_0$ ):
   $i \leftarrow \text{get}(\exists y. \text{top}(y))$ ;
   $\pi_{\text{place-on}}(i, b_0)$ ; [on( $i, b_0$ )]
   $x \leftarrow i$ ;
  while  $(\exists j'. \text{top}(j') \wedge \text{adj}(j', x))$  do
    {  $\pi_{\text{place-right}}(j', x)$ ;  $x \leftarrow j'$  }; [right( $j', x$ )]
   $k \leftarrow \text{get}(\exists y. \text{top}(y))$ ;
   $\pi_{\text{place-on}}(k, i)$ ; [on( $k, i$ )]
   $x \leftarrow k$ ;
  while  $(\exists j'. \text{top}(j') \wedge \text{adj}(j', x))$  do
    {  $\pi_{\text{place-right}}(j', x)$ ;  $x \leftarrow j'$  }; [right( $j', x$ )]
  ...

```

This program already executes correctly on the demonstrated instances, and each individual **while** already generalizes to rows of arbitrary length. What remains unrecognized is the repetition across anchor placements: i and k are syntactically unrelated variables, and the outer structure is unbounded straight-line code rather than a loop.

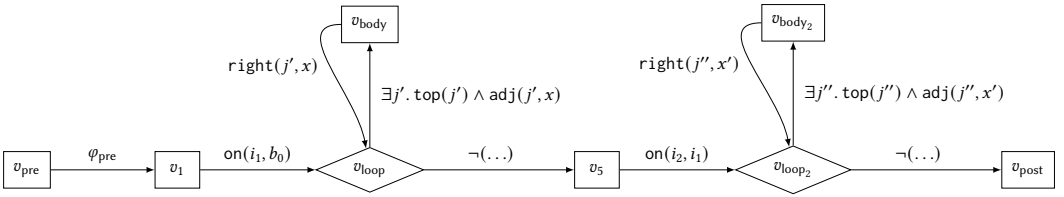


Fig. 19. Two-loop CFG produced by inner-loop quotienting: one loop per right-pair, connected by the one-time $\text{on}(i_2, i_1)$ edge at v_5 . The outer repetition is not yet recognized. Negated guards abbreviated $\neg(\dots)$.

Outer quotienting. Phase 1 of QUOTIENT now operates on the top-level sequence of Figure 19, treating each inner loop as an atomic unit:

$$(u_1, \dots, u_6) = (v_{\text{pre}}, v_1, v_{\text{loop}}, v_5, v_{\text{loop}_2}, v_{\text{post}}).$$

At $l = 1$ no adjacent pair matches: a basic block and a loop have different Kleene shapes. At $l = 2$, the two length-2 sub-sequences $f_1 = (v_1, v_{\text{loop}})$ and $f_2 = (v_5, v_{\text{loop}_2})$ encode as Kleene words

$$w_1 = \text{on}(\underline{i}_1, b_0) \cdot (\text{adj}(\underline{j}', i_1) \cdot \text{right}(j', i_1))^* \cdot \neg \text{adj}(j', i_1),$$

$$w_2 = \text{on}(\underline{i_2}, i_1) \cdot (\text{adj}(\underline{j'}, i_2) \cdot \text{right}(j', i_2))^* \cdot \neg \text{adj}(j', i_2).$$

Both match the common template

$$W(x, x') = \text{on}(\underline{x'}, x) \cdot (\text{adj}(\underline{j'}, x') \cdot \text{right}(j', x'))^* \cdot \neg \text{adj}(j', x')$$

under $\sigma_1 = \{x \mapsto b_0, x' \mapsto i_1\}$ and $\sigma_2 = \{x \mapsto i_1, x' \mapsto i_2\}$. The starred subterms of w_1 and w_2 share identical Kleene shape, differing only in the substituted argument; this comparison inspects only the shape of the already-quotiented inner loop's encoding, never how its actions are realized. Definition 3.1 merges the two fragments into the single cyclic CFG of Figure 20: v_1 is identified with the fresh outer guard v_{anchor} , and the edge following the second fragment closes a back-edge into v_{anchor} .

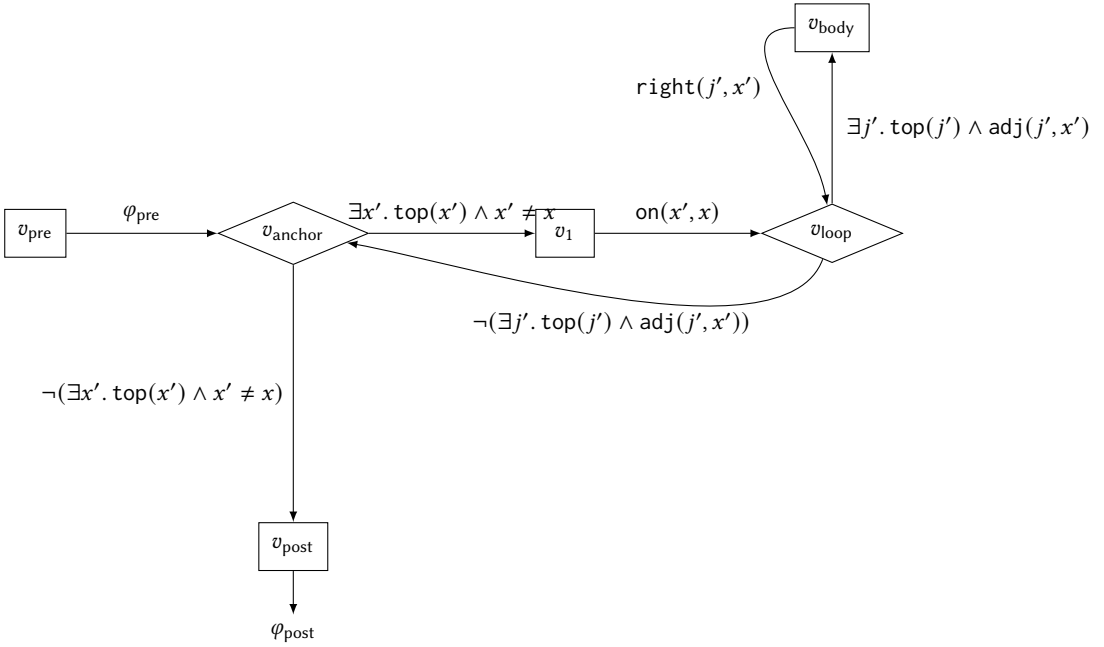


Fig. 20. Merged CFG: the two acyclic fragments of Figure 19 are identified into one cyclic node v_{anchor} . The inner loop $v_{\text{loop}}/v_{\text{body}}$ is nested inside one outer iteration; its exit edge returns to v_{anchor} , closing the outer cycle. Quotienting then reads the outer init and update off σ_1, σ_2 .

Initialization, update, and the final program. Reading off σ_1 and σ_2 from above:

$$\text{initialization: } b \leftarrow \sigma_1(x) = b_0, \quad \text{update: } b \leftarrow \sigma_1^{-1}(\sigma_2(x)) = x'.$$

Substituting these into the merged CFG of Figure 20 yields the final doubly-nested program—identical to what one would write by hand:

```

def Build( $b_0$ ):
   $b \leftarrow b_0$ ;
  while ( $\exists i. \text{top}(i) \wedge i \neq b$ ) do
  {
     $\pi_{\text{place-on}}(i, b)$ ;                                [on( $i, b$ )]
     $x \leftarrow i$ ;
    while ( $\exists j'. \text{top}(j') \wedge \text{adj}(j', x)$ ) do
      {  $\pi_{\text{place-right}}(j', x)$ ;  $x \leftarrow j'$ ; [right( $j', x$ )]
       $b \leftarrow i$ ;
    }
  }

```

The same substitution-composition arithmetic that extracted the inner loop's initialization and update now operates on the compound template $W(x, x')$, whose starred subterm is itself the Kleene encoding of an already-formed loop. No special case for nesting is needed: Phase 1 simply operates at length $l = 2$ instead of $l = 1$.

E Motion-Level Verification: Propagation Rules and Stack Illustration

Symbolic context by forward propagation. The VCs of Section 5.5 used the loop-body context $A = \widehat{I \wedge G(v)}[\sigma_0]$ as a fixed example. In general, the geometric context for any basic block in the relational CFG is obtained by a single forward traversal, formalized via the judgment

$$\langle A, \sigma \rangle \vdash S \rightsquigarrow \langle A', \sigma' \rangle,$$

where A is a quantifier-free NRA formula representing the current geometric assumption, σ is the current symbolic state, and relational formulas Φ are instantiated via $\widehat{\Phi}[\sigma]$ at the point of introduction. $\text{fresh}()$ denotes a fresh symbolic state with unconstrained attribute arrays. The rules are syntax-directed:

$$\text{(BLOCK)} \quad \frac{\text{discharge VCs for } \pi_\varphi \text{ under } A}{\langle A, \sigma \rangle \vdash \pi_\varphi \rightsquigarrow \langle A, \text{exec}(\pi_\varphi, \sigma) \rangle}$$

A is unchanged; the state advances by symbolic execution.

$$\text{(SEQ)} \quad \frac{\langle A, \sigma \rangle \vdash S_1 \rightsquigarrow \langle A_1, \sigma_1 \rangle \quad \langle A_1, \sigma_1 \rangle \vdash S_2 \rightsquigarrow \langle A_2, \sigma_2 \rangle}{\langle A, \sigma \rangle \vdash S_1; S_2 \rightsquigarrow \langle A_2, \sigma_2 \rangle}$$

Both A and σ thread forward; no reset occurs between consecutive blocks.

$$\text{(WHILE)} \quad \frac{\sigma^* = \text{fresh()} \quad \langle \widehat{I \wedge G(v)}[\sigma^*], \sigma^* \rangle \vdash S \rightsquigarrow \langle _, _ \rangle}{\langle A, \sigma \rangle \vdash \text{while}^I \text{ get}(\exists v. G(v)) \text{ do } S \rightsquigarrow \langle \widehat{I \wedge \neg \exists v. G}[\sigma^{**}], \sigma^{**} \rangle}$$

where $\sigma^{**} = \text{fresh}()$. The loop body is verified under $\widehat{I \wedge G(v)}[\sigma^*]$; post-loop, a fresh state carries $\widehat{I \wedge \neg \exists v. G}[\sigma^{**}]$. The invariant I is established separately by VCgen (Section 5.3).

$$\text{(IF)} \quad \frac{\langle A \wedge \widehat{C}[\sigma], \sigma \rangle \vdash S_1 \rightsquigarrow \langle A_1, \sigma_1 \rangle \quad \langle A \wedge \neg \widehat{C}[\sigma], \sigma \rangle \vdash S_2 \rightsquigarrow \langle A_2, \sigma_2 \rangle}{\langle A, \sigma \rangle \vdash \text{if } C \text{ then } S_1 \text{ else } S_2 \rightsquigarrow \langle A_1 \vee A_2, \text{fresh}() \rangle}$$

Branching instantiates C at the current σ ; the join uses a fresh state. Verification begins as $\langle \widehat{\varphi_{\text{pre}}}[\sigma_0], \sigma_0 \rangle \vdash S_{\text{program}} \rightsquigarrow \langle _, _ \rangle$, where $\sigma_0 = \text{fresh}()$.

$$\mathcal{O} \equiv \{b_0, b, b', \text{sym}\}, \quad \widehat{I_{\text{stack}}\mathcal{O}}[\sigma_0] \equiv \bigwedge_{o_1, o_2 \in \mathcal{O}} \widehat{\text{inv}(o_1, o_2)}[\sigma_0].$$

Fig. 21. Stack instantiation of the general finite expansion (Definition 5.4, formula (5)). Constants b_0 (base), b (tower top), b' (get-witness), sym (collision witness). Each $\widehat{\text{inv}(o_1, o_2)}$ instantiates $I_{\text{stack}} \equiv \forall u, v. \text{inv}(u, v)$ at (o_1, o_2) and applies $\widehat{(\cdot)}$.

Symbolic VCs for the Stack loop. At the symbolic level, VCgen generates three obligations for the Stack loop (Fig. 4), with invariant I_{stack} (Fig. 6) and guard $\exists b'. G(b', b)$, $G(b', b) \equiv (\forall n. n = b' \vee \neg n \langle \text{on}^* \rangle b') \wedge b' \neq b$. **Preservation** reduces, by Section 5.4, to $\exists b', x. \forall u. (I_{\text{stack}}$'s matrix $\wedge G(b', b) \wedge \neg \text{wp}(\pi(\mu, b', b); b \leftarrow b', I_{\text{stack}})$'s matrix) in negated form—EPR, regardless of G 's internal $\forall n$, exactly as Theorem 5.3 guarantees for any positive occurrence. Two facts close the case: $I_{\text{stack}} \Rightarrow b \langle \text{on}^* \rangle b_0$ (instantiating at $u := b$ with Reflexivity) and G 's $\forall n$ forces $u \langle \text{on}^* \rangle b' \Leftrightarrow u = b'$ for every u . **Exit** falls outside EPR (the negated exit VC has genuine $\exists^* \forall^* \exists^*$ shape) but Z3 discharges it directly under the background axioms of Section ??.

Illustration: motion-level verification (Stack task). We illustrate both motion checks on the Stack loop body. At the motion level, v instantiates as b' and w as sym ; writing $G(b', b) = \forall n. G_0(b', b, n)$ with $G_0 \equiv (n = b' \vee \neg n \langle \text{on}^* \rangle b') \wedge b' \neq b$, the instantiated guard is $\widehat{G}_O(b') \equiv \bigwedge_{a \in \mathcal{O}} (a = b' \vee \neg a \langle \text{on}^* \rangle b') \wedge b' \neq b$.

Incorrect candidate (rejected). Consider a 6-step candidate with the naive alignment step 5:

$$\pi_\varphi^\times = [\text{Move}(b', b', b', (0, 0, 0.15)), \text{Pick}(b'), \text{Move}(b', b', b, (0, 0, 0.15)), \\ \text{Move}(b, b, b, (0, 0, 0.15)), \text{Move}(\mathbf{b}, \mathbf{b}, \mathbf{b}, (\mathbf{0}, \mathbf{0}, \mathbf{0.05})), \text{Release}(b', 0.15)].$$

Steps 1–4 pre-position; step 5 (bold) lowers to release height targeting $(X[b], Y[b], Z[b] + 0.05)$; step 6 releases.

Collision check (step 4). Step 4, $\text{Move}(b, b, b, (0, 0, 0.15))$, carries b' horizontally from above its original position to above b . The in-scope constants are $\mathcal{O} = \{b_0, b, b', \text{sym}\}$ (Figure 21). With $h_i = b'$, formula (5) concretizes to

$$\exists b'. \exists \text{sym}. \widehat{I_{\text{stack}}\mathcal{O}}[\sigma_0] \wedge \widehat{G}_O(b, b')[\sigma_0] \wedge \text{collide}_i(\text{sym}, b') \wedge \text{sym} \neq b'.$$

Z3 proves this unsatisfiable: $\widehat{I_{\text{stack}}}$ forces every block other than b' to be in the tower below b or geometrically scattered, leaving no object in the swept volume for any guard-valid b' .

Alignment VC (step 5). Step 5 stores $\mathbf{X}[b'] = \mathbf{X}[b] + \eta_{m,x}$ via Δ . The alignment VC (6) requires $|\mathbf{X}[b'] - \mathbf{X}[b_0]| \leq \delta_X$. Substituting: $|\mathbf{X}[b] + \eta_{m,x} - \mathbf{X}[b_0]| \leq |\mathbf{X}[b] - \mathbf{X}[b_0]| + \varepsilon_m$. Since $\widehat{I_{\text{stack}}}$ includes $\text{on}^*(b, b_0)$, which carries chain-root bound $|\mathbf{X}[b] - \mathbf{X}[b_0]| \leq N_X$, the total $N_X + \varepsilon_m \geq 2\delta_X + \varepsilon_m > \delta_X$. Z3 finds a satisfying assignment witnessing the violation; $\pi_\varphi^\times$ is rejected.

Corrected program (accepted). The synthesizer replaces step 5 with $\text{Move}(b_0, b_0, b, (0, 0, 0.05))$:

$$\pi_\varphi = [\text{Move}(b', b', b', (0, 0, 0.15)), \text{Pick}(b'), \text{Move}(b', b', b, (0, 0, 0.15)), \\ \text{Move}(b, b, b, (0, 0, 0.15)), \text{Move}(\mathbf{b}_0, \mathbf{b}_0, \mathbf{b}, (\mathbf{0}, \mathbf{0}, \mathbf{0.05})), \text{Release}(b', 0.15)].$$

Collision check. The collision query for step 5 is unsatisfiable by the same invariant argument as step 4.

Alignment VC. Step 5 targets $(\mathbf{X}[b_0], \mathbf{Y}[b_0], \mathbf{Z}[b] + 0.05)$, storing $\mathbf{X}[b'] = \mathbf{X}[b_0] + \eta_{m,x}$. Thus $|\mathbf{X}[b'] - \mathbf{X}[b_0]| = |\eta_{m,x}| \leq \varepsilon_m \leq \delta_X$; Z3 discharges this without appeal to $\widehat{I_{\text{stack}}}$. By Lemma J.1, the pairwise separation across the whole stack is at most $2\delta_X$ in X and $2\delta_Y$ in Y .

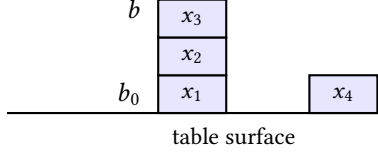


Fig. 22. Stacking scene: three blocks x_1, x_2, x_3 form a tower with base $b_0=x_1$ and top $b=x_3$, and an extra block x_4 lies on the table surface.

PROOF OF THEOREM 5.6. Condition (1) establishes contract realization. Each primitive’s noise perturbation (η_g, η_m , or η_r) appears in p_i^{post} as a free NRA variable; Δ_i stores p_i^{post} into the position arrays, so η_i propagates into σ_n . Check (4) is an NRA formula over these free variables; Z3’s unsatisfiability check certifies $\widehat{\varphi}[\sigma_n]$ for every Release position within the noise bound. Condition (2) provides frame preservation: η_i appears inside $c_i(t)$, so formula (5) is NRA over $\eta_i \in [-\varepsilon_i, \varepsilon_i]^3$; its unsatisfiability certifies that no actual trajectory within the noise bound produces a collision; since $(\widehat{\cdot})$ faithfully translates the block’s relational context (Definition 5.4), this covers every geometric state the context permits. Condition (3) certifies the per-step hypothesis of Lemma J.1 for each element newly placed by π_φ , keeping the geometric interpretation of transitive-closure atoms in A sound at the next iteration. Together, (1)–(3) discharge exactly the assumptions the symbolic VC makes about π_φ . $\square$

PROOF OF THEOREM 5.7. By Theorem 5.6, condition (2) establishes that each block’s symbolic abstraction is fully justified: contract realization and frame preservation hold for every controller. Condition (1) and the standard soundness of wp-based VCgen (Section 5.3) propagate these local guarantees globally: any execution from a state satisfying φ_{pre} must satisfy φ_{post} symbolically. Faithfulness of $(\widehat{\cdot})$ (Definition 5.4) lifts this to the geometric level, and frame preservation ensures no unreferenced object is disturbed at any step. $\square$

F Loop Invariant Inference: Worked Example

Learning Procedure. We illustrate the proposed learning procedure on the STACK task (Fig. 10). For clarity, we consider the predicate set $\Omega_{\text{inv}} = \{a\langle \text{on}^* \rangle b_0, y\langle \text{on}^* \rangle a, a\langle \text{on}^* \rangle b, a=y, b=a\}$, where a and y are universally quantified variables. Here, $a\langle \text{on}^* \rangle b_0$ indicates that a is in the partial stack above b_0 , $y\langle \text{on}^* \rangle a$ that y is above a , $a\langle \text{on}^* \rangle b$ that a is above the current top b , and $a=y, b=a$ denote equality relations. Fig. 23(a) lists the Boolean valuations of these predicates for all observed block pairs. The goal is to infer logical dependencies among these predicates.

We take $a\langle \text{on}^* \rangle b_0$ as the *target predicate* whose relation to the others we wish to learn. Samples are partitioned into sets S and U (Fig. 23(b)) according to whether $a\langle \text{on}^* \rangle b_0$ holds. The learning task is then to synthesize a classifier formula that separates the samples in S from those in U based on the remaining predicates, thereby capturing the conditions under which $a\langle \text{on}^* \rangle b_0$ holds. After removing overlapping samples between S and U , the learner infers a separating formula φ ; if derived from the reduced S , the resulting invariant takes the form $\varphi \Rightarrow a\langle \text{on}^* \rangle b_0$, and if derived from the reduced U , it takes the form $a\langle \text{on}^* \rangle b_0 \Rightarrow \varphi'$. Fig. 23(c) illustrates the resulting partition $(S - U, U)$.

Since the hypothesis space of possible invariants grows exponentially with the number of atomic predicates, our approach promotes simplicity and generalizability by selecting a *minimal* subset of predicates sufficient for separation. This principle mirrors Occam’s razor in logic learning: among all separating formulas, the most concise one is most likely to generalize across unseen program

a	y	$a\langle on^* \rangle b_0$	$y\langle on^* \rangle a$	$a\langle on^* \rangle b$	$a=y$	$b=a$
x_1	x_2	1	1	0	0	0
x_1	x_3	1	1	0	0	0
x_1	x_4	1	0	0	0	0
x_2	x_3	1	1	0	0	0
x_2	x_4	1	0	0	0	0
x_2	x_1	1	0	0	0	0
x_3	x_1	1	0	1	0	1
x_3	x_2	1	0	1	0	1
x_3	x_4	1	0	1	0	1
x_4	x_1	0	0	0	0	0
x_4	x_2	0	0	0	0	0
x_4	x_3	0	0	0	0	0
x_1	x_1	1	1	0	1	0
x_2	x_2	1	1	0	1	0
x_3	x_3	1	1	1	1	1
x_4	x_4	0	1	0	1	0

(a) Predicates Valuations over all block pairs (a, y) .

$a\langle on^* \rangle b_0$	$y\langle on^* \rangle a$	$a\langle on^* \rangle b$	$a=y$	$b=a$
S	1	0	0	0
S	0	1	0	1
S	1	1	1	1
U	0	0	0	0
U	1	0	1	0

(c) $S - U$, U partition for $a\langle on^* \rangle b_0$ from (b).

$a\langle on^* \rangle b_0$	$y\langle on^* \rangle a$	$a\langle on^* \rangle b$	$a=y$	$b=a$
S	1	0	0	0
S	0	0	0	0
S	0	1	0	1
S	1	0	1	0
S	1	1	1	1
U	0	0	0	0
U	1	0	1	0

(b) S , U partition for $a\langle on^* \rangle b_0$ from (a).

$a\langle on^* \rangle b_0$	$y\langle on^* \rangle a$	$a=y$	$b=a$
S	0	0	1
S	0	1	0
S	0	1	1
S	1	0	0
S	1	0	1
S	1	1	1
U	0	0	0
U	1	1	0

(d) Projection onto separating features (blue) from (c).

Fig. 23. Invariant learning for the Tower task.

states. We associate an integer variable sel_i with each predicate $\Pi_i \in \Omega_{\text{inv}}$ ($0 \leq i < n$). If Π_i is selected as part of the separating classifier, $sel_i = 1$; otherwise, $sel_i = 0$. The learning problem is encoded as the following constraint system:

$$\begin{aligned}
 \varphi_1 : & \bigwedge_{s \in S, u \in U} \bigvee_{0 \leq i < n} (s(\Pi_i) \neq u(\Pi_i) \wedge sel_i = 1) \\
 \varphi_2 : & \bigwedge_{0 \leq i < n} 0 \leq sel_i \leq 1 \\
 \varphi_c : & \min \left(\sum_{0 \leq i < n} sel_i \right)
 \end{aligned} \tag{9}$$

Here, φ_1 enforces separation between satisfying (S) and unsatisfying (U) samples: for every pair (s, u) , there must exist at least one predicate Π_i with differing evaluations and $sel_i = 1$. Constraint φ_2 ensures that each sel_i is binary, and φ_c minimizes the number of selected predicates, yielding a compact, generalizable invariant. The final selection of predicates forming a sufficient classifier is illustrated in Fig. 23(c). A truth table is then constructed over these predicates (Fig. 23(d)), rejecting all U -samples and accepting all remaining states, including those in S . Standard Boolean minimization is applied to derive the classifier's logical structure. The resulting formula is:

$$(\neg y\langle on^* \rangle a \wedge a=y) \vee (y\langle on^* \rangle a \wedge a \neq y) \vee (b=a) \Rightarrow a\langle on^* \rangle b_0.$$

This expression induces three candidate invariants:

$$\forall a, y. y\langle on^* \rangle a \vee a \neq y \vee a\langle on^* \rangle b_0, \quad \forall a, y. \neg y\langle on^* \rangle a \vee a=y \vee a\langle on^* \rangle b_0, \quad \forall a. b=a \Rightarrow a\langle on^* \rangle b_0.$$

Algorithm 7 $\text{REFINECFG}(\mathcal{G}, \mathcal{D}_V, v_0, \varphi)$: split a basic block v_0 of $\mathcal{G}$ on φ , partitioning its demonstration segment in lockstep

```

1: procedure  $\text{REFINECFG}(\mathcal{G}, \mathcal{D}_V, v_0, \varphi)$ 
2:   Let  $v_{\text{in}} \xrightarrow{\varphi_{\text{in}}} v_0 \xrightarrow{\varphi_{v_0}} v_{\text{out}}$  be the unique path through  $v_0$  in  $\mathcal{G}$ 
3:    $V \leftarrow V \setminus \{v_0\} \cup \{v_1, v_2\}$  ▷  $v_1, v_2$  fresh
4:    $E \leftarrow (E \setminus \{(v_{\text{in}}, v_0), (v_0, v_{\text{out}})\}) \cup \{v_{\text{in}} \xrightarrow{\varphi_{\text{in}}} v_1, v_1 \xrightarrow{\varphi} v_2, v_2 \xrightarrow{\varphi_{v_0}} v_{\text{out}}\}$ 
5:   for all  $d \in \mathcal{D}_V(v_0)$  do
6:     Let  $t^*$  be the first timestep in  $d$  with  $d_{t^*} \models \varphi$ 
7:      $\mathcal{D}_V(v_1) \leftarrow \mathcal{D}_V(v_1) \cup \{d_{0:t^*}\}$  ▷ prefix segment, up to  $\varphi$ 
8:      $\mathcal{D}_V(v_2) \leftarrow \mathcal{D}_V(v_2) \cup \{d_{t^*:\cdot}\}$  ▷ suffix segment, from  $\varphi$  onward
9:    $\mathcal{D}_V \leftarrow \mathcal{D}_V \setminus \{v_0 \mapsto \cdot\}$ 
10:  return  $\mathcal{G}, \mathcal{D}_V$ 

```

The first and third are tautological, while the second captures the desired inductive invariant: each block a either has no block y above it (i.e., is a top block) or already belongs to the partial stack above b_0 .

Quantifiers are then added to generalize a and y over all objects, extending the learned classifier to unseen states. To infer all salient relations, we iterate over each predicate Π_i as the target and learn its dependencies on the remaining predicates. For instance, setting $a\langle \text{on}^* \rangle b$ as the target yields:

$$\forall a. a\langle \text{on}^* \rangle b \Rightarrow b=a, \quad \forall a. b=a \Rightarrow a\langle \text{on}^* \rangle b.$$

The first states that b is the current top of the stack, while the second is tautological. Combining these learned relations yields the final inductive invariant:

$$(\forall a, y. \neg y\langle \text{on}^* \rangle a \vee a=y \vee a\langle \text{on}^* \rangle b_0) \wedge (\forall a. a\langle \text{on}^* \rangle b \Rightarrow b=a) \wedge b\langle \text{on}^* \rangle b_0,$$

which compactly characterizes the structural invariant required to verify the synthesized TOWER program.

G Synthesis Algorithms

CFG Refinement by Node Splitting

REFINECFG is exactly the node split shown in Section 2.3: v_0 is replaced by two fresh blocks v_1 and v_2 connected by a new edge labeled φ , with v_0 's former incoming and outgoing edge labels reattached to v_1 and v_2 unchanged. This is precisely the transformation depicted in Section 2.5:

splitting the unknown block $v_?$ on $\text{on}(b_1, b_0)$ yields $\varphi_{\text{pre}} \rightarrow v_1 \xrightarrow{\text{on}(b_1, b_0)} v_2 \rightarrow \varphi_{\text{post}}$.

Demonstration Partitioning (lines 5–8) implements the mechanism described in Section 2.5: every demonstration segment d previously assigned to v_0 is split at the first timestep t^* satisfying φ into a prefix $d_{0:t^*}$, assigned to v_1 , and a suffix $d_{t^*:\cdot}$, assigned to v_2 . CFG refinement and demonstration partitioning therefore proceed in lockstep within a single procedure: $\mathcal{D}_V$ is updated at exactly the same step that $\mathcal{G}$ is, using exactly the predicate φ that triggered the split. For example, $\text{on}(b_1, b_0)$ partitions every demonstration assigned to $v_?$ into the segment before any block lands on b_0 (assigned to v_1) and the segment after (assigned to v_2); a further split of v_2 on $\text{on}(b_2, b_1)$ partitions $\mathcal{D}_V(v_2)$ again, in exactly the same way.

Algorithm 8 QUOTIENT($\mathcal{G}, \mathcal{D}_V$): collapse relationally equivalent consecutive fragments into loops at each nesting level, then recurse into loop bodies

```

1: procedure QUOTIENT( $\mathcal{G}, \mathcal{D}_V$ )                                ▶ Phase 1: quotient the current level
2:    $(u_1, \dots, u_k) \leftarrow$  top-level nodes of  $\mathcal{G}$  in order, with each existing loop treated as a single
   atomic unit
3:   repeat
4:      $found \leftarrow \text{false}$ 
5:     for  $l \leftarrow 1$  to  $\lfloor k/2 \rfloor$  do
6:       for  $i \leftarrow 1$  to  $k - 2l + 1$  do
7:         if  $\exists W(\vec{x}), \sigma_1, \sigma_2$  s.t.  $\llbracket u_i \cdots u_{i+l-1} \rrbracket = W(\vec{x})\sigma_1$  and  $\llbracket u_{i+l} \cdots u_{i+2l-1} \rrbracket = W(\vec{x})\sigma_2$ 
           then
8:            $x \leftarrow \sigma_1(x)$  for each loop-carried  $x$                                 ▶ initialization
9:            $x \leftarrow \sigma_1^{-1}(\sigma_2(x))$  for each loop-carried  $x$                                 ▶ update
10:           $\mathcal{D}_{\text{body}} \leftarrow \{\}$ 
11:          for all  $d \in \mathcal{D}_V$  reaching  $u_i$  do
12:             $\mathcal{D}_{\text{body}} \leftarrow \mathcal{D}_{\text{body}} \cup \text{EXTRACTITERATIONS}(d, W(\vec{x}), x \leftarrow x')$ 
13:             $g(\vec{x}) \leftarrow \text{LOOPGUARDSYNTHESIS}(\mathcal{D}_{\text{body}}, W, \sigma_1, \sigma_2)$ 
14:            Replace  $u_i \cdots u_{i+2l-1}$  in  $\mathcal{G}$  with loop  $v_{\text{loop}} \xrightarrow{g(\vec{x})} v_{\text{body}} \xrightarrow{W(\vec{x})} v_{\text{loop}}$ , exit on
             $\neg g(\vec{x})$ ; update  $(u_1, \dots, u_k)$ 
15:             $\mathcal{D}_V(v_{\text{body}}) \leftarrow \mathcal{D}_{\text{body}}$ ;  $found \leftarrow \text{true}$ ; break
16:          if  $found$  then break
17:   until  $\neg found$                                 ▶ Phase 2: recurse into each loop body at this level
18:   for all loop  $L$  among  $(u_1, \dots, u_k)$  do
19:      $L.\text{body}, \mathcal{D}_V \leftarrow \text{QUOTIENT}(L.\text{body}, \mathcal{D}_V)$ 
20:   return  $\mathcal{G}, \mathcal{D}_V$ 
21:
22: procedure EXTRACTITERATIONS( $d, W(\vec{x}), x \leftarrow x'$ )
23:    $\text{segs} \leftarrow []$ ;  $t \leftarrow 0$ 
24:   while  $\exists t' > t. d_{t'} \models W(\vec{x})$  do
25:     Let  $t'$  be the first such timestep
26:      $\text{segs} \leftarrow \text{segs} \cup [d_{t:t'}]$                                 ▶ one iteration: entry to state establishing  $W$ 
27:      $x \leftarrow x'$ ;  $t \leftarrow t'$ 
28:   return  $\text{segs}$ 

```

Loop Quotienting

Internal binders. A guard formula may contain its own quantifiers, such as the $\forall n$ in $(\forall n. n = b' \vee \neg n \langle \text{on}^* \rangle b') \wedge b' \neq b$. The variable n is bound *within* the formula-letter $g(\underline{b'})$; it does not occupy a word-level position, is not underlined, and is not part of the substitution σ . When two guard-letters are compared for template matching, they are compared as formulas up to alpha-equivalence of their internal binders: $g_1(\underline{b'})$ and $g_2(\underline{b''})$ match the template $g(\underline{x'})$ if they become equal after substituting $b' \mapsto x', b'' \mapsto x'$, and renaming internal bound variables consistently.

Loop body preparation. EXTRACTITERATIONS re-traces every demonstration d reaching the merged fragments from the loop entry. It repeatedly finds the next timestep at which $W(\vec{x})$ holds under the current variable binding (line 24), records the segment as one loop-body iteration (line 26),

Algorithm 9 STRAIGHTLINESYNTHESIZE($\mathcal{D}_v, \varphi$): synthesize a straight-line program from a demonstration segment, preferring candidates that establish φ

```

1: procedure STRAIGHTLINESYNTHESIZE( $\mathcal{D}_v, \varphi$ )
2:    $\pi \leftarrow$  a random sequence of primitive actions with random continuous parameters
3:    $\tau_{\mathcal{D}} \leftarrow$  the state distribution induced by  $\mathcal{D}_v$ 
4:   pool  $\leftarrow []$  ▷ candidates within  $\epsilon$  of the best KL score seen so far
5:   for  $k = 1, \dots, K$  do ▷ MCMC budget
6:      $\pi' \leftarrow \text{MUTATE}(\pi)$  ▷ modify an instruction, change its operands, or reorder instructions
7:      $\pi' \leftarrow \text{CROSSENTROPYOPTIMIZE}(\pi', \tau_{\mathcal{D}})$  ▷ optimize continuous parameters of  $\pi'$  to
       minimize  $\text{KL}(\tau_{\pi'} \parallel \tau_{\mathcal{D}})$ 
8:      $\Delta \leftarrow \text{KL}(\tau_{\pi} \parallel \tau_{\mathcal{D}}) - \text{KL}(\tau_{\pi'} \parallel \tau_{\mathcal{D}})$ 
9:     With probability  $\min(1, e^{\Delta/T_k})$ :  $\pi \leftarrow \pi'$  ▷ MCMC accept;  $T_k$  an annealed temperature
10:    pool  $\leftarrow \{ \rho \in \text{pool} \cup \{ \pi \} \mid \text{KL}(\tau_{\rho} \parallel \tau_{\mathcal{D}}) \leq \min_{\rho' \in \text{pool} \cup \{ \pi \}} \text{KL}(\tau_{\rho'} \parallel \tau_{\mathcal{D}}) + \epsilon \}$  ▷ keep
       all candidates within  $\epsilon$  of the current best
11:    if  $\text{KL}(\tau_{\pi} \parallel \tau_{\mathcal{D}}) < \theta$  then ▷ convergence threshold
12:       $\pi^* \leftarrow \arg \max_{\rho \in \text{pool}} \text{POSTSCORE}(\rho, \mathcal{D}_v, \varphi)$  ▷ among KL-close candidates, prefer
       postcondition success
13:      return  $\pi^*$ , true
14:    return  $\pi$ , false ▷ budget exhausted without converging
15:
16: procedure POSTSCORE( $\pi, \mathcal{D}_v, \varphi$ )
17:   return  $\Pr_{s_0 \sim \mathcal{D}_v} [ \pi \text{ rolled out from } s_0 \text{ reaches a state } s \models \varphi ]$  ▷ fraction of rollouts from
       demonstration-initial states reaching  $\varphi$ 

```

advances the loop-carried variable (line 27), and repeats. This recovers every iteration actually exhibited by the demonstration—possibly more than the two that triggered detection—and the full collection becomes $\mathcal{D}_V(v_{\text{body}})$ (line 15).

Loop guard synthesis (detailed). Each loop-entry state s recovered by EXTRACTITERATIONS contributes one *positive example* (s, b_s) , where b_s is the object assigned to the existentially bound template variable in that iteration, and *local negative examples* (s, b_{neg}) for every other candidate object present in s . Positive examples are pooled across all loop-entry states—the guard must accept the demonstrated choice in every iteration—whereas negative examples remain local. Enumerative synthesis (bottom-up, same strategy as LEARNCLASSIFIER) learns $G_s(\vec{x})$ per state; states with no discriminating predicate contribute $G_s = \text{true}$ and vanish under conjunction. The final guard is $G(\vec{x}) = \bigwedge_s G_s(\vec{x})$ (e.g. Eq. 2). Together with the loop-carried update $x \leftarrow x'$ from Phase 1, this gives the executable form:

$$\text{while } (\exists x'. G(x, x')) \{ \pi_{\text{body}}; x \leftarrow x' \}.$$

Straight-Line Program Synthesis

Combining KL Divergence with Postcondition Satisfaction. Trajectory similarity alone can admit several candidates that resemble the demonstration equally well in distribution but differ in whether they actually establish the block’s postcondition—e.g. a near-miss that releases a block fractionally too early. Rather than folding postcondition satisfaction into a single weighted score with KL divergence, STRAIGHTLINESYNTHESIZE *filters, then ranks*: at every step it maintains pool, the set of all candidates seen so far whose KL-divergence score is within ϵ of the best

score found (line 10). Once some candidate’s score drops below the convergence threshold θ (line 11), `STRAIGHTLINESYNTHESIZE` does not simply return it; instead it selects, from among every candidate currently in pool, the one with the highest `PostScore`—the fraction of rollouts from the demonstration-initial states that actually reach φ (line 12). This way, postcondition satisfaction is only ever used to break ties *among* candidates that are already trajectory-faithful to the demonstration, rather than trading off against trajectory fidelity directly; a candidate far from $\tau_{\mathcal{D}}$ is never preferred merely because it satisfies φ more often. If no candidate is found within budget K , `STRAIGHTLINESYNTHESIZE` reports failure (line 14), which triggers `REFINECFG` in Algorithm 2 for this block.

For example, realizing v_1 in Section 2.5—the block summarized by $\text{on}(b_1, b_0)$, with entry condition φ_{pre} and exit condition $\text{on}(b_1, b_0)$ itself—begins from a random sequence of `Pick/Move/Release` instructions (Fig. 3) and converges, under repeated mutation and parameter optimization against $\mathcal{D}_V(v_1)$, to the sequence of approach, grasp, lift, transport, lower, and release actions for $\pi_{\text{on}(b', b)}$ shown in Fig. 5(b) (Section 2.6); among any KL-close variants of this sequence, `PostScore` prefers the one whose rollouts most reliably leave b_1 actually resting on b_0 , rather than merely passing nearby in trajectory space. When the loop body of Fig. 4 is realized, the same procedure is scored against the pooled per-iteration segments $\mathcal{D}_V(v_{\text{body}})$ from `EXTRACTITERATIONS` (Section 3.3), so the synthesized loop body must match every recovered iteration simultaneously while still being selected for reliably establishing $\text{on}(b', b)$.

Termination: Validate Procedure and FAIL Causes

Each refinement of $\mathcal{G}$ (Algorithm 2, line 14) is followed by a `VALIDATE` call (line 15), which checks whether the refined $\mathcal{G}$ can still yield a task-solving program. Given a segment $d = (s_0, s_1, \dots, s_H)$ and predicate φ , let $i_s(d, \varphi)$ be the first timestep where φ holds and $i_f(d, \varphi)$ the last. For each edge $v_\varphi \xrightarrow{\psi} v_\psi$, `VALIDATE` checks over every $d \in \mathcal{D}_V(v_\varphi) \cup \mathcal{D}_V(v_\psi)$ whether $i_s(d, \psi) \leq i_f(d, \varphi)$ (or $i_s(d, \psi) = 0$ when $\varphi = \varphi_{\text{pre}}$). If this fails, the new block does not meaningfully advance the task beyond its predecessor and the split is rejected, triggering `FAIL`.

The main cause of `FAIL` in practice is `STRAIGHTLINESYNTHESIZE` failing to converge even for a short, single-step demonstration segment: the segment’s state trajectory cannot be matched by any short composition of primitives in $\mathcal{L}$, or the MCMC search becomes trapped. We have observed this rarely.

H Decidability of RoboVerify Verification Conditions

Following [20], we restrict pre/postconditions, loop invariants, and CFG edge labels to a fragment of first-order logic for which wp is guaranteed to stay inside a decidable class, so that no VC ever escapes into a logic we cannot discharge automatically.

Definition H.1 (QFR and AFR). Let R range over the domain’s relation symbols ($\langle \cdot \text{on}^* \rangle$, `Higher`, `Scattered`, and any task-specific relations declared in $\mathcal{L}$). An *atom* is $R(t_1, \dots, t_k)$ or $t_1 = t_2$, for object variables or constants t_i . A *quantifier-free* formula (QFR) is a Boolean combination of atoms. An *alternation-free* formula (AFR) is a QFR formula, or a formula $\forall \bar{u}. \varphi$ or $\exists \bar{u}. \varphi$ with φ QFR; in particular $\text{QFR} \subset \text{AFR}$.

Every pre/postcondition and every loop invariant synthesized by RoboVerify is purely universal, $\forall \bar{u}. \varphi$ —hence AFR—by construction: the enumerative predicate learners of Sections 3.2.1 and 3.3 only ever search over universally quantified templates. The one place an existential appears anywhere in our specification language is a **get**-guarded condition, of the shape

$$B \equiv \exists v. G(v),$$

arising from a $v \leftarrow \text{get}(\exists v. G(v))$ statement (Section 3.4) used either as a loop guard or as the condition of an **if**. Unlike pre/postconditions and invariants, G itself is *not* always quantifier-free in this paper: the Stack guard $G(b') \equiv \forall n. n = b' \vee \neg n \langle \text{on}^* \rangle b'$ is itself universally quantified, so B is AFR (one $\exists$ block wrapping a $\forall$ block, still alternation-free *within the guard itself*) but not QFR. Whether G is QFR turns out to determine the decidability of every VC that reasons about G *failing*—a loop’s exit, or an **if**’s else-branch—though not of reasoning about G *succeeding*, as Section 5.4 shows. No other existential occurs anywhere else in our fragment, in contrast to [20], where alternation originates from *inverting* a function symbol out of a reachability relation: every abstract action in our DSL is an edge *addition*, $\text{put}(b', b)$ or $\text{put}(b', \text{tbl})$, never a value read that needs inverting, so that source of alternation does not arise for us at all.

THEOREM H.2 (CLOSURE UNDER wp). *For every abstract action a of the form $\text{put}(b', b)$ or $\text{put}(b', \text{tbl})$, and every AFR formula Q , $\text{wp}(a, Q)$ is expressible as an AFR formula.*

We establish Theorem 5.2 constructively, by exhibiting, for each relation symbol, an atom-rewriting operator that computes the effect of a directly on that relation’s truth value, without introducing any quantifier. Section 5.2 gives these operators in general; Theorem 5.2 follows because each operator only ever combines existing quantifier-free atoms with $\wedge$, $\vee$, and substitution of the action’s named objects.

I Decidability: EPR Obstruction from Non-QF Guards

When condition (3) of Theorem 5.3 fails, the quantifier structure escapes EPR as follows. For the *preservation* obligation $I \Rightarrow \forall v. G(v) \Rightarrow \text{wp}(S, I)$, writing $I = \forall u. I_0(u)$ and $\text{wp}(S, I) = \forall x. R_0(x, v)$ (AFR by Theorem 5.2), the negation Z3 checks is $(\forall u. I_0(u)) \wedge \exists v. (G(v) \wedge \exists x. \neg R_0(x, v))$. Since $G(v)$ appears un-negated, any internal quantifier in G commutes freely into the leading existential block, yielding $\exists v, x. \forall u. (\dots)$, which is $\exists^* \forall^*$ —still EPR. The obstruction arises at the *exit* obligation $I \wedge \forall v. \neg G(v) \Rightarrow Q$ with $Q = \forall r. Q_0(r)$. If $G(v) = \forall n. G_0(n, v)$ (as in Stack’s guard), then $\neg G(v) = \exists n. \neg G_0(n, v)$, and the negated exit VC becomes

$$\exists r. \forall u, v. \exists n. (I_0(u) \wedge \neg G_0(n, v) \wedge \neg Q_0(r)).$$

The $\exists n$ sits inside $\forall v$, making n functionally dependent on v ; pulling it outward requires Skolemizing n as $f(v)$ —exactly the function symbol EPR disallows. The same obstruction applies to the else-arm of any **if** whose guard contains an internal $\forall$.

J Alignment Lemma and Combined VC

LEMMA J.1 (TRANSITIVE ALIGNMENT). *Let r be a root element and S a finite set of elements each satisfying $|\mathbf{F}[a] - \mathbf{F}[r]| \leq \delta_F$ for every attribute F . Then any two elements $a, a' \in S$ satisfy $|\mathbf{F}[a] - \mathbf{F}[a']| \leq 2\delta_F \leq N_F$.*

PROOF. By induction on $|S|$, maintaining: (A) every $a \in S$ satisfies $|\mathbf{F}[a] - \mathbf{F}[r]| \leq \delta_F$; (B) every pair $a, a' \in S$ satisfies $|\mathbf{F}[a] - \mathbf{F}[a']| \leq 2\delta_F$. Base case ($S = \{r\}$): trivial. Inductive step: add b with $|\mathbf{F}[b] - \mathbf{F}[r]| \leq \delta_F$; (A) holds by assumption. For any existing a : $|\mathbf{F}[b] - \mathbf{F}[a]| \leq |\mathbf{F}[b] - \mathbf{F}[r]| + |\mathbf{F}[r] - \mathbf{F}[a]| \leq 2\delta_F$. $\square$

The alignment VC certifies the per-step hypothesis for each element x newly placed by π_ϕ . For each atom $R(x, y)$ in ϕ , define the *symmetric projection*

$$\widehat{R_{\Pi}}(x, r) := \bigwedge_{F \in \mathcal{F}} |\mathbf{F}[x] - \mathbf{F}[r]| \leq \delta_F, \\ |\mathbf{F}[x] - \mathbf{F}[r]| \leq N_F \in \widehat{R^*}(x, r)$$

Algorithm 10 VERIFIEDSYNTHESIS: the complete synthesis–verification loop of RoboVerify

```

1: procedure VERIFIEDSYNTHESIS( $\mathcal{M}, \mathcal{L}, \mathcal{D}, \varphi_{\text{pre}} \leadsto \varphi_{\text{post}}$ )
2:    $G \leftarrow (\{v?\}, \varphi_{\text{pre}} \rightarrow v? \rightarrow \varphi_{\text{post}}); \quad \mathcal{D}_V \leftarrow \{v? \mapsto \mathcal{D}\}; \quad \text{Pen} \leftarrow \lambda v. \{\}$ 
3:    $\triangleright \text{Pen}(v)$ : accumulated counterexample environments for block  $v$ 
4:    $\mathcal{P}, G \leftarrow \text{SYNTHESIZE}(\mathcal{M}, \mathcal{L}, G, \mathcal{D}_V) \quad \triangleright \text{Alg. 2: CFG refinement + straight-line synthesis}$ 
5:    $I \leftarrow \text{INVINFERENCE}(\mathcal{D}_V) \quad \triangleright \text{Sec. 4: initial invariant from demonstration traces}$ 
6:   while true do  $\triangleright \text{symbolic CEGIS: strengthen } I \text{ or refine } \mathcal{P}$ 
7:      $\text{ok}_s, s_0 \leftarrow \text{SYMBOLICVERIFY}(\mathcal{P}, I, \varphi_{\text{pre}}, \varphi_{\text{post}}) \quad \triangleright \text{Sec. 5.3: WP-based VC discharge}$ 
8:     if  $\text{ok}_s$  then
9:       break
10:    else if  $s_0$  is a loop-VC counterexample then
11:      Extend  $\mathcal{D}_V$  by traces from executing  $\mathcal{P}$  from loop-head state  $s_0$ 
12:    else  $\triangleright \mathcal{P}$  cannot reach  $\varphi_{\text{post}}$  from  $s_0$ 
13:      Request  $\mathcal{D}_{s_0}$  from user  $\triangleright$  demonstrations from  $s_0$ -configurations
14:      Extend  $\mathcal{D}_V$  by traces from  $\mathcal{D}_{s_0}$ 
15:       $\mathcal{P}, G \leftarrow \text{SYNTHESIZE}(\mathcal{M}, \mathcal{L}, G, \mathcal{D}_V)$ 
16:       $I \leftarrow \text{INVINFERENCE}(\mathcal{D}_V) \quad \triangleright \text{re-learn loop invariants from updated } \mathcal{D}_V$ 
17:    while true do  $\triangleright \text{motion CEGIS; symbolic proof holds throughout}$ 
18:       $\text{ok}_m, \text{CEx} \leftarrow \text{MOTIONVERIFY}(\mathcal{P}, I) \triangleright \text{Sec. 5.5: contract realization, frame preservation}$ 
19:      if  $\text{ok}_m$  then
20:        return  $\mathcal{P} \quad \triangleright \text{program is fully verified}$ 
21:      for all  $(v, \mu_k) \in \text{CEx}$  do
22:         $\text{Pen}(v) \leftarrow \text{Pen}(v) \cup \{\mu_k\} \quad \triangleright \text{concrete failure-causing object attribute values}$ 
23:        Apply Alg. 9 to synthesize a straight-line program for block  $v$  under objective (11)

```

retaining two-sided bounds and replacing chain-level N_F with per-step δ_F . The root $r \in \mathcal{O}$ is identified by enumerating $\mathcal{O}$ to find the constant for which $\text{wp}(\pi_\varphi, I) \Rightarrow \forall u. R^*(y, u) \Rightarrow R^*(u, r)$. The combined VC certifying both contract realization and alignment is:

$$A \models \varphi[R_\Pi(x, r) \wedge \overline{R(x, y)} / R(x, y)][\sigma_n]. \quad (10)$$

This is a quantifier-free NRA check; no parameter beyond Definition 5.4 is needed.

K Complete Synthesis–Verification Loop

Symbolic counterexamples. When any VC in $\text{VCgen}(\mathcal{P})$ is invalid, the SMT solver returns a model s_0 that falsifies the respective implication. Such a counterexample serves two roles.

Invariant refinement. If the failure occurs in a loop VC (preservation or exit), s_0 is a concrete loop-entry state that the current invariant admits but exposes its weakness. The execution traces of $\mathcal{P}$ from s_0 are added to $\mathcal{D}_V$, and $\text{INVINFERENCE}(\mathcal{D}_V)$ re-learns from the augmented set. Convergence rests on a key property of the learning algorithm [43]: it never produces an invariant that is true for all possible loop-head states—even unseen ones—but fails to be inductive. Consequently, when the verifier finds s_0 as a counterexample to the current invariant, s_0 ’s execution traces reveal a loop-head state not yet in $\mathcal{D}_V$; adding it forces the next learned invariant to cover a strictly larger set of loop-head states, making monotone progress in the finite space Spec_{inv} (unless the vocabulary is exhausted; Appendix L). Section 6 (RQ2) reports iteration counts empirically.

Program refinement. If the failing VC is the entry condition ($\varphi_{\text{pre}} \Rightarrow \text{wp}(\mathcal{P}, \varphi_{\text{post}})$, whose left-hand side contains no invariant symbol), the failure is not attributable to a weak invariant but to an

uncovered case in the program structure: s_0 witnesses an initial configuration from which the synthesized program does not reach φ_{post} . This gap reveals a demonstration coverage problem: the synthesizer has seen only configurations up to a certain scale and has produced a straight-line or under-looped program that cannot generalize. Consider stacking n blocks from demonstrations of $n = 2$: the synthesizer produces a straight-line program, which the verifier refutes with s_0 containing $n = 3$ blocks. New demonstrations cannot be inferred from s_0 alone—they require the user to perform the task in an s_0 -configuration. The user is shown s_0 (line 14) and asked to provide demonstrations from such configurations ($\mathcal{D}_{s_0}$); these are added to $\mathcal{D}$. With 3-block demonstrations now in evidence, the synthesizer discovers a loop; an execution on $n = 4$ blocks is then added to $\mathcal{D}_V$ so that INVINFERENCE learns an invariant that is truly inductive over all n (Section 6).

Motion counterexamples and penalty-augmented synthesis. The motion verifier returns a counterexample model μ_k recording the concrete object positions that witness failure. μ_k is added to $\text{Pen}(v)$ (line 22), and STRAIGHTLINESYNTHESIZE is re-invoked with the penalized objective

$$\text{KL}(\tau_{\pi'} \parallel \tau_{\mathcal{D}}) + \lambda \cdot |\{\mu_k \in \text{Pen}(v) \mid \pi'(\theta') \text{ fails the motion check at } \mu_k\}|. \quad (11)$$

Since φ_v is unchanged, the symbolic proof remains valid throughout (line 17). $\text{Pen}(v)$ grows monotonically; each μ_k is a Z3 model so no spurious constraints are added.

Limitations. Both loops may fail to terminate when $\mathcal{L}$ lacks predicates needed for the task. In symbolic CEGIS, if no predicate in $\mathcal{L}$ distinguishes the counterexample s_0 from states the invariant already admits, the invariant cannot be strengthened and CEGIS stalls. In motion CEGIS, if no instruction sequence satisfies all environments in $\text{Pen}(v)$, the loop does not terminate. In both cases, accumulated counterexamples are diagnostic: they identify exactly which distinction the current vocabulary cannot express, guiding predicate extension. Concrete examples of both failure modes are given in Appendix L.

L CEGIS Limitation Cases

Symbolic CEGIS stall. INVINFERENCE($\mathcal{D}_V$) learns the strongest invariant expressible in the current predicate basis. If that invariant is not inductive, SYMBOLICVERIFY returns a counterexample s_0 ; its traces are added and INVINFERENCE re-runs. If the newly learned invariant is unchanged—because no predicate in $\mathcal{L}$ distinguishes s_0 from states the invariant already admits—the loop is stuck. The fix requires adding predicates that can express the missing distinction.

Motion CEGIS non-termination. If no instruction sequence can satisfy all environments in $\text{Pen}(v)$, CEM exhausts its budget every iteration. Concretely: block b sits too close to the partially assembled tower; moving b directly sweeps through it. With only Move, no offset avoids the collision—the task requires pushing b clear first, which requires $\text{Scattered}(b) \in \mathcal{L}$. Without it the invariant cannot condition on clearance, the CFG cannot guard on it, and CEM cannot discover the two-phase strategy. The fix: extend $\mathcal{L}$ with Scattered and provide push-then-move demonstrations.

M Extended Related Work

Programmatic and verifiable policies. Programmatic reinforcement learning distills policies into interpretable programs [39, 40], and VIPER extracts decision-tree policies amenable to verification [2]. These methods verify (or merely inspect) a policy against a *fixed, finite* model: the guarantees do not quantify over unbounded object sets, and the policy’s primitive actions are trusted. In contrast, RoboVerify’s symbolic proofs are inductive—a single loop invariant covers arbitrarily many objects—and its primitive actions are untrusted learned controllers whose contracts are independently discharged at the motion level.

Invariant inference and decidable reasoning about reachability. RoboVerify learns quantified loop invariants from execution traces, in the spirit of dynamic invariant detection [10], conjunct-pruning [13], and learning-based frameworks such as ICE [15]; interactive and automated inference of universally quantified invariants for parameterized systems is central to Ivy [29]. Our verification fragment builds directly on effectively-propositional (EPR) reasoning about reachability in linked structures [20, 31]: like [20], we restrict assertions so that weakest preconditions stay within a decidable class, with transitive closure ($\langle on^* \rangle$) playing the role of list reachability. We depart from [20] in where alternation arises—not from inverting function symbols out of a reachability relation, but from quantifiers internal to **get**-guarded conditions—and our occurrence-polarity analysis (Theorem 5.3) characterizes decidability uniformly for loop guards and branch conditions. Moreover, our invariants serve a second role that has no analogue in that line of work: translated by $\widehat{(\cdot)}$, they become the geometric environment description against which continuous motion is verified.

Verified robotics and cyber-physical systems. Formal verification of robot and cyber-physical systems typically starts from a *given* model: differential dynamic logic proofs in KeYmaera X [14], verified controller executables in VeriPhy [4], and reactive synthesis from temporal-logic mission specifications [24]. These frameworks verify or synthesize controllers against hand-written models of the plant and its safety envelope. RoboVerify inverts the workflow: both the program and the model-side artifacts (relational summaries, invariants, geometric constraints) are inferred from demonstrations, and verification is used to police the boundary between the learned symbolic abstraction and the learned continuous controllers. The two approaches are complementary—our motion-level checks certify discrete footprint and collision properties of nominal trajectories, while hybrid-systems tools address continuous-time dynamics that we abstract away.